



Software-Architekturen

Prof. Dr. Thomas Fuchß

Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
**Informatik und
Wirtschaftsinformatik**

- **Software-Engineering**
- **Was ist Architektur?**
- **Wie beschreibt man Architektur?**
- **Funktionale und nicht-funktionale Anforderungen**
- **Die Aufgaben des Architekten: Wählen, Bewerten und Testen**
- **Referenzarchitekturen und Designprinzipien**
- **Layers – Adapters – Onions**
- **Blackboard und KI**
- **Pipes and Filters**
- **Middleware-Konzepte zwischen gestern, heute und morgen**
- **Patterns für den Business Layer**
- **Zwischen Architektur und Design**

Die Vorlesung basiert im Wesentlichen auf Literatur und Materialien aus folgenden Quellen.

- Gorton, Ian:
Essential Software Architecture, 2. ed. Springer, 2011.
- Clements, P.; Bass, L. and Kazman, R.:
Software Architecture in Practice, 2. ed. Addison-Wesley, 2003.
- Vogel, O.; et al.:
Software-Architektur: Grundlagen – Konzepte – Praxis, 2. Auflage. Spektrum, 2009.
- Avgeriou, P.; et al. (editors):
Relating Software Requirements and Architectures. Springer, 2011.
- Larman, Craig:
Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, 3. ed. Prentice Hall, 2004.
- Sommerville, Ian:
Software Engineering, 9. Auflage. Pearson Studium, 2012.
- Fowler, M.:
Patterns of Enterprise Application Architecture. Addison-Wesley, 2003.
- Richardson, C.:
Microservices Patterns: With Examples in Java. Manning, 2019.

- Heineman, G. T. and Councill, W. T.:
Component-Based Software Engineering: Putting the Pieces Together. Addison-Wesley, 2001.
- Gamma, Erich et al.:
Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1995.
- Fowler, Martin: Analysemuster:
Wiederverwendbare Objektmodelle. Addison-Wesley, 1999.
- Pattern-Oriented Software Architecture, Volume 1:
Buschmann, Frank:
A System of Patterns. John Wiley & Sons, 1996.
- Pattern-Oriented Software Architecture, Volume 2:
Schmidt, Douglas C.:
Patterns for Concurrent and Networked Objects. John Wiley & Sons, 2000.
- Pattern-Oriented Software Architecture, Volume 3:
Kircher, M. and Prashant, J.:
Patterns for Resource Management. John Wiley & Sons, 2004.
- Pattern-Oriented Software Architecture, Volume 4:
Buschmann, Frank; Henney, Kevlin; and Schmidt, Douglas C.:
A Pattern Language for Distributed Computing. John Wiley & Sons, 2007.
- Vernon, V.:
Domain-Driven Design kompakt. dpunkt.verlag, 2017.

- Reussner, R. und Hasselbring, W. (Hrsg.):
Handbuch der Software-Architektur. dpunkt, 2006.
- Goll, J. und Dausmann, M.:
Architektur- und Entwurfsmuster der Softwaretechnik. Springer Vieweg, 2013.
- Horn, E. und Reinke, Th.:
Softwarearchitektur und Softwarebauelemente. Hanser, 2002.
- Andresen, A.:
Komponentenbasierte Softwareentwicklung mit MDA und UML 2 und XML, 2. Auflage. Hanser, 2004.
- Rupp, C., Queins, S. und Zengler, B.:
UML 2 glasklar: Praxiswissen für die UML-Modellierung, 3. Auflage. Hanser, 2007.
- OMG Object Management Group:
Unified Modeling Language: Infrastructure / Superstructure (2.4.1). OMG, 2011.
- OMG Object Management Group:
Unified Modeling Language (2.5). OMG, 2013.



Software-Architekturen

—

Software-Engineering

Prof. Dr. Thomas Fuchß

Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
Informatik und
Wirtschaftsinformatik

**Die Technologie- und Management-Disziplin für
die systematische Erstellung und Wartung von
Software.**

+IKA Was ist Software-Engineering?

„Software-Engineering zielt auf die ingenieurmäßige
Entwicklung, Wartung, Anpassung und Weiterentwicklung
großer Software-Systeme
unter Verwendung bewährter, systematischer Vorgehensweisen,
Prinzipien, Methoden und Werkzeuge hin.“

[M. Broy und D. Rombach.

Software Engineering: Wurzeln, Stand und Perspektiven.

Informatik Spektrum 25(6), Springer-Verlag, Heidelberg, 2002.]

Zur Beherrschung dieser Aufgaben ist ein systematisches Vorgehen unabdingbar.

“Engineers, in general, design and build artifacts based on theories and methods, while working under financial and organizational constraints.”

[Ian Sommerville]

- Systematik im Prozess
- Systematik in der Analyse
- Systematik im Design
- Systematik im Testen
- ...

**Verantwortungsbewusstsein
gegenüber
Auftraggeber, Nutzer, Team, ...!**

Ein Entwicklungsprozess definiert „wer tut was, wann und wie“, um das vorgegebene Ziel zu erreichen. Ein effektiver Prozess stellt Richtlinien bereit, die eine effiziente Entwicklung ermöglichen. Er reduziert die Risiken und erhöht die Vorhersehbarkeit.

- **Technologie:**

Der Prozess muss auf der Technologie beruhen, die heute verfügbar ist. (Sprachen, Betriebssysteme, Entwicklungsumgebungen, Netzwerke, ...)

- **Werkzeuge:**

Der Prozess ist werkzeugunterstützt. Werkzeuge und Prozesse entstehen parallel. Der Prozess unterstützt die Entwicklung des Werkzeugs und umgekehrt.

**Allen modernen Prozessmodellen
gemeinsam ist ein agiles, iteratives und
risikoorientiertes Vorgehen.**

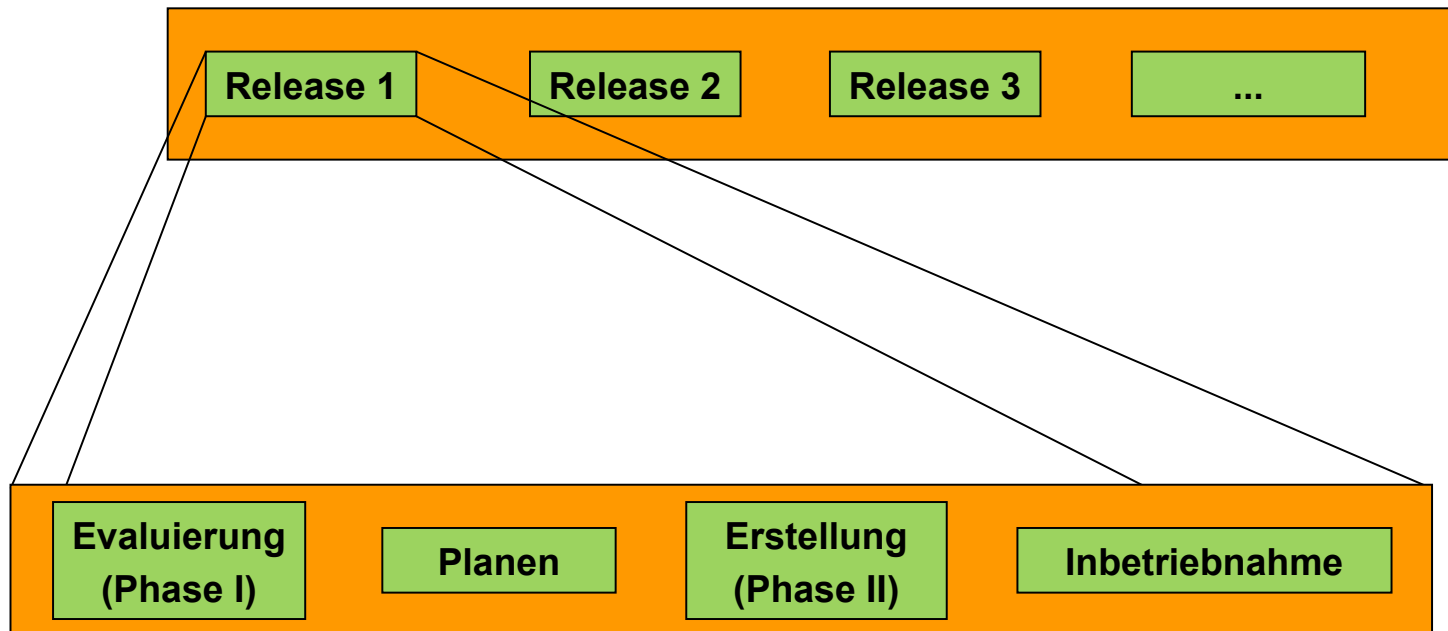
+IKA Vorteile eines iterativen, agilen Vorgehens

- Neu auftretende Nutzeranforderungen können integriert werden.
(Erste Erfahrungen mit lauffähigen Zwischenprodukten.)
- Anforderungen können im Laufe der Versionen verfeinert werden.
- Der frühe Einsatz eines Kernsystems beim Auftraggeber ist möglich.
- Der Auftraggeber kann in die Entscheidung über die Versionsreihenfolge einbezogen werden.
(Wichtige und kritische Aspekte müssen zuerst realisiert werden.)

Erst ein iteratives Vorgehen macht eine strukturierte, agile Software-Entwicklung möglich.

[Larman, Craig: Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, 2. ed. Prentice Hall, 2002.]

In vier Schritten zum Release



CI/CD (Continuous Integration and Delivery/Deployment)

- **Continuous Integration (CI):**

Man arbeitet an einer gemeinsamen Code-Basis und verfügt über einen eigenen Feature-Branche. Änderungen werden in regelmäßig (kurzen) Abständen per Pull-Request in die Code-Basis integriert.

- Automatisches Bauen
- Automatisches Testen
- Abgesichert durch ein Quality Gate

- **Pull-Request:**

Der Pull-Request entspricht einer Inversion of Control. Änderungen am Main-Branch kann man nicht selbst durchführen, sondern sie müssen von diesem gezogen werden. Hierdurch kann sichergestellt werden, dass nur geprüfter Code integriert wird..

CI/CD (Continuous Integration and Delivery/Deployment)

- **Continuous Delivery (CD):**
Die Software ist jederzeit auslieferbar. Die Bereitstellung (Deployment) erfolgt aber weiterhin nicht automatisch und ist im Allgemeinen verbunden mit einer manuellen Freigabe der Software.
- **Continuous Deployment (CD):**
Nach einem erfolgreichen Build erfolgt ein automatisiertes Deployment. Eine manuelle Freigabe entfällt.

+IKA Continuous Deployment – Zielplattformen

- **Container- und Orchestrierungsplattformen (CaaS):**

Man stellt Container-Images bereit und nutzt Orchestrierungsdienste, um diese zu starten, zu stoppen, usw.

Kubernetes, Amazon Elastic Container Service (ECS)

- **Cloud-Plattformen (PaaS):**

Man nutzt eine definierte Plattform, die den Betrieb der Anwendung übernimmt und regelt.

Azure, Google Cloud Platform, ...

- **Hosting-Plattformen (SaaS):**

Man nutzt vorgefertigte Plattformen und stellt lediglich den Code der Anwendung bereit. Die Verwaltung der Server entfällt.

Netlify, Vercel, ...

+IKA Continuous Deployment – Zielplattformen

- **Server und Virtuelle Maschinen (IaaS):**

Man nutzt lediglich vordefinierte Zielsysteme, die typischerweise in der Form virtueller Maschinen (Linux, Windows, ...) bereitgestellt werden.

- VMware:
- Amazon Elastic Compute Cloud (EC2)

- **App Stores und Distributionsplattformen:**

Man nutzt vorgefertigte und etablierte Distributionswege.

**Der CI-Prozess sichert die Code-Qualität.
Der CD-Prozess garantiert die Auslieferung bzw.
Bereitstellung des korrekten Codes.**

+HKA Typischer CI/CD-Cycle

1) Pull-Request:

Code-Änderungen erfolgen im Feature-Branch und werden per Pull-Request eingereicht.

2) Die CI-Pipeline startet:

Checkout → Build → Unit- und Integrationstest sowie Code-Analyse.

3) Quality Gate:

Die Ergebnisse der Tests und Analysen werden bewertet.

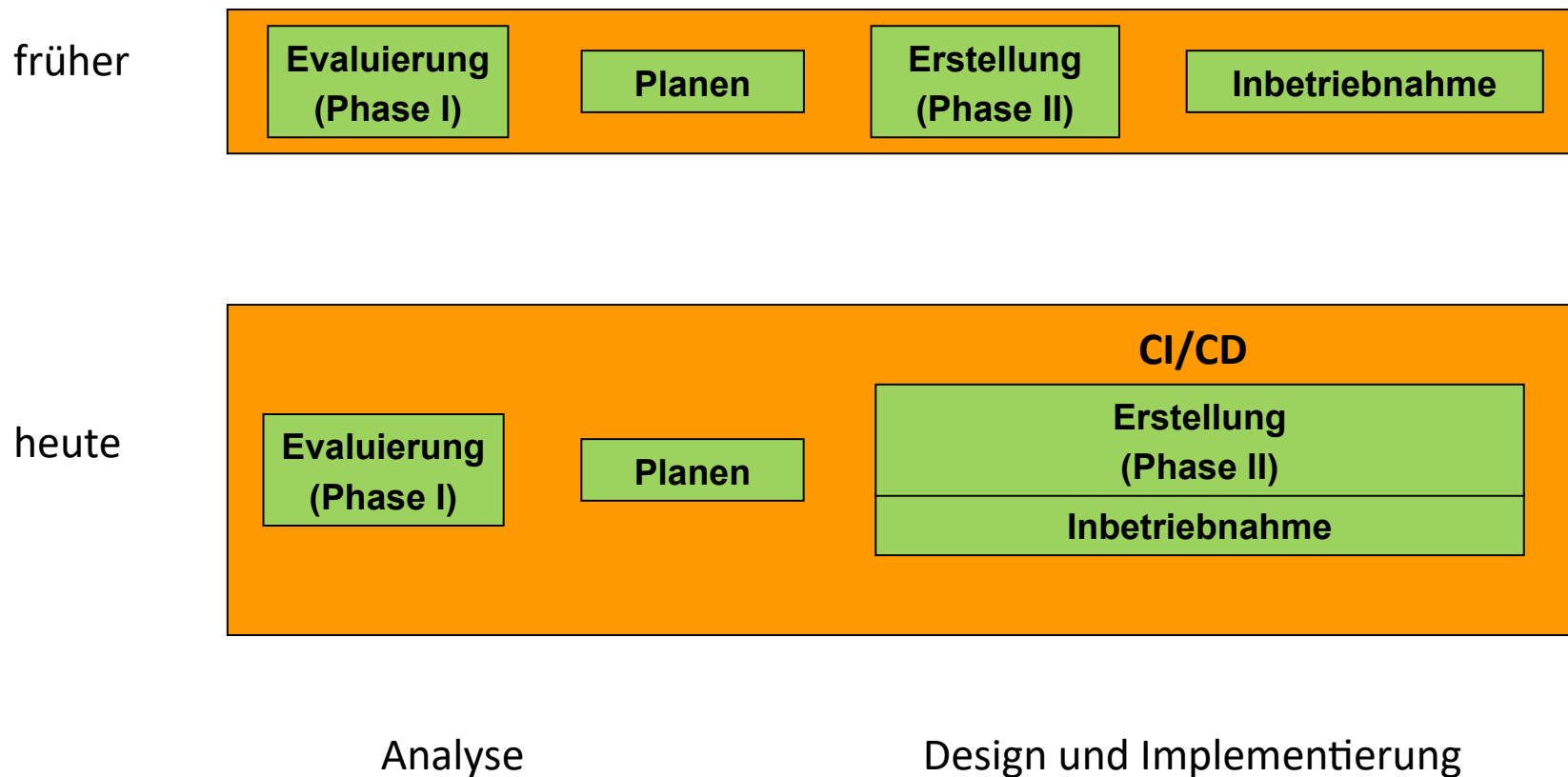
4) Merge und Container Build:

Nach bestandener Kontrolle wird der Pull-Request genehmigt und der Code in den Main-Branch gemergt. Das Image wird gebaut und versioniert.

5) Continuous Delivery / Deployment

Das Deployment-Manifest wird aktualisiert und referenziert das neue Image. Das Image wird in eine Registry gepusht und die Zielplattform z.B. Kubernetes zieht es dann selbstständig.

Und dennoch bleiben die zentralen Herausforderungen gleich!



+IKA Der grobe Ablauf

- **Evaluierung:**
Anforderungen (Requirements) definieren, Prototypen erstellen und erste Iterationen durchführen, bis man die Komplexität der Aufgabe einschätzen kann.
Ziel: Probleme verstehen, Planungssicherheit schaffen.
Primäre Architekturentscheidungen treffen!
- **Planen:**
Zeit, Budget, Personal, usw. anpassen, Backlogs füllen.
- **Erstellung und Inbetriebnahme (CI/CD):**
Das eigentliche Produkt realisieren, neue Anforderungen integrieren, Fehler beseitigen, ...

+IKA Die Herausforderungen

- **Frühe Phasen:**

Architektur konsolidieren!

In den ersten Iterationen der eigentlichen Entwicklung sollten die zentralen, domänenspezifischen Use Cases umgesetzt werden, um eventuell falsche Architekturentscheidungen zu identifizieren und zu korrigieren.

- **Späte Phasen:**

Architektur bleibt stabil!

Die späteren Iterationen konzentrieren sich auf die Vervollständigung der Funktionalität. Dies sollte die Architektur nur unwesentlich beeinflussen.

- Schnittstellen werden erweitert.
- Neue Komponenten kommen hinzu.

Analyse heißt:

Das Problem verstehen und beschreiben, sodass ein korrekter Entwurf möglich wird.
Hierzu wird ein Analysemodell erstellt.

- Use-Case-Diagramme und Activity-Diagramme
- Klassendiagramme und (System-)Sequenz-Diagramme
- ...

Beschrieben werden:

- Die zentralen Anforderungen
- Die zentralen Abläufe
- Die zentralen Konzepte und Objekte
- Die zentralen Ereignisse und Operationen

Oder kurz:

Welche Schnittstellen hat das System?

- Was sind die zusammenhängenden funktionalen Einheiten?
- Welche Schnittstellen gibt es?
- Welche Operationen werden an den Schnittstellen gebraucht?
- Was ist deren Aufgabe?
- Welche Konzepte sind relevant und wie hängen sie voneinander ab?

Design heißt:

Die Lösung beschreiben, sodass die während der Analyse identifizierten Anforderungen, im Sinne des Kunden (der Stakeholder) umgesetzt werden.

Dies umfasst:

- **Systementwurf (oftmals das Bindeglied zwischen Analyse und Design)**
 - Aufbrechen des Systems in Teilsysteme
 - Strategien für Steuerung und Datenhaltung

Primäre Architekturentscheidungen treffen!

- **Objektentwurf (Design)**
 - Anreichern des Analysemodells
 - Umsetzung der Strategien des Systementwurfs
 - Implementierungsentscheidungen treffen
 - Algorithmen entwerfen

Oder kurz:

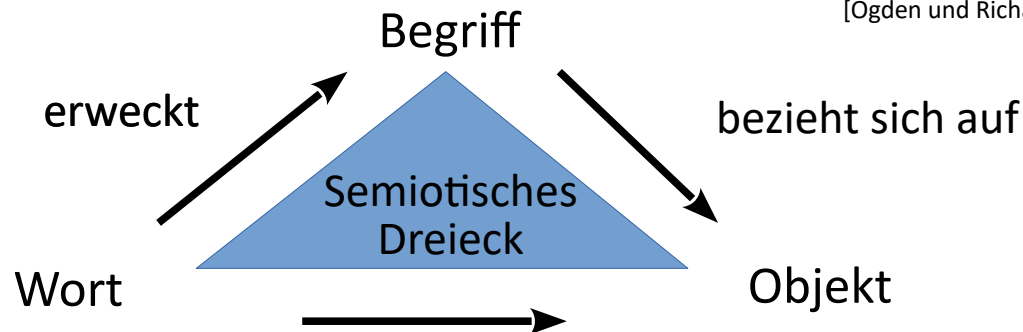
Wie wird die Aufgabe (Schnittstelle) realisiert?

- Architektur: Das System organisieren und die Zugriffsmöglichkeiten definieren.
- Bedienkonzept: Das Erscheinungsbild der Schnittstelle präzisieren.
- **Verantwortlichkeiten zuweisen (der eigentliche Entwurf)**
 - **Verantwortlichkeiten, etwas zu tun (selbst tun oder delegieren)**
 - **Verantwortlichkeiten, etwas zu wissen (speichern, berechnen, assoziieren)**
- Reorganisation der Objekte

Die Operationen der Schnittstelle entwerfen.

+IKA Systematik im Problem

[Ogden und Richards: The Meaning of Meaning, 1923.]



- **Wort:**
Zeichen, Symbole, Modelle, ...

Die Beschreibung von etwas.

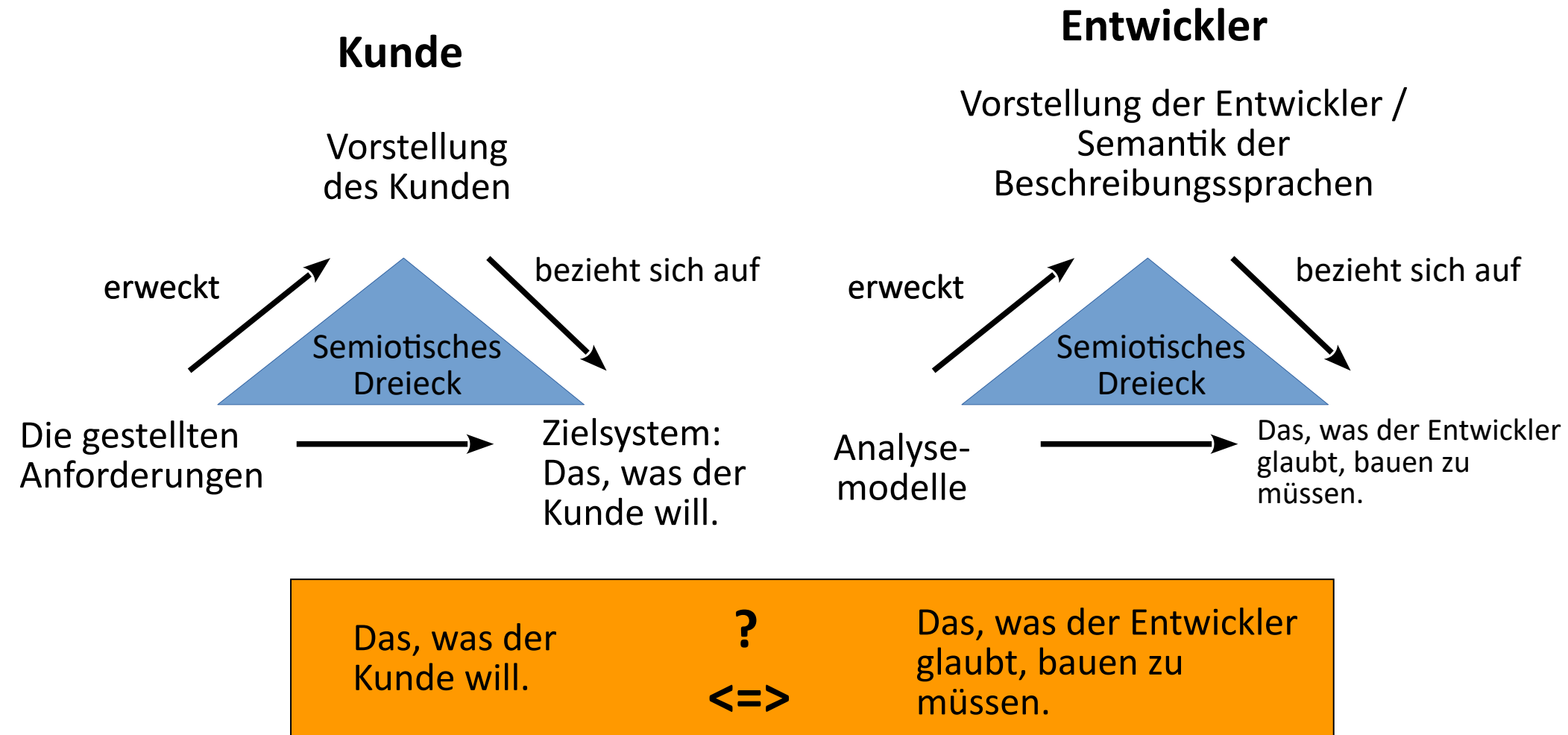
- **Objekt:**
Ding, Realität, ...

Das, was man wirklich will.

- **Begriff:**
Denkeinheit, Semantik, ...

Was steckt dahinter?

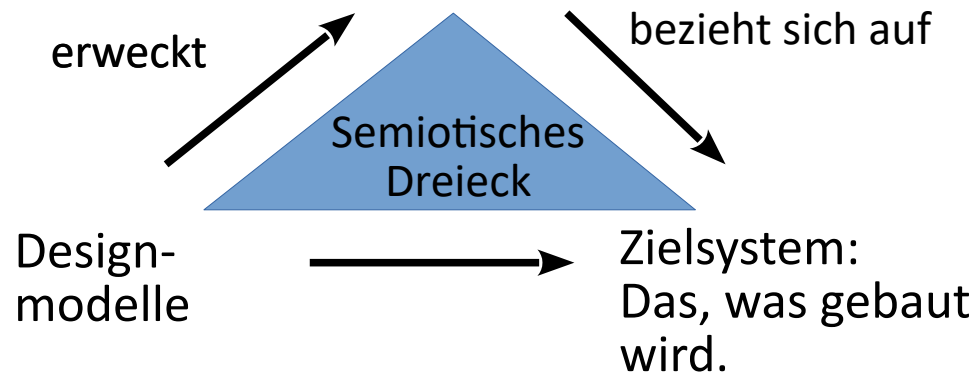
+IKA Systematik im Problem (Analyse)



+HKA Systematik im Problem (Design)

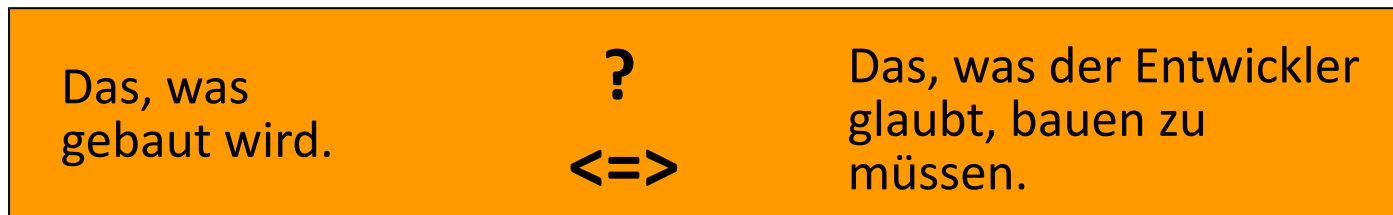
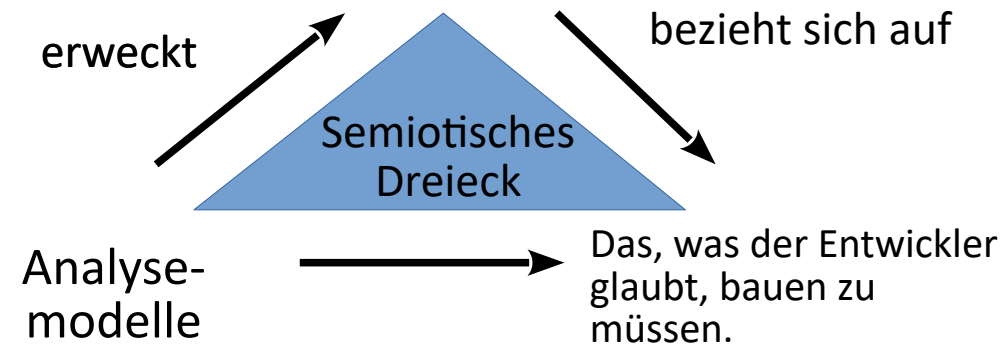
Entwickler

Vorstellung der Entwickler /
Semantik der
Beschreibungssprachen

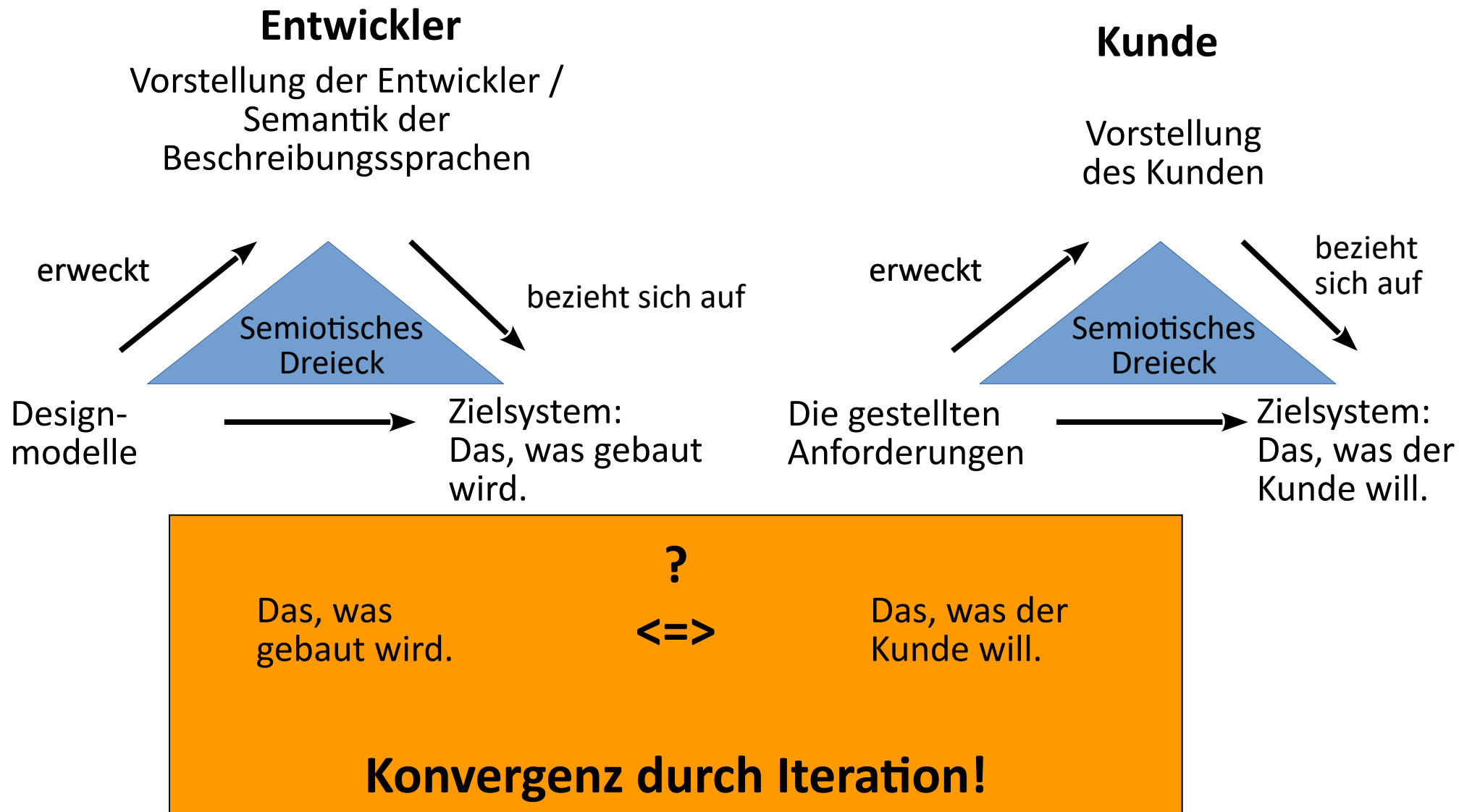


Entwickler

Vorstellung der Entwickler /
Semantik der
Beschreibungssprachen



+IKA Systematik im Problem (Lösung)





Software-Architekturen

—

Was ist Architektur?

Prof. Dr. Thomas Fuchß

Hochschule Karlsruhe

University of
Applied Sciences

Fakultät für

**Informatik und
Wirtschaftsinformatik**

Architektur ist das zentrale Glied in dieser Systematik des Vorgehens.

+IKA Was ist Architektur?



- **Salisbury Cathedral**
„one of the leading examples of Early English architecture” [wiki]
- **Stonehenge**
„the most architecturally sophisticated prehistoric stone circle in the world” [UNESCO. World Heritage List, 373. whc.unesco.org]

+IKA Was ist Architektur?

„[...], but only perfect proportions and colors in a well-balanced harmony deserve that title of honor: beautiful. Just the same is true in architecture. **Only perfect harmony in its technical functions as well as in its proportions can result in beauty.** That makes our task so manifold and complex.

**[Walter Gropius: Architecture at Harvard University.
In Architectural Record, 1937.]
(online – archrecord.construction.com)**

[IEEE-SA Standards Board. IEEE Std 1471-2000 – IEEE Recommended Practice for Architectural Description of Software-Intensive Systems. IEEE Computer Society, 2000.]

Eine systematische Betrachtung:

- **Architecture:**
The fundamental organization of a system embodied in its components, their relationships to each other, and to the environment, and the principles guiding its design and evolution.
- **Architecting:**
The activities of defining, documenting, maintaining, improving, and certifying proper implementation of an architecture.
- **Architect:**
The person, team, or organization responsible for systems architecture.
- **Architectural Description (AD):**
A collection of products to document an architecture.

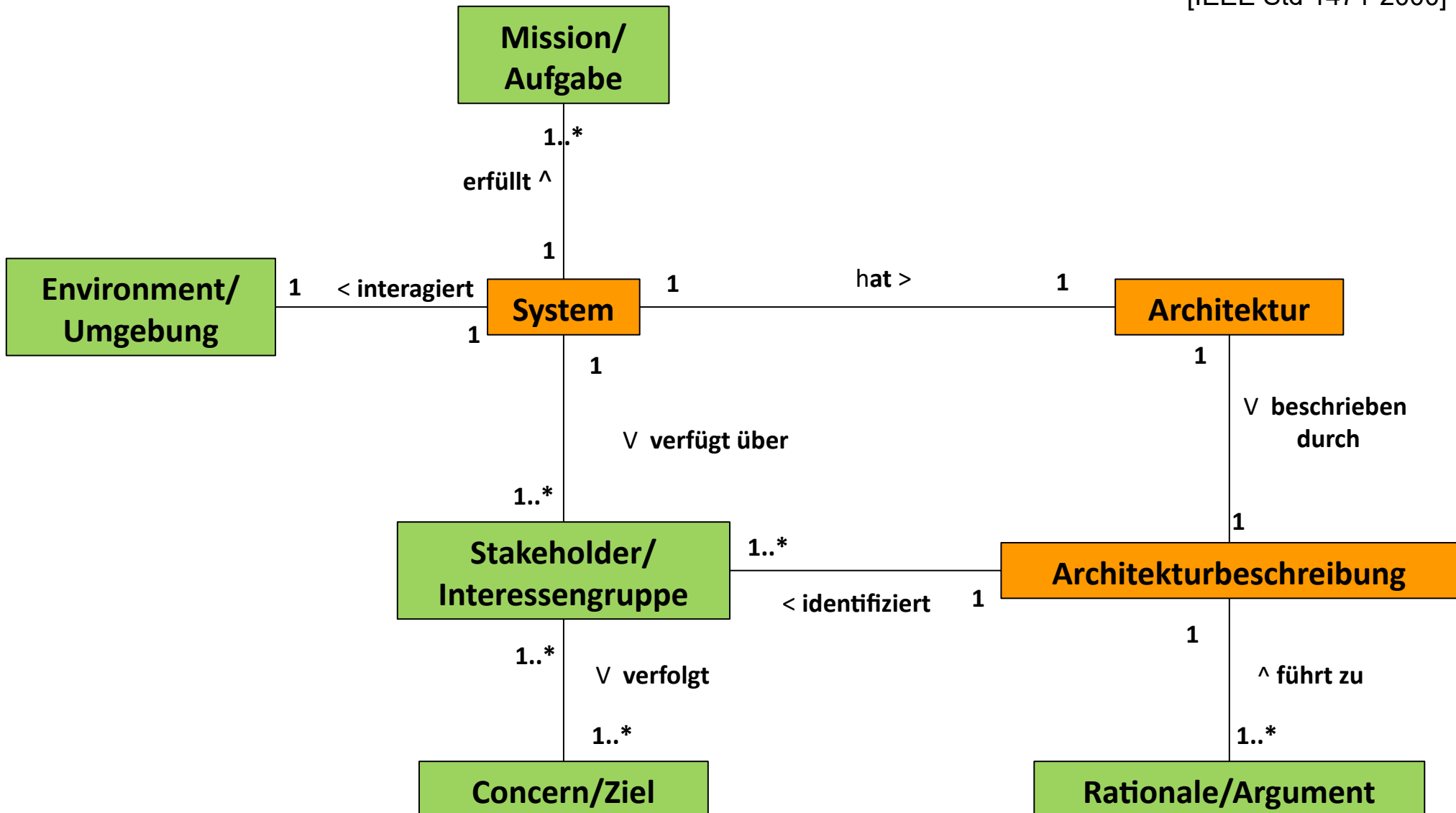
+IKA Was ist Architektur?

[IEEE-SA Standards Board. IEEE Std 1471-2000 – IEEE Recommended Practice for Architectural Description of Software-Intensive Systems. IEEE Computer Society, 2000.]

- **System:**
A collection of components organized to accomplish a specific function or set of functions.
- **System Stakeholder:**
An individual, team, or organization (or classes thereof) with interests in, or concerns relative to, a system.
 - Designer,
 - Entwickler,
 - Projektmanager,
 - Produktmanager,
 - Vertriebsmanager,
 - Kunde, ...



Architekt



+IKA Was ist Architektur?

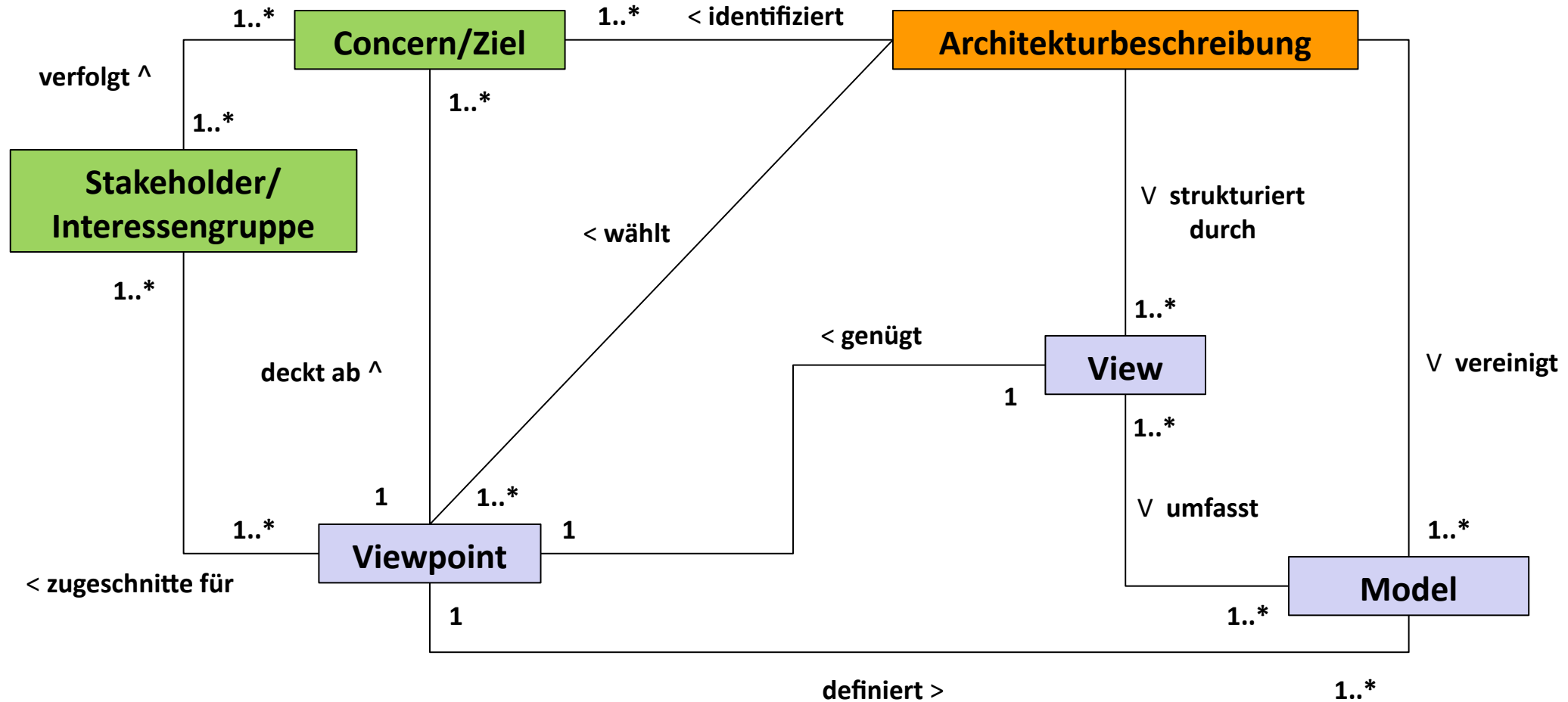
[IEEE-SA Standards Board. IEEE Std 1471-2000 – IEEE Recommended Practice for Architectural Description of Software-Intensive Systems. IEEE Computer Society, 2000.]

- **View:**

A representation of a whole system from the perspective of a related set of concerns.

- **Viewpoint:**

A specification of the conventions for constructing and using a view. A pattern or template from which to develop individual views by establishing the purposes and audience for a view and the techniques for its creation and analysis.



Concerns sind vielfältig und abhängig von der jeweiligen Interessensgruppe, aber elementar für die entstehende Architektur.

Concerns sind:

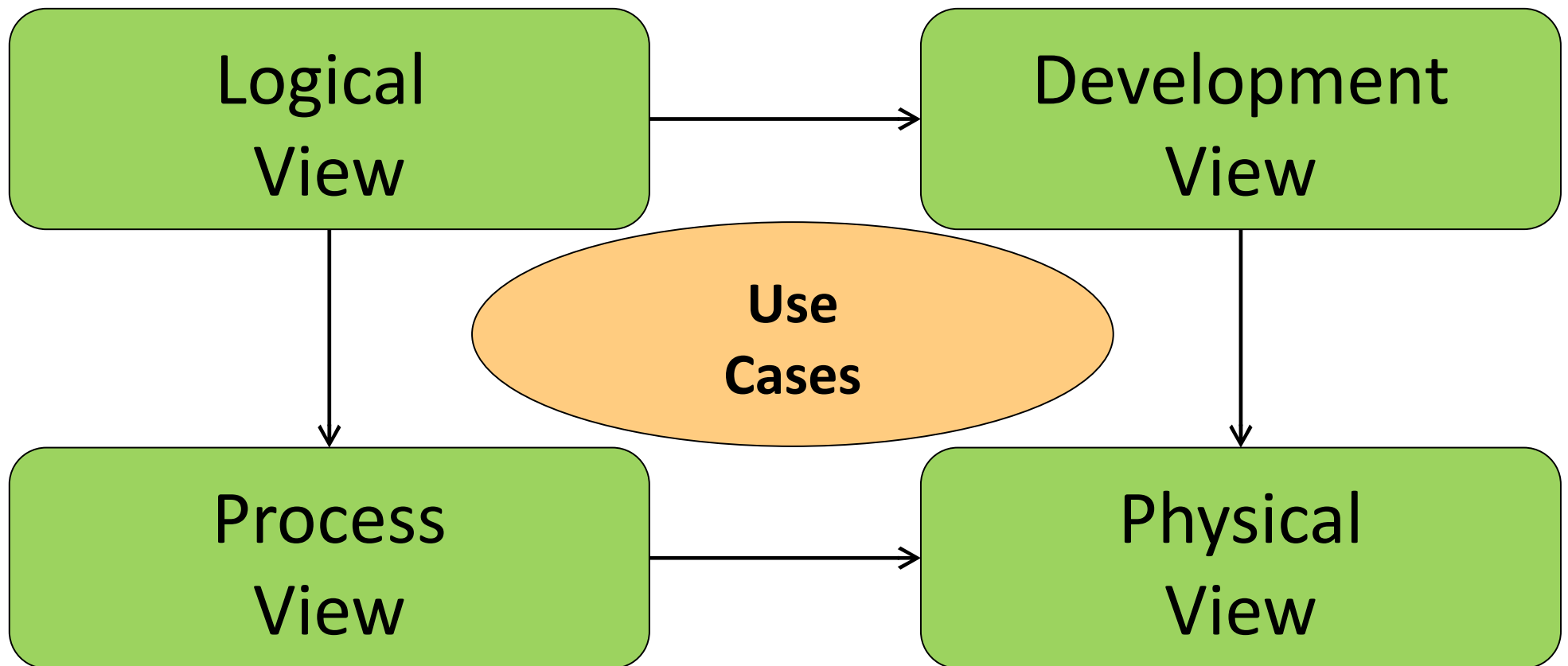
- Zentrale Anforderungen,
- Unabdingbare Constraints,
- Lediglich Absichten,
- Vage Ideen



Oftmals sind diese Dinge, die ein Stakeholder bewegen, nur schwer zu entdecken und doch entscheidend für den Erfolg des Projekts.

+HKA 4+1 View Model

[Philippe Kruchten: Architectural Blueprints – The “4+1” View Model of Software Architecture. In IEEE Software 12 (6),1995.]



+HKA 4+1 View Model

Jede(r) View(point) beschreibt die Architektur aus einer anderen Sicht.

- **Die Logical View :**
Sie beschreibt Strukturen der Architektur aus Anwendersicht.

Modelliert werden die Zusammenhänge zwischen den zentralen Bausteinen. Wie diese **strukturiert** und organisiert sind. Wie sie sich aus kleineren zusammensetzen, wie sie miteinander verbunden sind und **zusammenarbeiten**, um die funktionalen Anforderungen, die an das System gestellt werden, zu erfüllen.

Class Diagrams, Sequence Diagrams, Component Diagrams, usw.

Jede(r) View(point) beschreibt die Architektur aus einer anderen Sicht.

- **Die Process View:**
Sie beschreibt die Abläufe innerhalb der Architektur.

Im Fokus stehen Nebenläufigkeit und die Verteilung der Aufgaben auf selbständig agierende Entitäten (vom einzelnen Thread bis zum komplexen Server) und deren **Kommunikation und Synchronisation**. Adressiert werden dabei nicht-funktionale Anforderungen wie Performance, Verfügbarkeit und Fehlertoleranz.

Activity Diagrams, Interaction Overview Diagrams, Sequence Diagrams, Composite Structure Diagrams, usw.

Jede(r) View(point) beschreibt die Architektur aus einer anderen Sicht.

- **Die Physical View:**
Sie beschreibt die reale (physische) Struktur.

Im Fokus steht die Verteilung der realen „Baugruppen“ und deren Aufgaben auf die realen Hardware-Komponenten (Knoten).
Adressiert werden insbesondere nicht-funktionale Anforderungen wie Skalierbarkeit, Ausfallsicherheit und Performance.

Deployment Diagrams

Jede(r) View(point) beschreibt die Architektur aus einer anderen Sicht.

- **Die Development View:**
Sie beschreibt die Verteilung der zu entwickelnden Software-Elemente (Pakete/Module/Bibliotheken) aus Entwicklersicht.

Im Vordergrund steht die Strukturierung des Systems aus Sicht der Entwickler und dessen Konfigurationsmanagement. Welche Bibliothek stellt welche Schnittstelle bereit? Wie feingranular sind die einzelnen Bibliotheken? Welches Paket beinhaltet welche Unterpakete, usw.?

Deployment Diagrams, Package Diagrams, Class Diagrams, usw.

Jede(r) View(point) beschreibt die Architektur aus einer anderen Sicht.

- **Die Use Case View:**
Sie beschreibt die funktionalen Anforderungen an das System mittels Use Cases und Szenarien.

Im Vordergrund steht die Beschreibung von Abläufen, die besonders viel Information zur Architektur des Systems beitragen. Aufgrund der integralen Sichtweise mit dem Fokus auf die wichtigsten Bedürfnisse der Anwender sind diese Use Cases das Maß, an dem sich die Architektur orientieren muss.

Use Diagrams, Activity Diagrams, usw.

+IKA Ist dies noch modern?

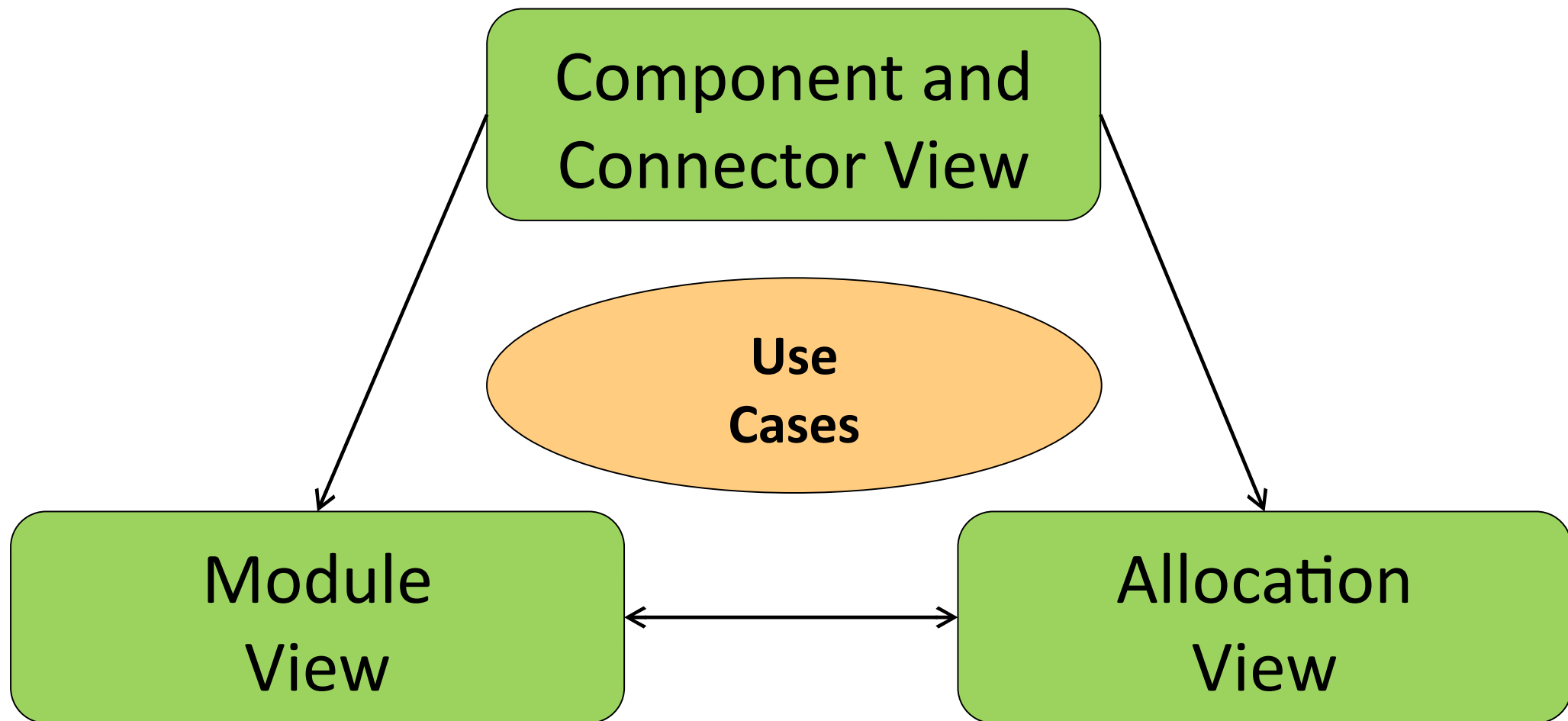
Oftmals ist eine Abgrenzung der Betrachtung der unterschiedlichen Modelle (Viewpoints) und der damit verbundenen Zuordnung zu speziellen Stakeholder-Gruppen nur bedingt möglich.

**Insbesondere stellt sich die Frage:
Wie trennt man logische Sicht, Entwicklersicht und
Prozesssicht?**

+HKA 3+1 View Model

Clements, P. et. al., editors. Documenting Software Architectures: Views and Beyond (2nd Edition). Addison-Wesley, 2010

Christensen, H.B; Corry, A. and Hansen, K.M. An Approach to Software Architecture Description Using UML: Revision 2.0



Component and Connector View

- **Beschrieben wird, was das System „macht“, d. h. die dynamischen Abläufe (Funktionen, Daten- und Kontrollfluss).**

Komponenten:

Objekte, Prozesse, Threads, usw. Sie repräsentieren die für die Erbringung der Funktionalität verantwortlichen Teile des Systems.

Konnektoren:

Shared Memory, Sockets, JDBC, ESB, usw. Sie beschreiben, wie die Komponenten miteinander verbunden sind, auf welche Art und Weise Kontrollfluss und Daten von einer Komponente zur anderen übergehen.

- **Stakeholder:**
Architekten, Entwickler, Kunden, Anwender

Module View

- Beschrieben werden Strukturen, Klassen, Interfaces, die Pakete, denen sie zugeordnet sind und die Beziehungen untereinander...

Im Zentrum der Beschreibung steht die Umsetzung der Funktionalität, deren Zerlegung und Verteilung.

Die Modularisierung der Software

- Stakeholder:
Architekten, Entwickler

Allocation View

- **Beschrieben werden Bibliotheken, deren Konfigurationsmanagement, Deployment bis hin zur Zuordnung auf die einzelnen Entwicklungsteams.**
 - **Deployment:**
Zuordnung von Software-Komponenten zu Hardware.
 - **Implementierung:**
Zuordnung von Software-Komponenten zu Dateistrukturen (im Rahmen der Entwicklung, Konfiguration und Integration).
 - **Arbeitspakete:**
Zuordnung von Software-Komponenten zu Entwicklungsteams.

Allocation View

Im Zentrum der Beschreibung steht die Software und ihre Beziehung zur realen Entwicklungs- und Ausführungsumgebung.

- **Stakeholder:**
Architekten, Entwickler, Betreiber, Kunden, Anwender, ...

Alle

+IKA Was ist Architektur? Essentials

Die Architektur eines Softwaresystems ist ein Bauplan für die Organisation des Gesamtsystems vom Entwurf über die Entwicklung bis zum Betrieb.

Die Aufgabe der Architektur ist es, die signifikanten (hervorstechenden) statischen und dynamischen Aspekte des Gesamtsystems zu erfassen und zu organisieren, d. h. die Struktur der Subsysteme und Komponenten, deren Interfaces, Verbindungen, Interaktion, Verantwortlichkeiten und wie sie zusammenarbeiten.

Hierzu gehören Aspekte wie:

- Verwendung,
- Leistung,
- Technologie,
- Realisierung,
- Konfiguration, ...

**Auch die Beschreibung der Architektur ist strukturiert.
Unterschiedliche Views adressieren unterschiedliche
Gesichtspunkte:**

Zentrale Gesichtspunkte sind:

- Modularisierung und Struktur,
- Modularisierung und Kommunikation,
- Modularisierung und Deployment (Bereitstellung),
Konfiguration, Integration und Arbeitsteilung



Software-Architekturen

—

Wie beschreibt man Architektur?

Prof. Dr. Thomas Fuchß

Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
Informatik und
Wirtschaftsinformatik

+IKA Architektur definiert Struktur

- Jede komplexe Aufgabe wird dadurch gelöst, dass sie in kleinere, beherrschbare Teile („Komponenten“) zerlegt wird.

Divide et impera

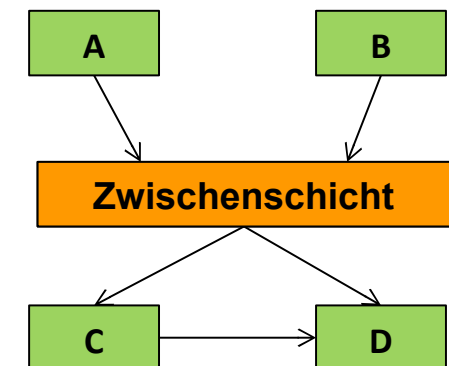
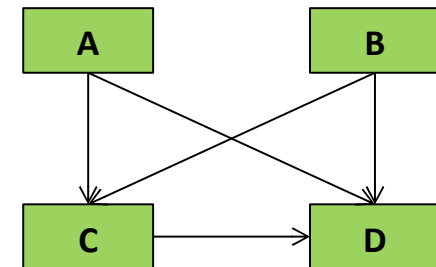
- Jede Komponente übernimmt eine dedizierte, überschaubare Teilaufgabe.
- Die strukturierte Zusammenarbeit der Komponenten löst das ursprüngliche Problem.

Die Architektur definiert diese Komponenten, strukturiert sie und beschreibt deren Interaktion.

Die Herausforderung liegt dabei in der Minimierung der Abhängigkeiten zwischen den Komponenten.

Definition:

Zwei Komponenten A und B sind abhängig, gdw. eine Änderung in einer der Komponenten eine Änderung in der anderen nach sich zieht.



+IKA Architektur definiert Struktur

Je mehr Abhängigkeiten zwischen den Komponenten bestehen, desto aufwendiger wird:

- Der Einbau neuer Funktionalität.
- Die Änderung bestehender Funktionalität.
- Das Testen des Systems.
- Das Bauen des Systems.

Und insbesondere die Zusammenarbeit und Abstimmung zwischen den einzelnen Entwicklungsteams und Teammitgliedern.

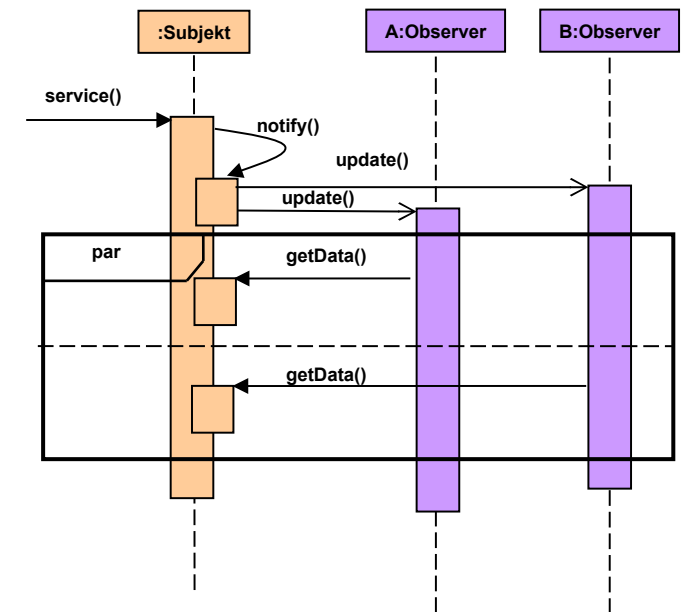
+IKA Architektur beschreibt Kommunikation

Komponenten, die zusammenarbeiten müssen kommunizieren, sie müssen Informationen austauschen.

Aufgabe der Architektur ist es, diesen Informationsaustausch zu definieren und zu beschreiben.

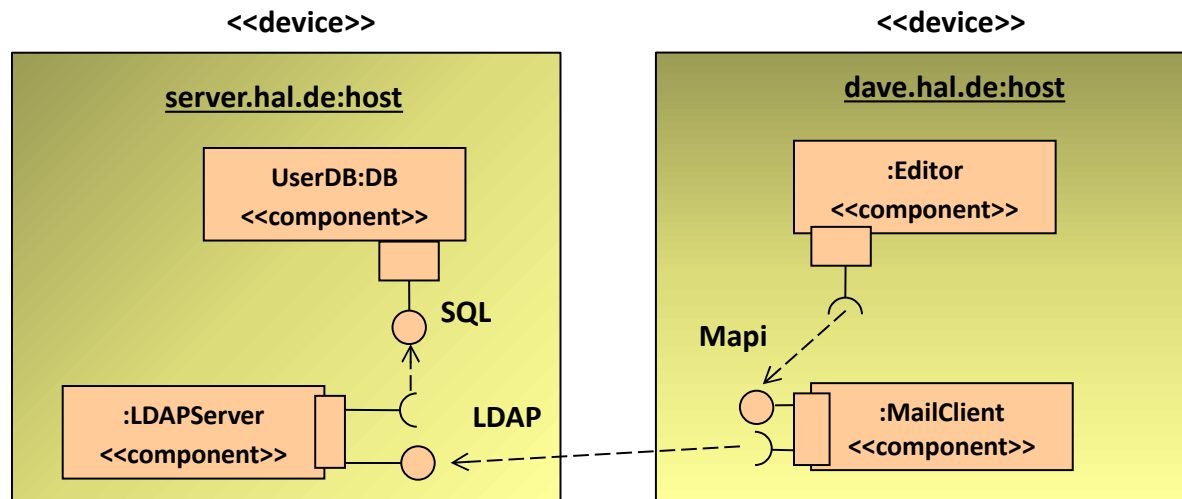
+IKA Architektur beschreibt Kommunikation

- Liegen die Komponenten im gleichen Adressraum?
 - Im gleichen Thread?
 - In unterschiedlichen Threads?
- Liegen die Komponenten im gleichen Prozessraum?
- Erfolgt der Informationsaustausch synchron oder asynchron?
- Müssen mehrere Komponenten informiert werden?
- Muss eine dedizierte Komponente informiert werden?
- Muss eine aus vielen gleichartigen Komponenten informiert werden?
- ...



+IKA Architektur beschreibt das Deployment

- Welche Komponenten liegen auf welchem Knoten?
- Welche Bibliothek entspricht welcher Datei?
- Welche Konfiguration hat welcher Server/Rechner?
- Welches Team ist für welche Komponente verantwortlich?
- ...



Auch die Definition der CI/CD-Pipeline ist Teil der Architektur!

Definitionen:

[C. Szyperski, D. Gruntz, and S. Murer, Component Software: Beyond Object-Oriented Programming, second ed. Addison-Wesley, 2005.]

- frei nach Szyperski:

Eine Komponente ist ein kontextabhängiger Softwarebaustein mit definierten Schnittstellen.

[B. Meyer, The Grand Challenge of Trusted Components, ICSE 25, IEEE Computer Press, 2003.]

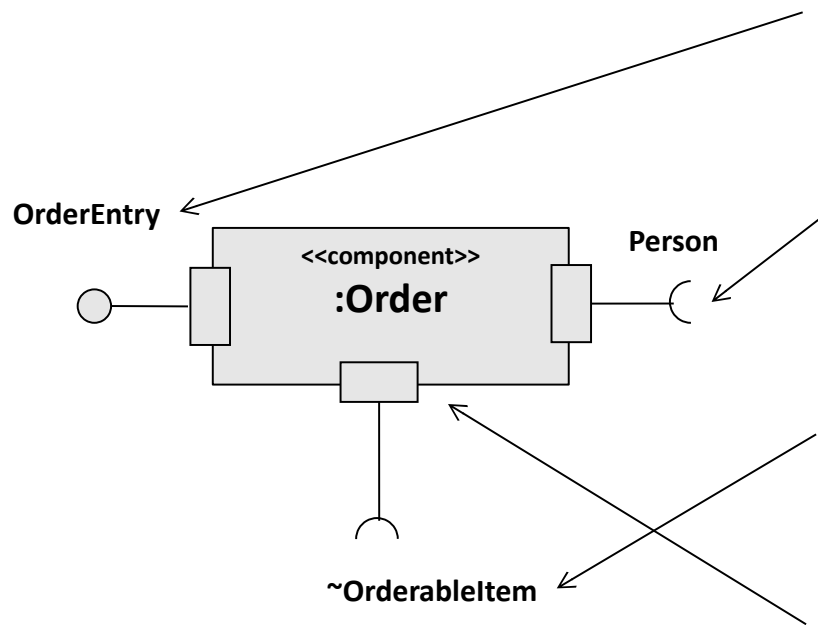
- frei nach Meyer:

Eine Komponente ist eine modulare Einheit, die folgende Bedingungen erfüllt:

1. Sie kann von anderen Komponenten (Client) verwendet werden.
2. Sie besitzt eine definierte Schnittstelle, die zur Nutzung ausreicht.
3. Sie ist nicht an einen vorgegebenen Client gebunden.

+IKA Komponenten: Modellierung mittels UML

[Unified Modeling Language Superstructure (2.4.1). OMG, 2011.]



Provided Interface:

- Eine Schnittstelle, die von Komponenten für andere Komponenten zur Verfügung gestellt wird.

Required Interface:

- Eine Schnittstelle, die von einer anderen Komponente zur Verfügung gestellt wird.
- Eine Schnittstelle, von der erwartet wird, dass sie in dieser Form von einer anderen Komponente zur Verfügung gestellt wird (~ konjugiert).

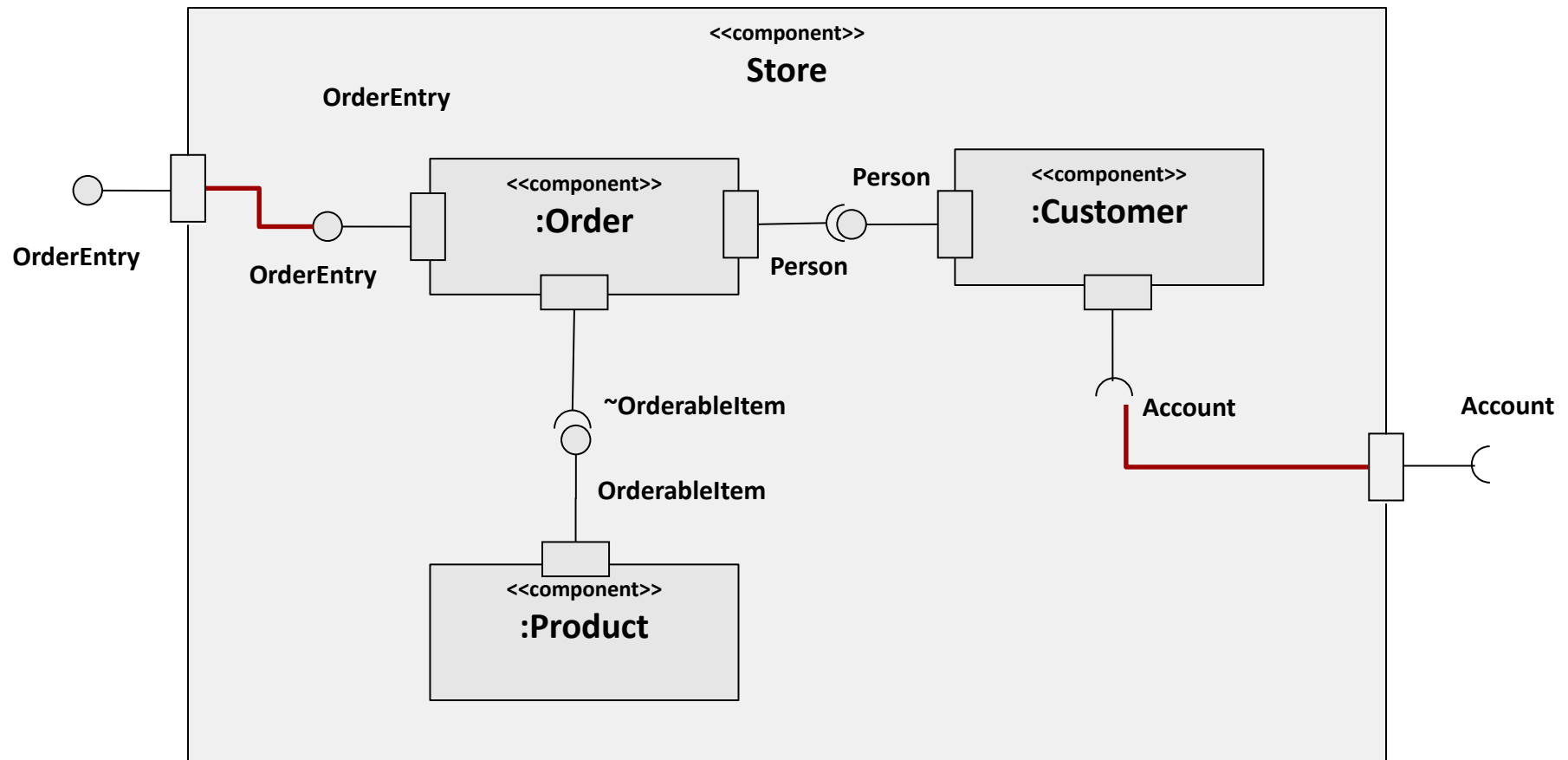
Port:

- Ein dedizierter Interaktionspunkt mit der Umgebung, beschrieben durch die Menge der zur Verfügung gestellten und benötigten Interfaces.

+IKA Konnektoren: Modellierung mittels UML

Ein Komponentendiagramm mit Konnektoren in Form von Delegates (Delegate-Connector).

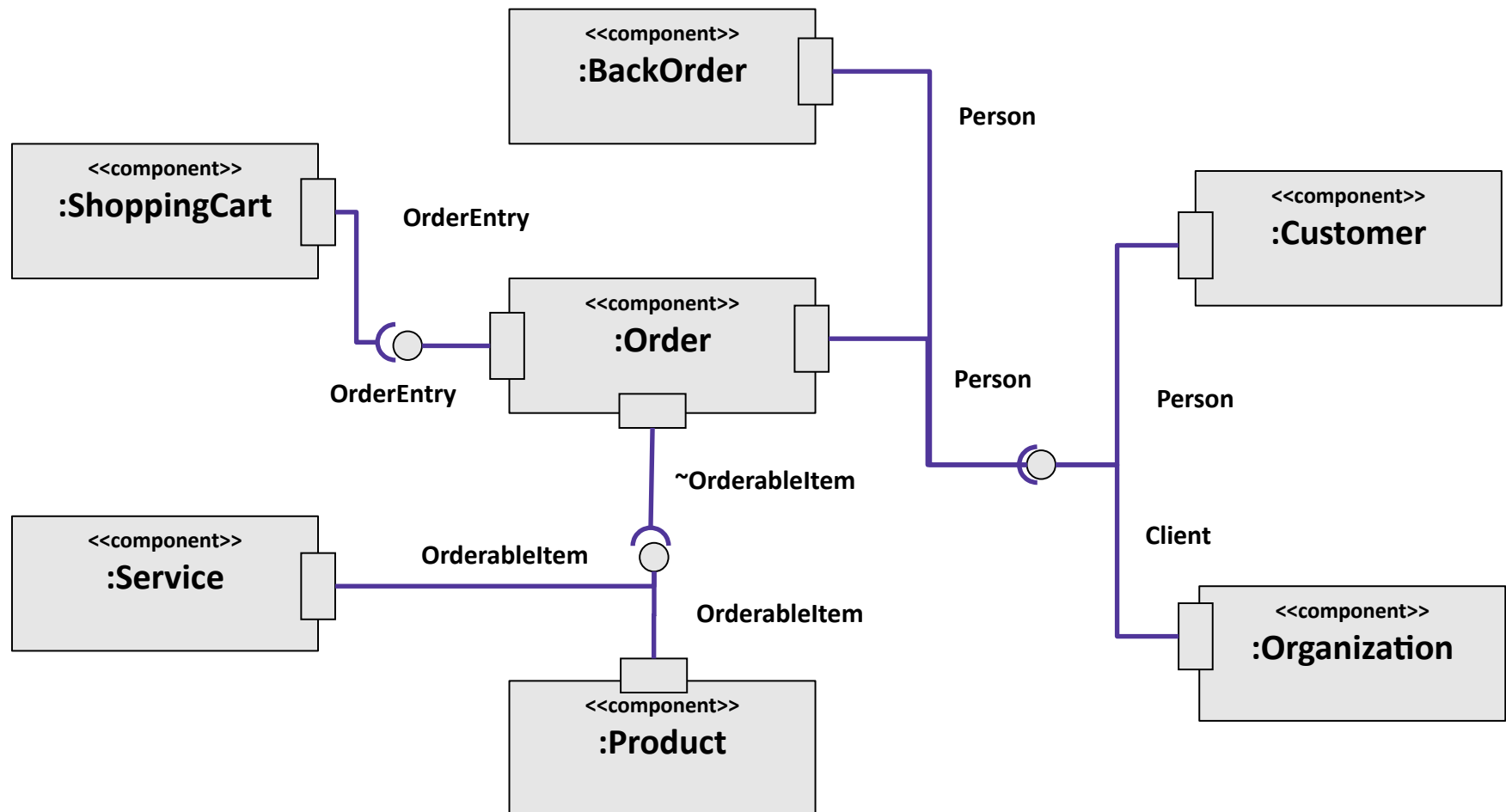
[Unified Modeling Language Superstructure (2.4.1). OMG, 2011.]



+IKA Konnektoren: Modellierung mittels UML

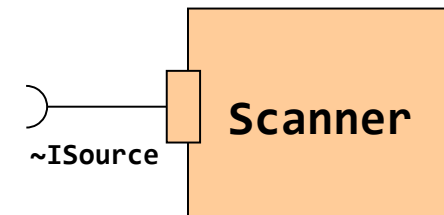
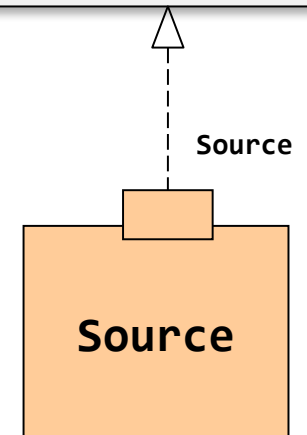
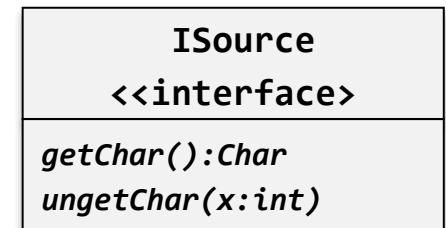
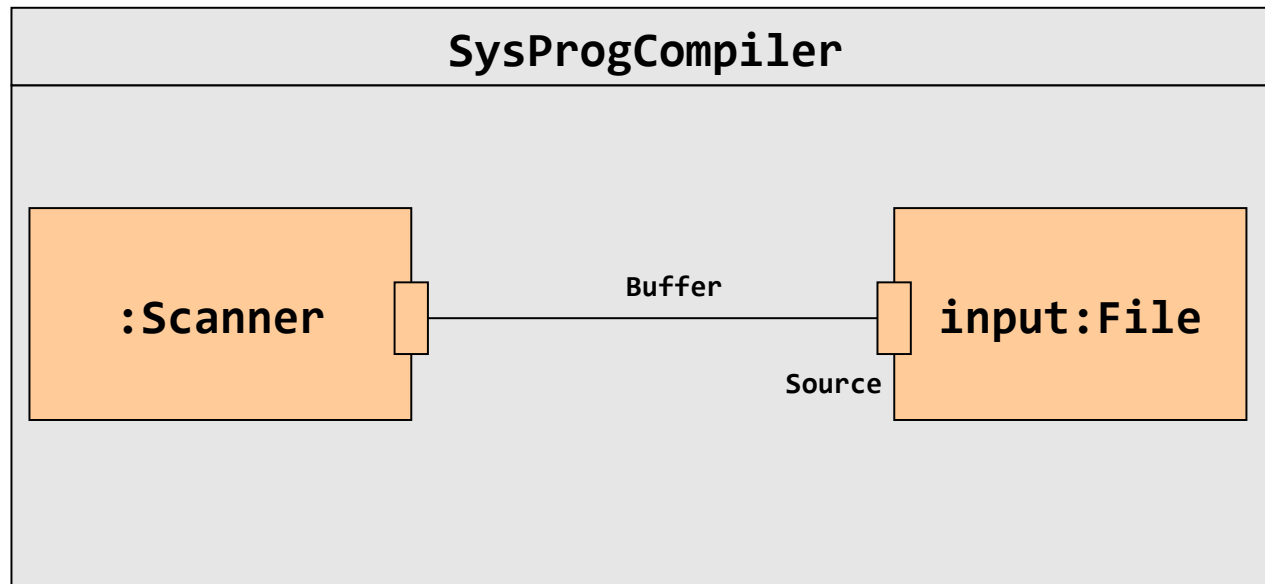
Ein Komponentendiagramm mit
Konnektoren in Form von Interfaces
(Assembly-Connector).

[Unified Modeling Language Superstructure (2.4.1). OMG, 2011.]



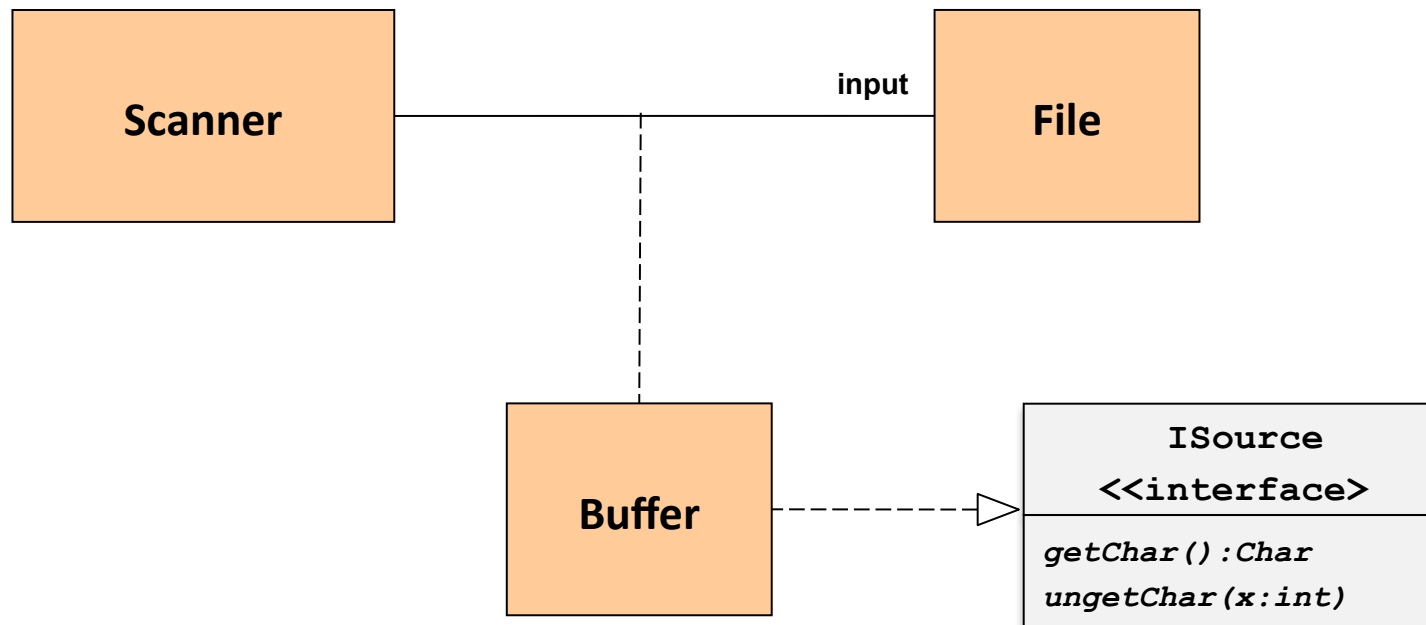
+K A Konnektoren: Modellierung mittels UML

Ein Kompositionsstrukturdiagramm
mit dezidiertem Konnektor zwischen zwei Ports.



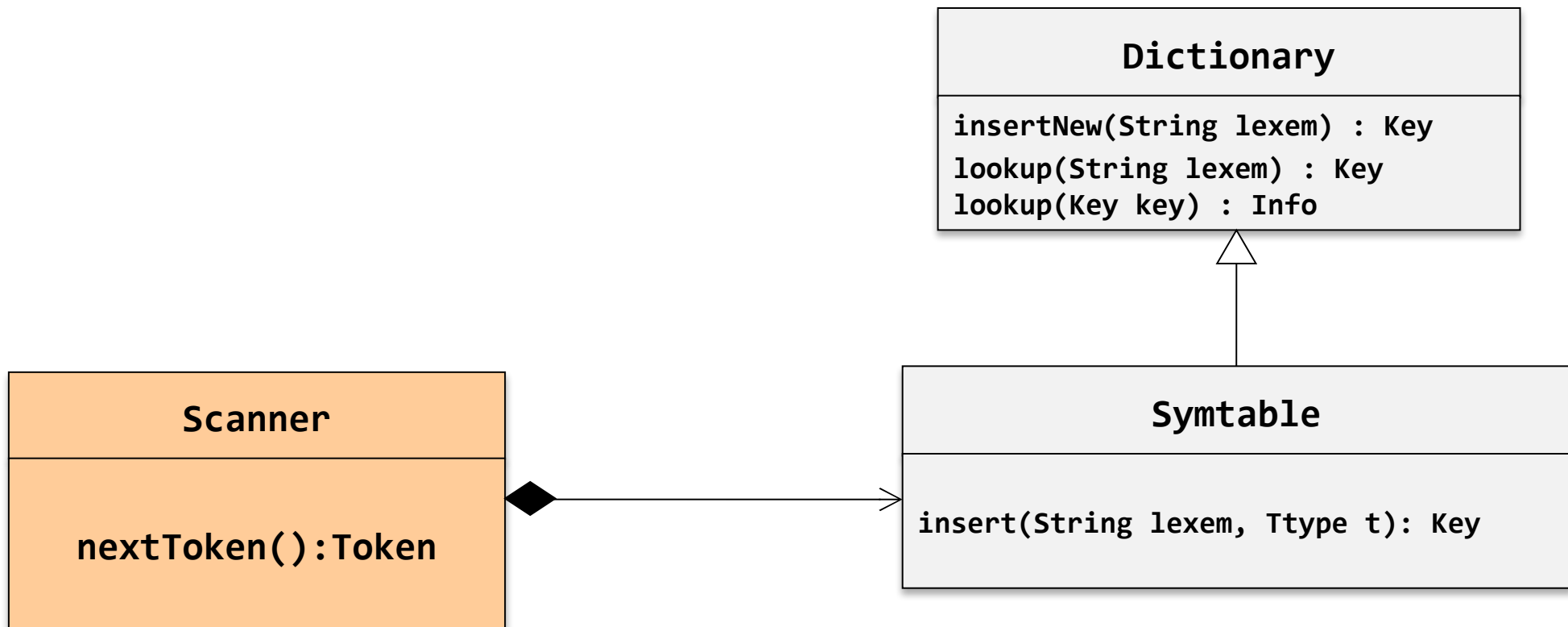
+IKA Konnektoren: Modellierung mittels UML

Ein Klassendiagramm mit einer Assoziationsklasse als Konnektor.



+KA Konnektoren: Modellierung mittels UML

Ein Klassendiagramm mit Assoziation als Konnektor.



Bei der Modellierung von Architekturen sollten implizite Konnektoren vermieden werden.

- Konnektoren bekommen eine eindeutige Bezeichnung.
- Konnektoren bekommen ihre eigene Spezifikation, die beschreibt:
 - Welche Schnittstellen angeboten werden (Funktionen).
 - Wie die Schnittstelle angeboten wird:
 - synchron, asynchron.
 - Welches Protokoll gesprochen wird:
 - FIFO, LIFO, Producer Consumer, ...
 - Wie die Schnittstelle umgesetzt wird:
 - JSON, RPC, JAVA-RX, Shared Memory, ...
 - Welche nicht-funktionalen Anforderungen die Schnittstellen erfüllen müssen.

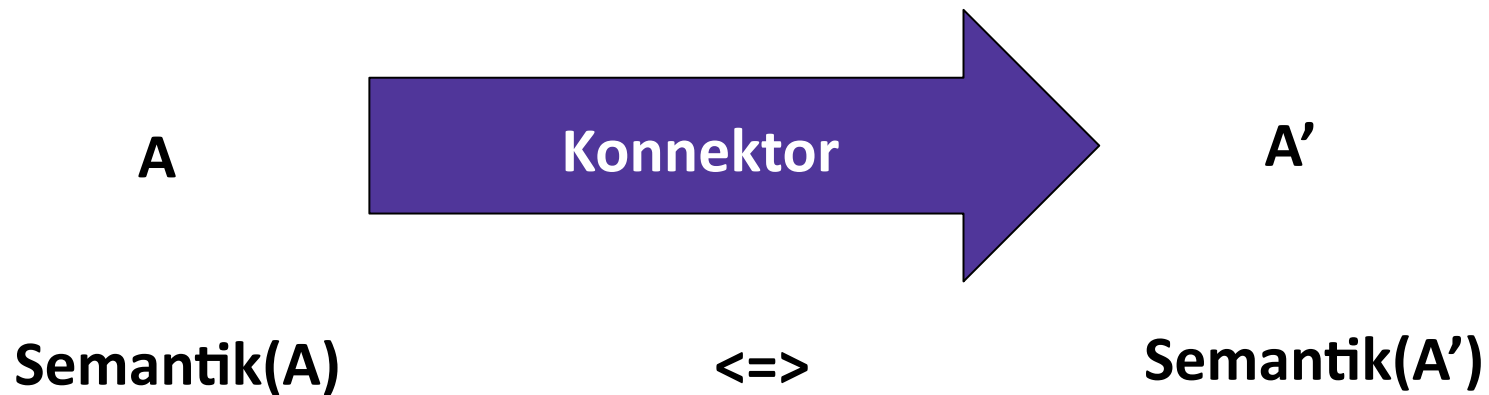
Bei der Definition und Beschreibung von Konnektoren gilt:

**Ein Konnektor „verändert“ nicht die Daten,
die er überträgt!**

**Es wird keine Information hinzugefügt oder
entfernt!**

+IK A Konnektoren:

Ein Konnektor „verändert“ nicht die Daten, die er überträgt!



Die Semantik ist abhängig vom Anwendungskontext!
Bsp.: Datenübertragung bei mobilen System erfordert oftmals eine Adaption!

+IKA Alternative Modellierungsansätze:

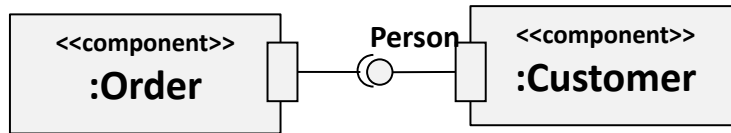
Spezifische Modellierungssprachen, Architecture Description Languages (ADLs)

[ABLE Research Group,
The Acme Project, Carnegie Mellon University,
<http://acme.able.cs.cmu.edu/>]

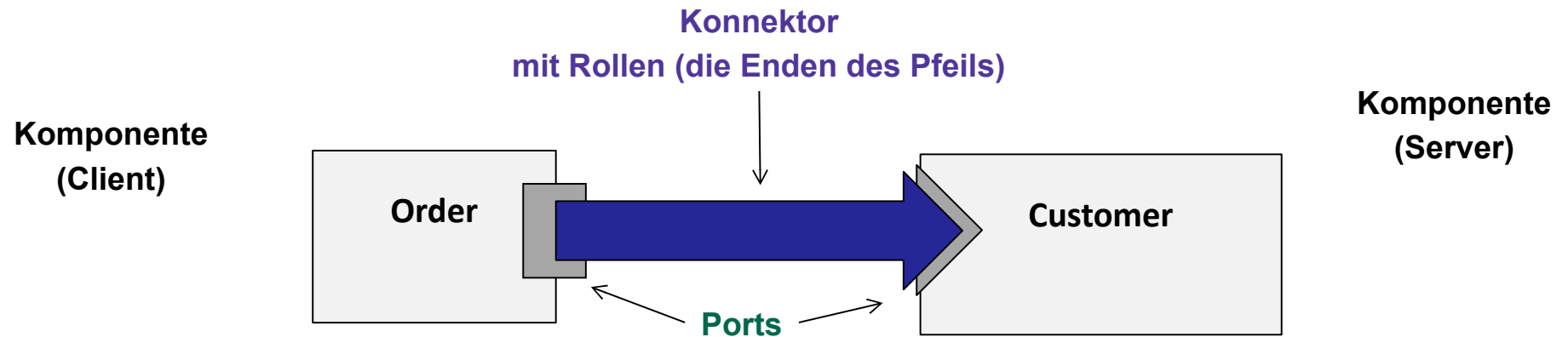
Bsp.: ACME

- **Austauschformat:** Werkzeugentwicklung wird möglich, Werkzeuge sind interoperabel.
- **Framework:** ACME kann erweitert und individualisiert werden. ACME selbst kann als Basis zur Entwicklung von Werkzeugen dienen.
- **ADL:** Einfache Sprachkonstrukte ermöglichen die Beschreibung von architektonischen Strukturen (Komponente) und deren Zusammenhänge (Konnektoren)

+IKA Alternative Modellierungsansätze (ACME)



[David Garlan, Robert T. Monroe, David Wile Acme: Architectural Description of Component-Based Systems. In Foundations of Component-Based Systems, Cambridge University Press, 2000.]



```
System mysys = {
    Component Order = {Port shipping}
    Component Customer = {Port address}
    Connector deliveryRequest = {Roles {caller, callee}
                                Properties {synchronous: boolean = true}}
    Attachments: {Order.shipping to deliveryRequest.caller; Customer.address to deliveryRequest.callee }
}
```



Software-Architekturen

—

Funktionale und nicht-funktionale Anforderungen

Prof. Dr. Thomas Fuchß

Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
Informatik und
Wirtschaftsinformatik

+IKA Funktionalen Anforderungen

Ein Softwaresystem wird erstellt, um Aufgaben für seine Anwender (Menschen, Systeme, ...) zu erledigen. Um ein erfolgreiches System zu entwickeln, müssen diese Aufgaben (funktionale Anforderungen) also erfasst und verstanden werden.

**Die funktionalen Anforderungen legen
den Umfang der Lösung fest.**



Was wird man bekommen.

+IKA Nicht-funktionalen Anforderungen

Ein Softwaresystem wird erstellt, um die Erwartungen seiner Anwender (Menschen, Systeme, ...) zu erfüllen. Um ein erfolgreiches System zu entwickeln, ist es also notwendig, dass diese Erwartungen (nicht-funktionale Anforderungen) erfasst und verstanden werden.

Die nicht-funktionalen Anforderungen (Constraints) definieren die Qualität einer Lösung.



Wie gut ist das, was man bekommt.

+IKA Typische nicht-funktionale Anforderungen

- **Leistung (Performance):** Das System muss schnell sein und möglichst viele Transaktionen gleichzeitig bearbeiten können.
- **Zuverlässigkeit (Reliability):** Das System muss bei Bedarf erreichbar sein und fehlerfrei arbeiten.
- **Sicherheit (Security):** Das System muss sensible Daten vertraulich behandeln und garantieren, dass keine unbefugte Person auf diese Daten zugreifen kann.
- **Skalierbarkeit (Scalability):** Das System muss mit wachsender Last und Nutzerzahl problemlos umgehen können.
- **Benutzerfreundlichkeit (Usability):** Das System sollte für den Endbenutzer einfach und intuitiv zu bedienen sein.
- **Wartbarkeit (Maintainability):** Das System sollte leicht zu ändern und instand zu halten sein.
- ...

Die Aufgabe der Architektur ist es, die unterschiedlichen nicht-funktionalen Anforderungen in das System abzubilden.

Die Umsetzung dieser Anforderungen bestimmt letztlich die Qualität der Architektur und des ganzen Systems.



**Nicht-funktionale Anforderungen
sind
Software Quality Attributes !**

+IKA Nicht-funktionalen Anforderungen

**Funktionale Anforderungen sind objektiv.
Funktionalität ist entweder vorhanden oder sie fehlt.**

**Nicht-funktionale Anforderungen sind subjektiv.
Qualität ist für jeden etwas anderes. Jeder hat andere
Ansprüche und Vorstellungen.**



**Um nicht-funktionale Anforderungen
messbar und somit bewertbar zu machen,
sind Metriken erforderlich.**

+IKA Technische Anforderungen

Ein Softwaresystem muss gebaut werden. Technische Anforderungen sind Vorgaben zu Programmiersprachen, Plattformen, Hardware, Frameworks, ..

**Sind technische Anforderungen
funktional oder nicht-funktional?**



Eine verallgemeinerte Sichtweise!

Alle Anforderungen, deren Erfüllungsgrad sich mit einem „Ja“ oder „Nein“ beantworten lässt, sind objektiv oder „funktional“!

Alle Anforderungen, über deren Erfüllungsgrad man sich streiten kann, sind subjektiv und somit „nicht-funktional“!

Über Metriken können nicht-funktionale Anforderungen durch die Festlegung der Maßzahlen in objektive (pseudo-)funktionale Anforderungen verwandelt werden.

Eine verallgemeinerte Sicht!

- **Sicherheit (Security):**

Das System muss sensible Daten vertraulich behandeln und garantieren, dass keine unbefugte Person auf diese Daten zugreifen kann.

„nicht-funktional“

- **Zugriffskontrollen:**

Nach drei Fehlversuchen muss der Account für 5 Minuten gesperrt werden.

„funktional“
und
überprüfbar

- **Datenverschlüsselung:**

Alle personenbezogenen Daten sind mit AES-256 zu verschlüsseln.

+IKA Typische Metriken

- **Leistung (Performance):**
 - Antwortzeit: Die Zeit, die ein System benötigt, um auf etwas zu reagieren.
 - Durchsatz: Die Anzahl der Transaktionen pro Zeiteinheit.
- **Zuverlässigkeit (Reliability):**
 - Fehlerrate: Die Häufigkeit, mit der ein Fehler auftritt.
 - mittlere Betriebsdauer: Die durchschnittliche Zeitspanne, bis zum Ausfall.
- **Skalierbarkeit (Scalability):**
 - Skalierungszeit: Die Zeit, die das System benötigt, um seine Ressourcen anzupassen.
- **Wartbarkeit (Maintainability):**
 - Die durchschnittliche Zeit zur Fehlerbehebung.

[Clements, P.; Bass, L. and Kazman, R.
Software Architecture in Practice, 2. ed. Addison-Wesley, 2003.]

Qualität muss man wie Funktionalität definieren!

Qualität ist nicht NULL oder EINS!

Qualität muss man bewerten!

Qualität muss man messen!

Qualität muss man testen!

Wie kann man dies tun?



Mit Hilfe von Szenarien!

Ein Ansatz zur Beschreibung von Software Quality Attributes

[Clements, P.; Bass, L. and Kazman, R.
Software Architecture in Practice, 2. ed. Addison-Wesley, 2003.]

**Ein Szenario ist die konkrete Ausprägung eines Use Cases.
Es beschreibt die Interaktion zwischen einem „Nutzer“ und einem System
in einer besonderen Situation.**

Bsp.: Der erfolgreiche Kauf eines Produkts.
Der Kauf eines Produkts mit Kreditkarte.
Das Scheitern aufgrund der Eingabe einer falschen PIN.

**Ein Anwendungsfall ist eine Menge von Szenarien, die durch ein gemeinsames
Ziel verbunden sind.**

**Man analysiert Anwendungsfälle durch zentrale Szenarien.
Mit Interaktionsdiagrammen, Zustandsdiagrammen,
Aktivitätsdiagrammen und prototypischen Implementierungen.**

Zur Beschreibung eines Szenarios für Software Quality Attributes sind folgende Angaben wichtig:

- **Quelle (Source of Stimulus):**
Entspricht einem Akteur; diese löst ein definiertes Ereignis aus.
- **Stimulus:**
Ein definiertes Ereignis, auf das das System geeignet reagieren muss.
- **Artefakt:**
Teil des Systems, der durch den Stimulus beeinflusst wird.
- **Environment:**
Definierte Ausprägungen des Systems unter denen das Szenario betrachtet wird.
- **Antwort (Response):**
Beschreibung des Systemverhaltens unter Einwirkung des Stimulus.
- **Maße und Metriken (Response Measure):**
Angaben zur späteren Bewertung des Systems.

+IKA Beispiel (Verfügbarkeit)

Verfügbarkeit ist eine besondere Ausprägung der Zuverlässigkeit eines Systems.

Gesucht sind Szenarien, die quantitative Aussagen zu folgenden Gesichtspunkten liefern:

- Wie häufig tritt ein (bestimmter) Fehler auf?
- Wie verhält sich das System im Fehlerfall?
- Über welchen Zeitraum erstreckt sich ein Systemausfall?
- Wie werden Fehler erkannt?
- Wie werden Fehler korrigiert?
- Wer oder was wird über das Auftreten eines Fehlers informiert?
- ...

+HKA Beispiel (Verfügbarkeit) Concerns

- **Der Zeitraum, in dem ein System nicht verfügbar ist (Downtime):**
 - Geplante Downtime: Wartung, Installation, Upgrades, ...
 - Ungeplante Downtime: Verursacht durch nicht maskierbare Hardware- oder Software-Ausfälle.
- **Die Reparaturzeit, bis ein Fehler beseitigt wurde (Time to Repair):**
 - Dauer des Austausches einzelner Komponenten.
 - Dauer des Einspielens eines Patches.
 - Schließen einer Sicherheitslücke.
- **Die Zeit, die zum Wiederauffahren benötigt wird (Time to Recover):**
 - Nach Stromausfall.
 - Nach Feuer.
 - Nach Datenverlust.

+IKA Beispiel (Verfügbarkeit) Szenarien

Typische Ausprägungen von Szenarien mit deren Hilfe Aussagen zur Verfügbarkeit getroffen werden.

- **Quelle (Source of Stimulus):**
 - fehlerhafte Komponenten,
 - ausgefallene Konnektoren,
 - externe Störer, ...
- **Stimulus:**
 - ein Ausfall einer Komponente / Konnektors,
 - unerwartetes Ereignis,
 - falsch formatierte Nachricht, ...
- **Artefakt:**
 - Zusammenhängende Systemteile,
 - Einzelne Komponente oder Konnektoren,
 - Datenbank oder Web-Server,...

+IKA Beispiel (Verfügbarkeit) Szenarien

Typische Ausprägungen von Szenarien mit deren Hilfe Aussagen zur Verfügbarkeit getroffen werden.

- **Environment:**
 - typische Systemkonfigurationen und Lastzustände,
 - kritische Belastungsgrenzen, ...
- **Antwort (Response):**
 - Fehler erkannt, weitergeleitet, bereinigt;
 - Teile des Systems schalten sich ab;...
- **Maße und Metriken (Response Measure):**
 - konkrete Angaben,
 - prozentuale Angaben:
 - Wahrscheinlichkeit,
 - Häufigkeit,
 - Dauer, ...

+IKA Metriken für Verfügbarkeit

- Verfügbarkeit:

$$\text{Verfügbarkeit} = \frac{\text{Gesamtzeit - Downtime}}{\text{Gesamtzeit}}$$

- Mittlere Betriebsdauer (Mean Time Between Failures):

$$\text{MTBF} = \frac{\text{Gesamtzeit}}{\text{Anzahl der aufgetretenen Fehler}}$$

+IKA Beispiel (Modifizierbarkeit)

Modifizierbarkeit definiert den Aufwand, der betrieben werden muss, um eine bestehende Anwendung an neue funktionale und nicht-funktionale Anforderungen anzupassen.

Gesucht sind Szenarien, die quantitative Aussagen zu folgenden Gesichtspunkten liefern:

- Was verursacht eine Änderung?
- Wer verursacht die Änderung?
- Wann kommt es zur Änderung?
- Wo kommt es zur Änderung?

Welche Kosten entstehen bei der

- Erstellung neuer Analysen,
- Erstellung neuer Designs,
- Erstellung neuer Implementierungen,
- Beschaffung neuer Hardware,...?

Wie hoch ist die Wahrscheinlichkeit,

- dass sich eine spezifische Komponente ändert?
- dass sich ein spezifischer Konnektor ändert?

+IKA Beispiel (Modifizierbarkeit) Szenarien

Typische Ausprägungen von Szenarien mit deren Hilfe Aussagen über die Modifizierbarkeit gemacht werden können.

- **Quelle (Source of Stimulus):**
 - Der Endanwender, der mit der Funktionalität nicht zufrieden ist.
 - Der Entwickler, der eine Implementierung nicht durchführen kann.
 - Eine „zugekaufte“ Komponente, die mehr versprochen hat als sie hält. ...
- **Stimulus:**
 - Änderungswünsche,
 - **Nicht erfüllte nicht-funktionale Anforderungen (Qualitätsanforderungen)** (Useability, Security, Scalability, Performance, ...),
- **Artefakt:**
 - zusammenhängende Systemteile,
 - einzelne Komponente oder Konnektoren,
 - Datenbank oder Web-Server,...

+IKA Beispiel (Modifizierbarkeit) Szenarien

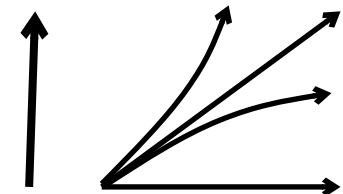
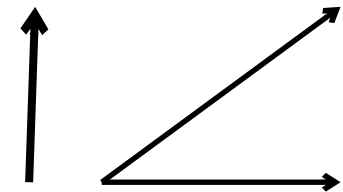
Typische Ausprägungen von Szenarien mit deren Hilfe Aussagen über die Modifizierbarkeit gemacht werden können.

- **Environment:**
 - typische Systemkonfigurationen Im Betrieb,
 - typische „Konstellationen“ während der Entwicklung des Systems,
 - Betrachtungen im Rahmen des Build-Prozesses, ...
- **Antwort (Response):**
 - Wo muss was verändert werden,
 - Wer kann diese Änderung durchführen,
 - Wie pflanzt sich eine Änderung fort? ...
- **Maße und Metriken(Response Measure):**
 - Kosten (Betrag x €),
 - Zeit (Manntage),
 - Umfang (LOC),

Wie verhält sich die Lösung, wenn eine Dimension des Problemraums sich ausdehnt?

Typische Dimension:

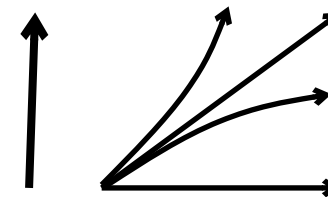
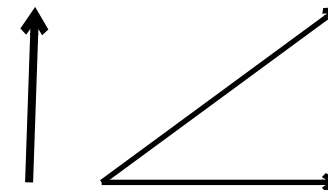
- Lastverhalten:
 - Antwortzeit pro Anfrage in Sekunden
 - Durchsatz an Transaktionen pro Sekunde
- Anzahl simultaner Verbindungen:
 - Ressourcenverbrauch pro Verbindung



+IK A Weitere nicht-funktionale Anforderungen (Scalability)

Typische Dimension:

- Größe der Daten:
 - Anzahl der Daten
 - Größe eines einzelnen Datums
- Anzahl an Installationen:
 - Individuelle Anpassungen
 - Installationsaufwand
 - Wartungsaufwand
 - ...
- ...



+IKA Weitere nicht-funktionale Anforderungen (Scalability)

Problem: Oftmals ist Skalierbarkeit zu Beginn des Projekts kein Thema und damit auch keine „offizielle“ Anforderung!

- Nicht skalierbare Architekturen sollten prinzipiell vermieden werden.

Problem: Test, um sicherzustellen, dass eine Architektur skaliert, sind oftmals teuer und werden deshalb weggelassen!

- Man sollte auf **Bewährtes** zurückgreifen!

+IKA Weitere nicht-funktionale Anforderungen (Performance)

Performance (Leistung) ist die Summe der erbrachten Arbeit in einer definierten Zeit.

Typische Leistungsgrößen:

- Durchsatz (Transaktionen pro Sekunde)
 - im Mittel oder im Grenzfall,
 - in einem definierten Zeitraum, ...
- Antwortzeit (Latenzzeit), die Zeit, zwischen Aktion und Reaktion
 - garantierte Antwortzeit,
 - durchschnittliche Antwortzeit,...
- Deadlines
 - feste Zeitschranken,
 - relative Zeitschranken,...

$$P = \frac{\Delta W}{\Delta t}$$

Eine gute Architektur muss all diesen Anforderungen und oftmals widersprüchlichen Zielsetzungen gerecht werden.



Eine echte Herausforderung!



Software-Architekturen

—

Die Aufgaben des Architekten: Wählen, Bewerten und Teste

Prof. Dr. Thomas Fuchß

Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
**Informatik und
Wirtschaftsinformatik**

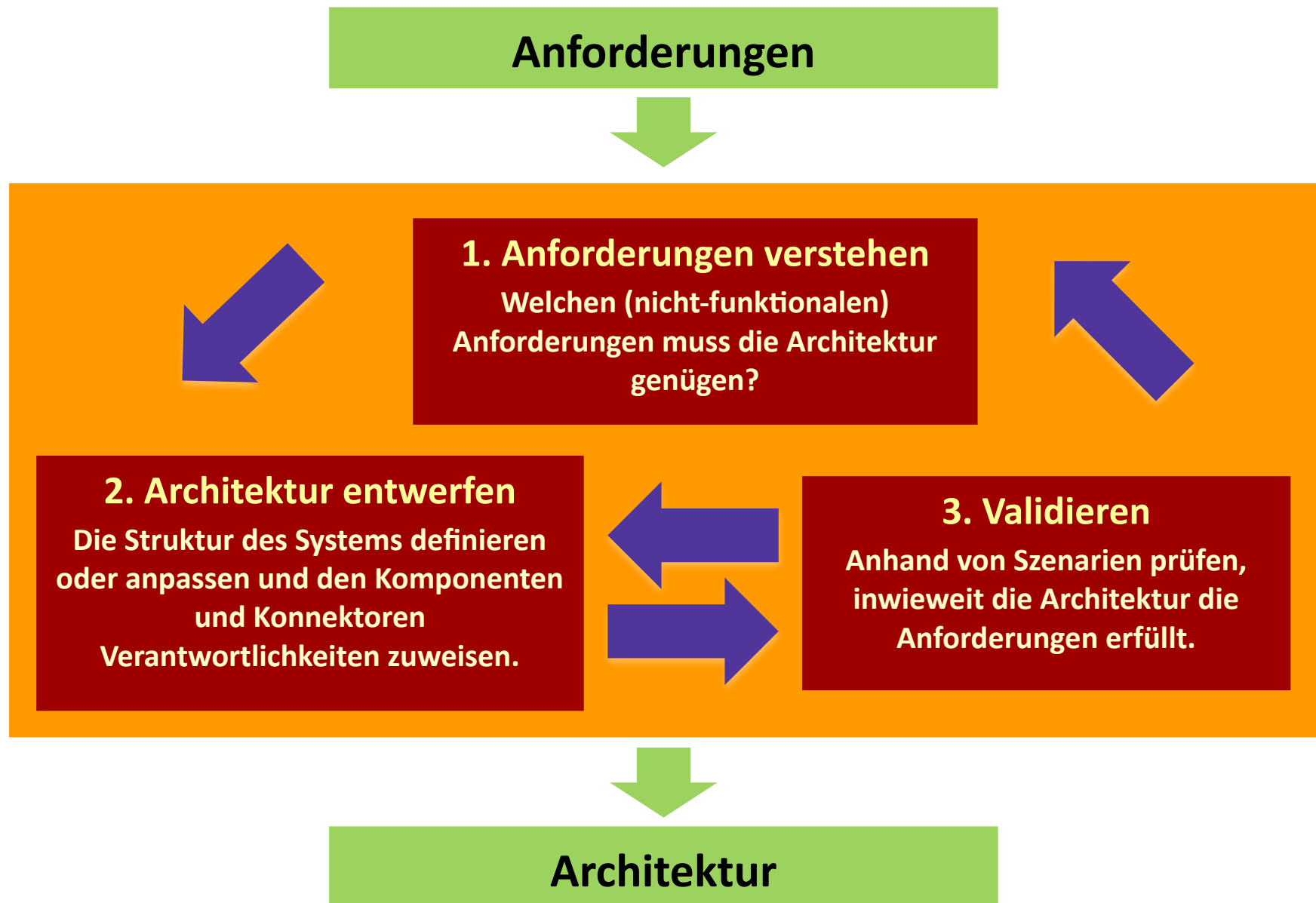


+HKA Architekturentwurf, eine erste Idee

1. Zentrale nicht-funktionale Anforderungen führen zur
 - Auswahl von Sprachen, Frameworks und Middleware-Konzepten.
 - Definition erster Komponenten und Konnektoren.

Ein erster Architekturentwurf entsteht.

2. Entwurf wird überprüft
 - Frameworks werden beibehalten, verworfen oder angepasst.
 - Die Strukturierung erscheint plausibel und wird beibehalten.
 - Die Strukturierung hat Defizite und muss überarbeitet werden.
 - **Die Anforderungen (Szenarien) sind nicht aussagekräftig genug.**
 - Anforderungen müssen genauer analysiert werden (weitere Szenarien).
 - Weitere nicht-funktionale Anforderungen sind zu berücksichtigen.
 - **Rücksprache mit dem Auftraggeber** (Product Owner).



Achtung!

Die Entwicklung der Architektur ist kein monolithischer Schritt.

Allen modernen Prozessmodellen gemeinsam ist ein agiles, iteratives und risikoorientiertes Vorgehen.



Auch die Entwicklung der Architektur erfolgt agil. Sie ist inkrementell, iterativ und risikoorientiert.

Achtung!

Die Entwicklung der Architektur ist nicht losgelöst von der Umsetzung der funktionalen Anforderungen.

Architektur und Use Cases beeinflussen sich gegenseitig und reifen während des Entwicklungsprozesses.

[RUP]



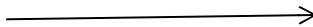
Die Entwicklung der Architektur geht Hand in Hand mit der Entwicklung des Gesamtsystems.

+IK A Agile Software-Entwicklung (SCRUM)

Requirements

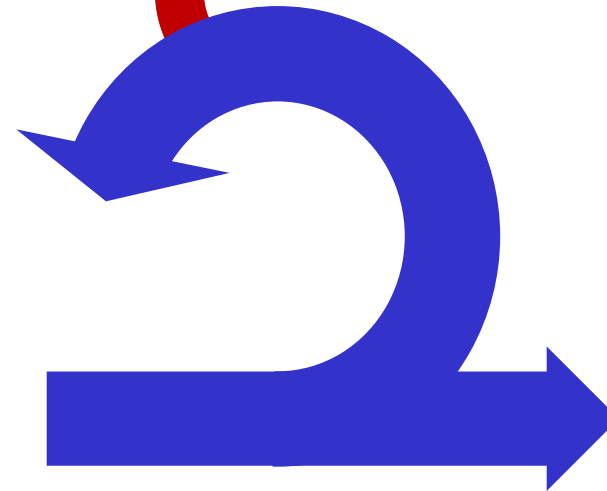
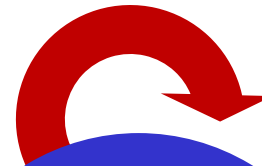


Grooming



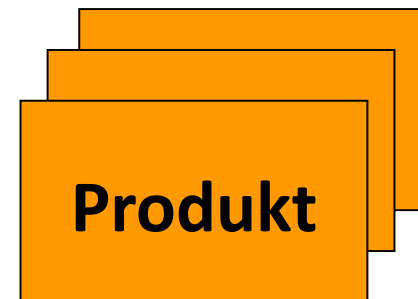
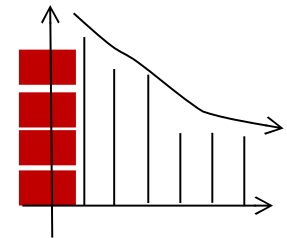
Arbeitspakete
(Tasks)

(1 Tag)



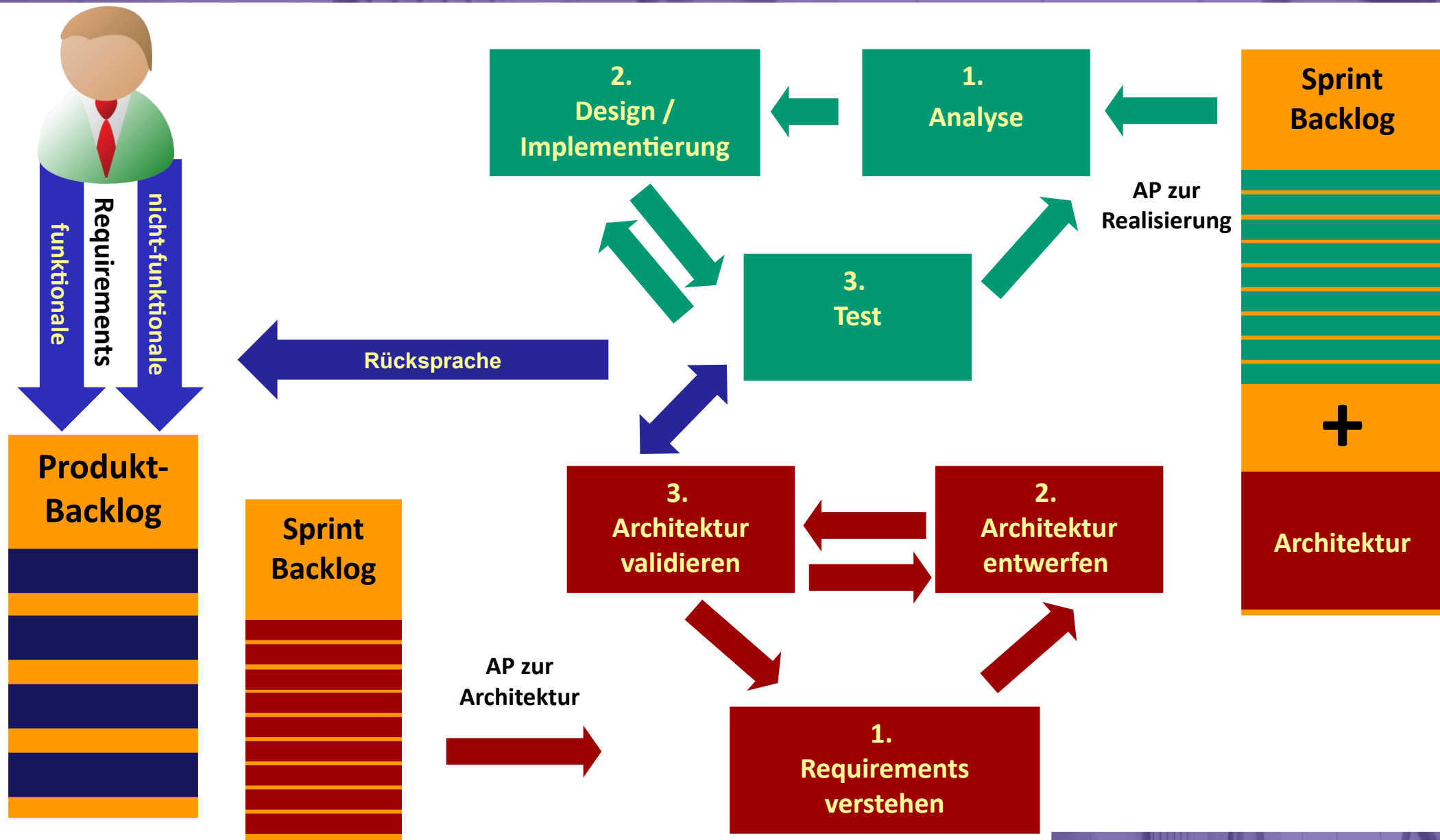
Sprint (1 Monat)

Burndown Chart



Produkt

+IKA Architekturentwurf (iterativ)



+IKA Architekturentwurf (agil)

- Die Architektur entsteht im Rahmen der ersten Sprints

Das System hat noch einen eher prototypischen Charakter.

- Nachfolgende Sprints bauen auf dieser Architektur auf.

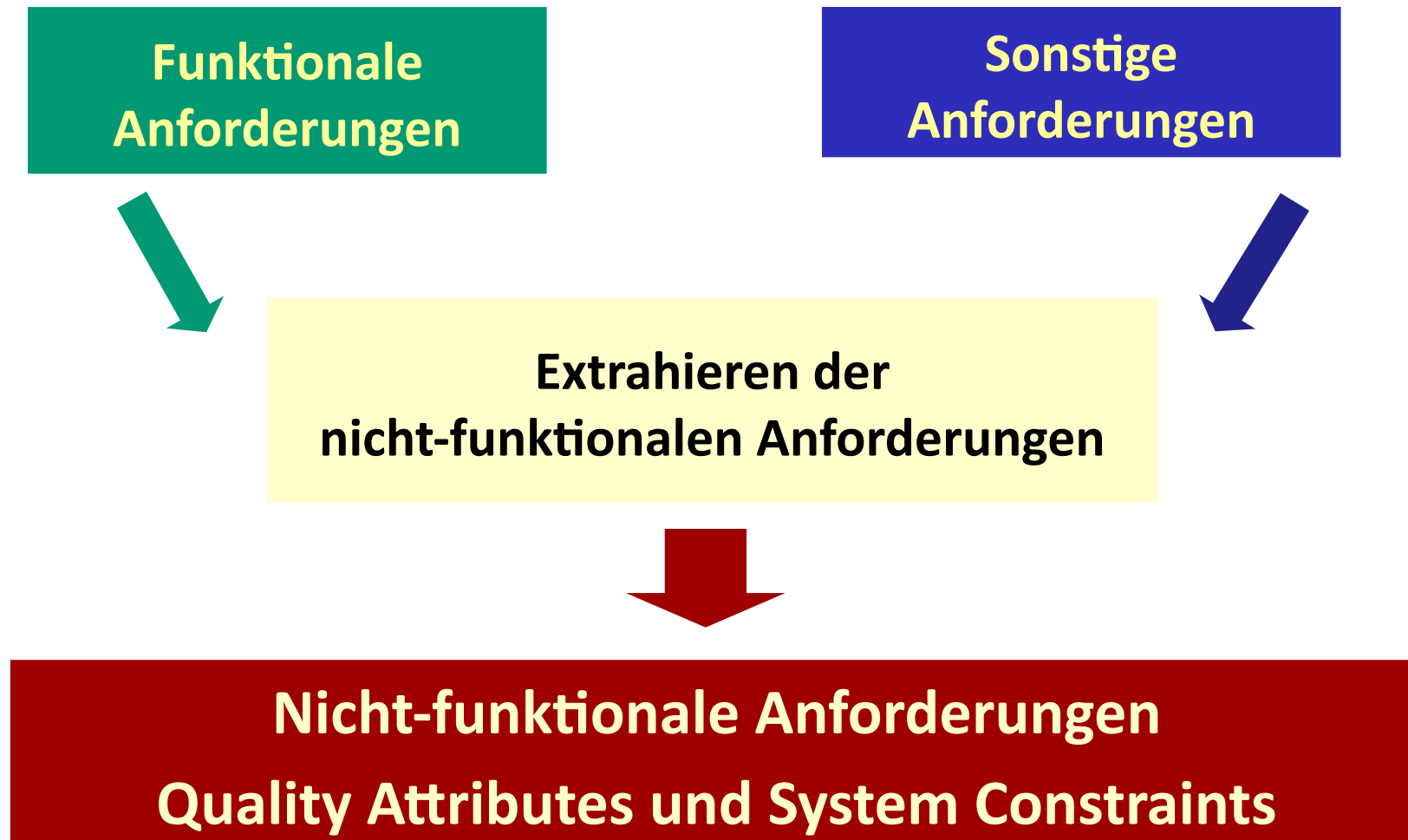
Der Kern stabilisiert sich.

Funktionalität kommt hinzu.



Anpassung durch permanentes Refactoring.

+HKA 1. Schritt: Requirements erfassen



+HKA 1. Schritt: Requirements erfassen

Quality Attribute	Architekturanforderung
Performance	Die Antwortzeit auf eine Anfrage muss ein flüssiges Arbeiten ermöglichen. (Ziel: Antwortzeiten < 1 s)
Security	Alle Benutzer müssen eindeutig identifiziert sein.
Security	Jeder Kommunikationsweg, der nicht vollständig im eigenen Back-End liegt, muss verschlüsselt sein (Mechanismenstärke hoch).
Verfügbarkeit	Das System muss permanent (24 x 7 x 365) laufen, bei einer Verfügbarkeit von 95 %.
Verfügbarkeit	Die MTBF muss mindestens 2 Wochen betragen.
Modifizierbarkeit	Die Architektur muss so gewählt werden, dass eine Anbindung mobiler Endgeräte leicht möglich wird.
...	...

+HKA 1. Schritt: Requirements erfassen

Technische Constraints	Architekturanforderung
Plattform	Als Application Server muss Tomcat X.Y ausreichend sein.
Programmiersprache	Die Programmiersprache muss Echtzeitanforderungen genügen.
...	...

Business Constraints	Architekturanforderung
Zeitplan	Unser Mitbewerber arbeitet an einem ähnlichen Produkt. Um erfolgreich zu sein, müssen wir spätestens Ende des Jahres eine erste Version ausliefern können.
Kosten	Das Budget darf 100.000 Euro nicht überschreiten.
...	

+IKA 2. Schritt: Priorisieren

Wenige klar definierte Stufen erzeugen Übersichtlichkeit.

- **Hoch: Die Anwendung muss das Requirement erfüllen.**
Es ist die primäre Aufgabe der Architektur, diese Aufgaben zu erfüllen.
- **Mittel: Die Anwendung sollte das Requirement erfüllen.**
Die Architektur sollte so angelegt sein, dass im Laufe der Weiterentwicklung des Produkts die Anforderung in einem späteren Release erfüllt werden kann.
- **Niedrig: Es wäre schön, wenn die Anwendung das Requirement erfüllt.**
Gegen diese Anforderungen sollte nicht mutwillig verstoßen werden.

+IKA 2. Schritt: Priorisieren

Probleme entstehen sobald Anforderungen sich widersprechen.

- **Modifizierbarkeit:** Die Architektur muss so gewählt werden, dass eine Anbindung mobiler Endgeräte leicht möglich wird.
- **Zeitplan:** Unser Mitbewerber arbeitet an einem ähnlichen Produkt. Um erfolgreich zu sein, müssen wir spätestens Ende des Jahres eine erste Version ausliefern können.

Was ist wichtiger: Die Anbindung mobiler Geräte oder das Einhalten des Termins?

- **Kosten:** Das Budget darf 100.000 Euro nicht überschreiten.

Die Entwicklungsdauer kann nur bedingt durch den Kauf oder die Lizenzierung von Fremdsoftware gesenkt werden.

+IKA 3. Schritt: Entwurf

Aufgabe des Architekten ist es, diese Konflikte im Rahmen des Architekturentwurfs zu entschärfen.

- **Wahl der primären Architektur. (Client-Server, N-Tier, ...)**
- **Zerlegen des Problems in Komponenten.**
- **Definieren der Konnektoren.**

Für diese Aufgabe benötigt man Wissen und Erfahrung.

+HKA 4. Schritt: Validieren

- Szenarien durchspielen.

Achtung: Die Architektur beruht auf zentralen Szenarien. Zum Testen sollten diese variiert werden. Ansonsten ist es unwahrscheinlich, dass man Fehler findet.

- Prototypen bauen.

Achtung: Prototypen sind für die Beantwortung von Fragen in Bezug auf Skalierung und Performance nur bedingt geeignet.

+IKA 4. Schritt: Validieren



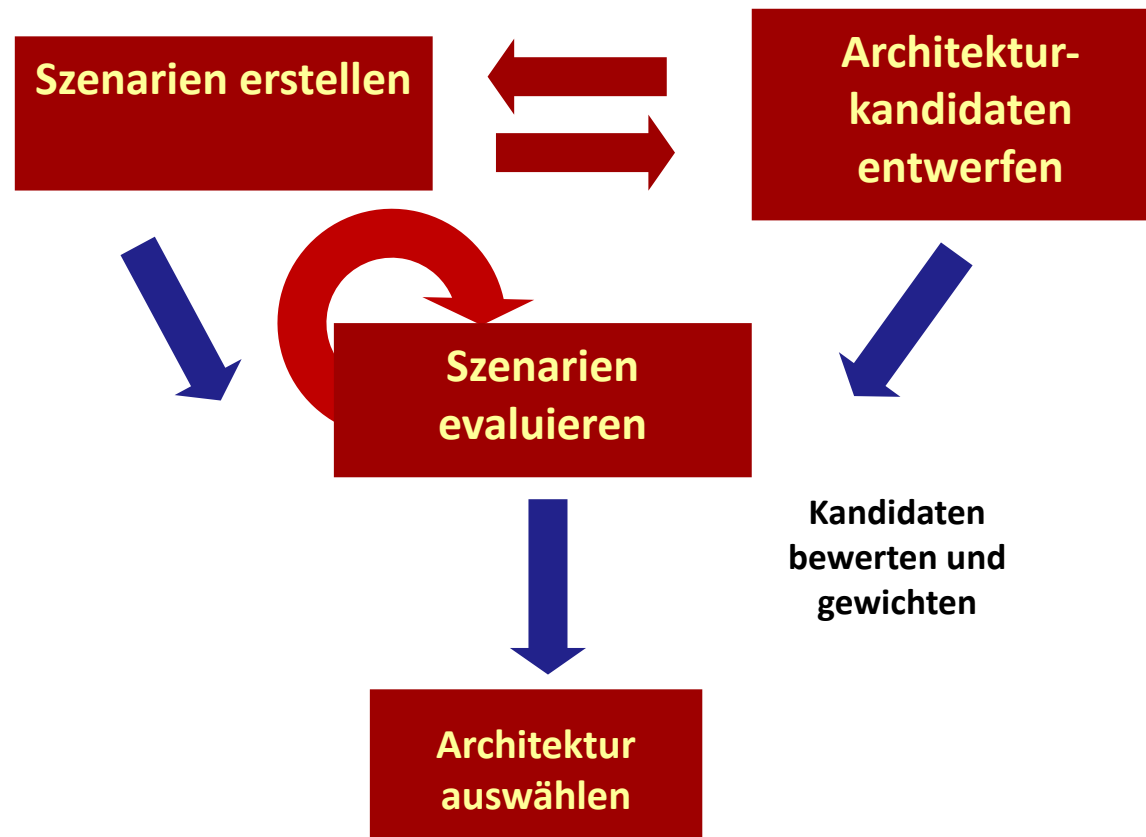
Zentrale Architekturanforderungen werden in den ersten Sprints bearbeitet.

Tests zur Validierung der Architektur können am „echten Produkt“ durchgeführt werden.

+HKA 4. Schritt: Validieren

[Kazman, R.; Abowd, G.; Bass, L.; Clements, P.:
Scenario-Based Analysis of Software Architecture, IEEE Software 13(6), 1996.]

Software Architecture Analysis Method (SAAM)





Software-Architekturen

—

Referenzarchitekturen und Designprinzipien

Prof. Dr. Thomas Fuchß

Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
Informatik und
Wirtschaftsinformatik

**Für viele dieser immer wiederkehrenden Probleme,
Aufgaben und Situationen, gibt es bewährte
Lösungen.**

**Patterns beschreiben diese Lösungsmodelle
systematisch und sind damit ein zentraler
Ausgangspunkt für ein ingenieurmäßiges Herangehen
an die Entwicklung großer Software-Systeme.**

Diese Beschreibung geschieht in der Regel durch eine verallgemeinerte Darstellung von Interfaces, Komponenten, Klassen, Methoden, Attributen und deren Beziehungen.

- Wiederverwendung von (bewährten) Architekturen.



Qualitativ bessere Entwürfe!

- Dokumentation von Entscheidungen.



Das Verstehen der Software wird verbessert!

- Effizientere Kommunikation durch ein Standardvokabular.



Wissensaufbau und Wissenserhaltung!

Das Problem beruht im Allgemeinen darauf, dass in einer gegebenen Situation naheliegende Realisierungen unterschiedlicher Anforderungen nicht zu vereinen sind.

Patterns lösen diese Konflikte!



Ein Pattern ist eine Relation zwischen dem Kontext (Situation), dem System sich widerstrebender Kräfte (Anforderungen) und der Beschreibung einer bewährten Lösung für diesen Konflikt.

Die Qualität eines Patterns beruht auf der Lösungsbeschreibung!

Sie sind:

- **Konstruktiv:** Patterns liefern Instruktionen zur Lösung einer Problemsituation.
- **Abstrakt:** Patterns bieten ein breites Anwendungsspektrum mit vielen verschiedenen Anwendungsfällen.
- **Stabil:** Patterns erfassen den stabilen Kern der Lösung.



Patterns sind langlebig und keine Modeerscheinung!

+IKA Aufgaben von Architekturmustern

- **Helfen die grundsätzlichen Strukturen einer Anwendung zu verstehen und zu beschreiben (vermeiden „Chaos“).**
 - Liefern Ansätze zur sinnvollen Zerlegung des Systems
 - Liefern Ansätze zur sinnvollen Verteilung von Aufgaben
 - Liefern Ansätze zur Verwendung geeigneter Konnektoren
- **Beschreiben die wesentliche Merkmale der inneren Struktur einzelner Komponenten.**
 - Zentrale Klassen und Interfaces
 - Beschreiben Details zur Umsetzung einer effektiven Kommunikation

**Component and
Connector View**

**Vorlagen zur
Strukturierung**

Module View

**Vorlagen zur
Implementierung**

- **Liefern Lösungen für ganze Klassen von Systemen.**
 - Architekturen für verteilte Systeme
 - Architekturen für interaktive Systeme
 - Architekturen für adaptierbare Systeme
 - ...

Die Auswahl der richtigen Architekturmuster ist bestimmt durch die nicht-funktionalen Anforderungen!

Ziele:



...



...



...

Ziel:

Wartbarkeit, Erweiterbarkeit, Wiederverwendbarkeit, wenig Kopplung

- **(S) Single Responsibility Principle:**
Jede Klasse/Komponente hat nur eine Verantwortlichkeit
- **(O) Open/Closed Principle:**
Offen für Erweiterung, geschlossen für Modifikation
- **(L) Liskov Substitution Principle:**
Subklassen sollen Basisklassen ohne Probleme ersetzen können
- **(I) Interface Segregation Principle:**
Viele spezifische Interfaces statt wenige "God-Interfaces"
- **(D) Dependency Inversion Principle:**
Abhängigkeiten von Abstraktionen, nicht von konkreten Implementierungen

Prinzipien

Aufgabe:

- Was bedeutet dies für den Entwurf einer Architektur?
- Auf was muss man achten?
- Was sollte man vermeiden?

**Wann folgt ein Entwurf diesen Prinzipien?
Lässt sich dies messen?**

+IKA Domain-Driven Design (DDD)

[Eric Evans: Domain-Driven Design -- Tackling Complexity in the Heart of Software. 2003]

Die Fachdomäne ist der Ausgangspunkt für die Zerlegung in Komponenten. Die Domäne und die damit verbundenen Aufgaben stehen im Vordergrund. Nicht die Feinheiten einer technischen Realisierung.

Dies erfordert:

- Eine enge Zusammenarbeit zwischen fachlichen und technischen Experten.
- Eine gemeinsame Sprache, um sich auszutauschen.
- Eine präzise Modellierung der Fachkonzepte.
- Eine Isolation und klare Abgrenzung der unterschiedlichen Aufgaben.

Die resultierende Architektur muss die fachliche Struktur des Problems widerspiegeln.

- **Isolating the Domain:**

Klar abgegrenzte fachliche Teilbereiche. Diese Teilbereiche (Komponenten) realisieren die damit verbundene Geschäftslogik.

- Spezifische Datenmodelle
- Spezifische Logik
- Spezifische Schnittstellen

**Die Komponenten sind in sich kohärent
und nach außen unabhängig:
hohe Kohäsion und lose Kopplung.**

+IKA DDD – zentrale Prinzipien

- **A Model Expressed in Software:**

Die Beziehungen der Domäne spiegeln sich in der Realisierung der Software wider. Packages, Klassen, Objekte, Assoziationen und Operationen sind klar strukturiert.

- Wer kennt wen?
- Wer übernimmt welche Aufgabe?

- **The Lifecycle of a Domain Object:**

Jedes Objekt hat einen Lifecycle. Objekte entstehen und verschwinden wieder.

- **Application Scoped:**

Unabhängig vom Anwender und unabhängig von einer Aufgabe.

- **Session Scoped:**

Abhängig vom Anwender und oftmals auch von einer spezifischen Aufgabe.

- **Request Scoped:**

Nur relevant für die Realisierung einer spezifischen Funktion.

Aufgabe:

- Worin unterscheiden sich die beiden Prinzipien?
- Kann man beiden Prinzipien gleichzeitig gerecht werden?
- Gibt es Szenarien, die eindeutig für die Anwendung eines dieser Prinzipien sprechen?

Sollte man diese Prinzipien immer anwenden?

Es gilt die Objekte des Anwendungsgebiets zu organisieren und mit den Anforderungen an Performance, Flexibilität, Skalierbarkeit usw. in Einklang zu bringen.

Komponenten, die Verwaltungs- und Hilfsfunktionalität bereitstellen, müssen integriert werden.

Die Fundamente moderner Architekturen.

- **Layers:**

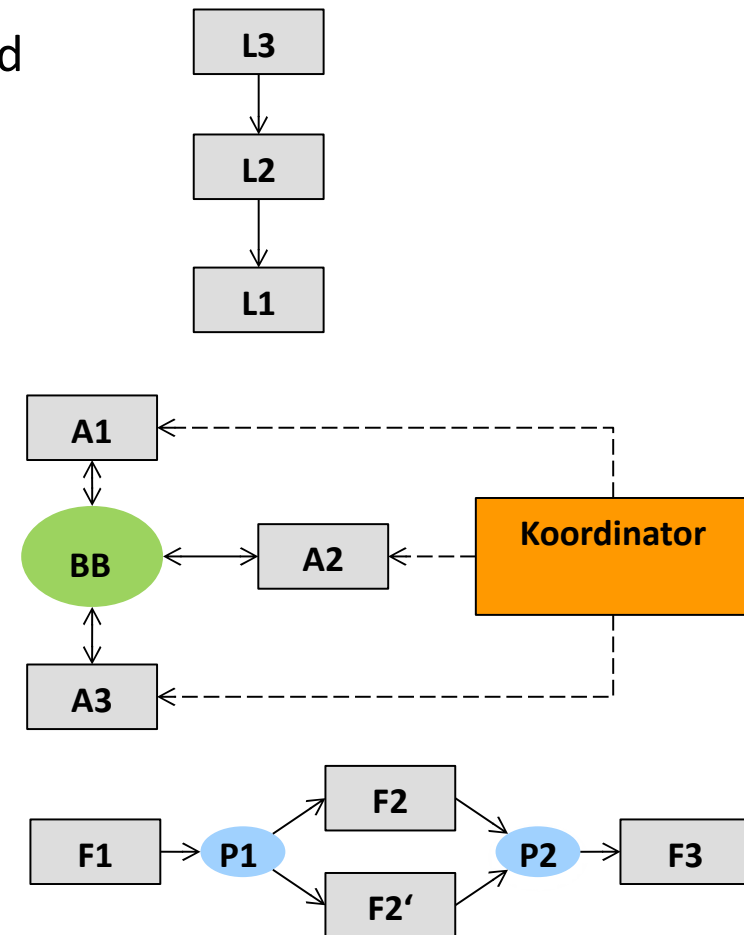
Abstraktionsebenen schaffen Verantwortlichkeiten und Hierarchien helfen bei der Organisation und Strukturierung von Aufgaben.

- **Blackboards:**

Sind die Basis für eine kooperative Problemlösung. Hochspezialisierte Komponenten erarbeiten gemeinsam und koordiniert eine Lösung.

- **Pipes-and-Filters:**

Trennen Konnektoren (Pipes) von Komponenten (Filter) und ermöglichen eine kontinuierliche Verarbeitung der Daten.





Software-Architekturen

—

Layers – Adapters – Onions

Prof. Dr. Thomas Fuchß

Hochschule Karlsruhe

University of
Applied Sciences

Fakultät für

**Informatik und
Wirtschaftsinformatik**

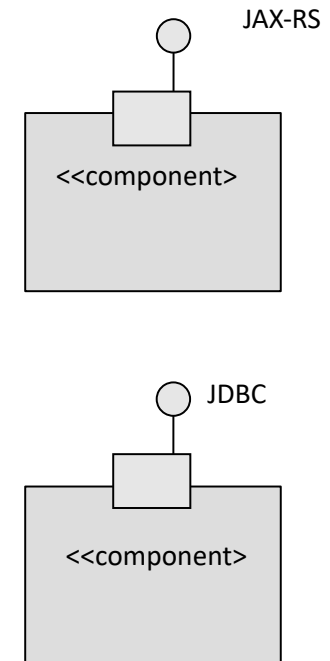
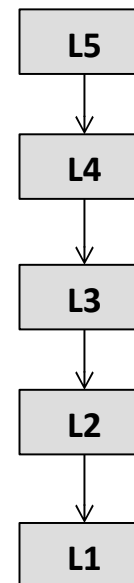
+IKA Layers (Schichten)

[Frank Buschmann: (Pattern-Oriented Software Architecture) Band: 1 A System of Patterns – John Wiley & Sons, 1996.]

Ziel einer Schichtung ist es, eine Anwendung so zu strukturieren, dass Teilaufgaben voneinander abgetrennt werden, die auf unterschiedlichen Abstraktionsebenen liegen.

**Hohe Schicht, hoher
Abstraktionsgrad**

**Niedere Schicht, niederer
Abstraktionsgrad**



+IKA Layers (Schichten)

IDEE:

Es ist besser, etwas modular zu implementieren als in einem monolithischen Block. Dinge, die bereits konzeptionell verschieden sind, sollten auch getrennt realisiert werden.

- Layer können unabhängig entwickelt werden.
- Layer können inkrementell entwickelt werden.
- Layer können ausgetauscht werden.
- ...



Eine klare Hierarchie muss erkennbar sein.

Frage: Was bedeutet Hierarchie?

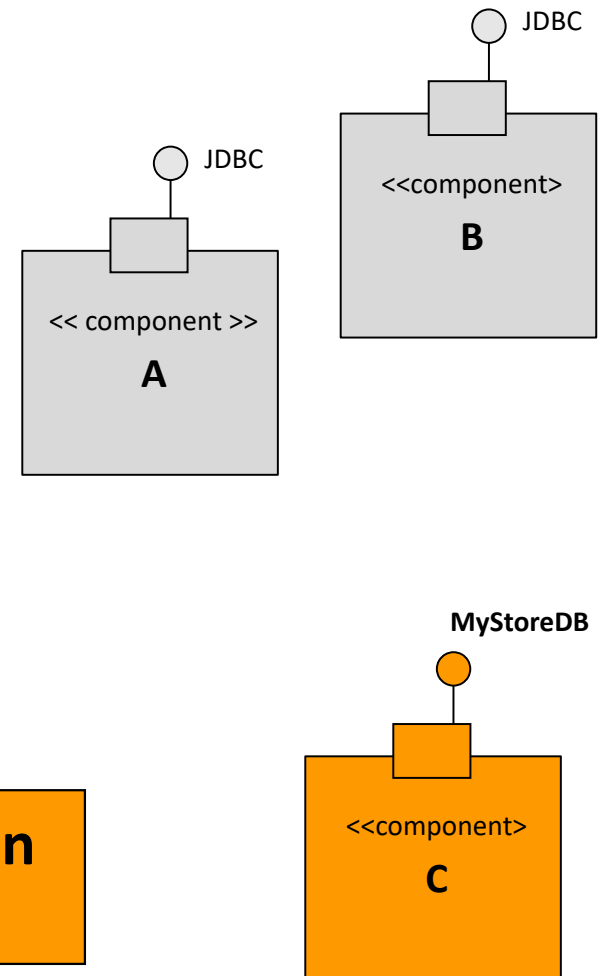
+IKA Layers: Übergeordnete Ziele!

- Teile des Systems müssen austauschbar sein, ohne dass dies große Wellen schlägt.
- Späte Änderungen dürfen sich nicht fortpflanzen.



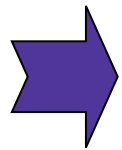
**Schnittstellen müssen stabil sein
(standardisiert).**

**Achtung: Die Aufgaben der jeweiligen Schichten
sind oftmals problemabhängig.**



+IKA Layers: Was muss beachtet werden?

- Höhere Schichten verwenden tiefere Schichten, **niemals umgekehrt!**



Dies muss bei der Umsetzung von Konnektoren beachtet werden.

- **Auf niederen Stufen:** Sensormessungen, Behandlung von Hardware-Fehlern,
 - **Auf höheren Stufen:** Paketieren von Daten, Chiffrieren,
 - **Auf hohen Stufen:** Aufbereiten für die Visualisierung,
- Auch innerhalb derselben Schicht können Aufgaben voneinander unabhängig sein.



Partitionen: Eine zusätzliche horizontale Zerlegung auf der gleichen Abstraktionsebene (Schicht).

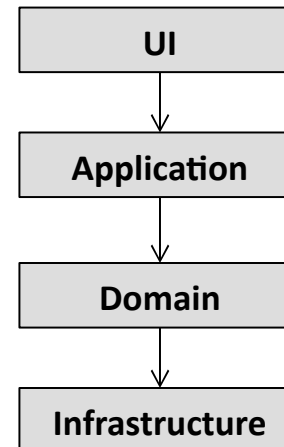
Achtung: Auch Partitionen sollten sich nicht gegenseitig nutzen.

+IK A Layers: Was muss beachtet werden?

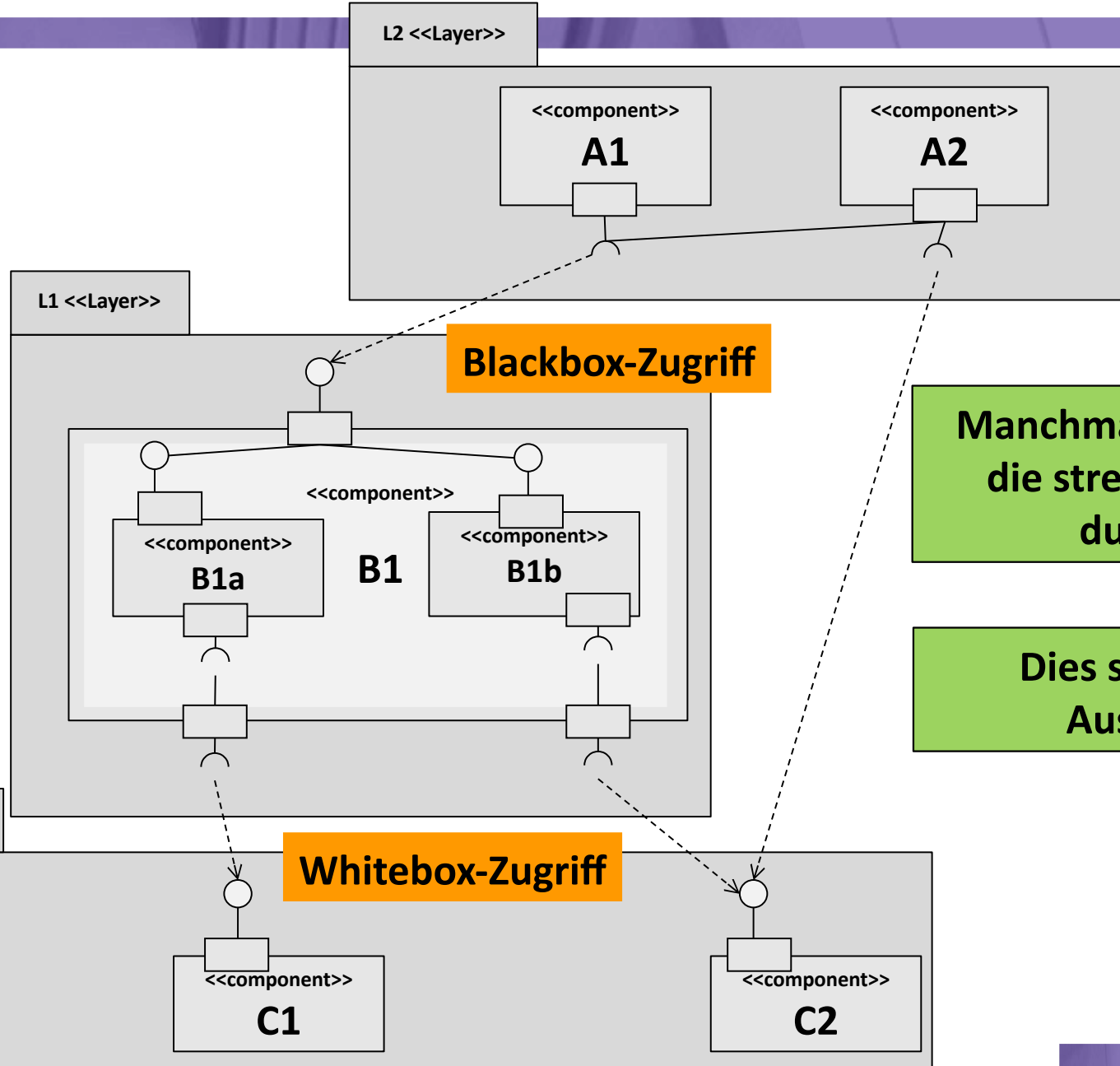
- Ähnliche Aufgaben sollten auf ähnlichen Abstraktionsebenen liegen.
- Alle Komponenten einer Schicht haben denselben Abstraktionsgrad.
- Die Granularität muss richtig gewählt werden.
- Komplexe Komponenten müssen eventuell weiter zerlegt werden.
- Die Performance muss erhalten bleiben.
(Datenübertragung über Grenzen hinweg kann teuer werden).

**Hohe Schicht, hoher
Abstraktionsgrad**

**Niedere Schicht, niederer
Abstraktionsgrad**



+IKA Layers: Umsetzung

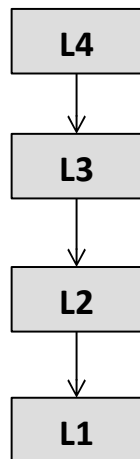


Manchmal ist es angebracht die strenge Hierarchie zu durchbrechen.

Dies sollte jedoch die Ausnahme sein!

+IKA Layers: Dynamische Aspekte

- **Top-Down delegieren:**
Anfragen an Schicht N+1 werden unter Zuhilfenahme von Funktionalität aus Schicht N realisiert.
- **Bottom-Up informieren:**
Ereignisse, die in Schicht N auftreten, werden an Schicht N+1 gemeldet.



Herausforderung: Schicht N darf N+1 nicht kennen.

Die Schnittstellen zur Weitergabe von Informationen müssen auf den tieferen Schichten definiert werden.

Indirekte Benachrichtigung, Callbacks.

+IKA Layers: Implementierung

- Abstraktionskriterien für die Schichten festlegen.
 - sichtbare Elemente,
 - anwendungsspezifische Module,
 - allgemeine Dienste,
 - Betriebssystemschnittstellen,
 - Hardwareschnittstellen
- Anzahl der Abstraktionsebenen (Schichten) bestimmen.
- Schichten benennen und jeder Schicht ihre Aufgaben zuweisen.



Die Funktionalität, die die oberste Schicht anbietet, decken die Aufgaben des Gesamtsystems ab.

+IKA Layers: Implementierung

- Aufgaben festlegen:

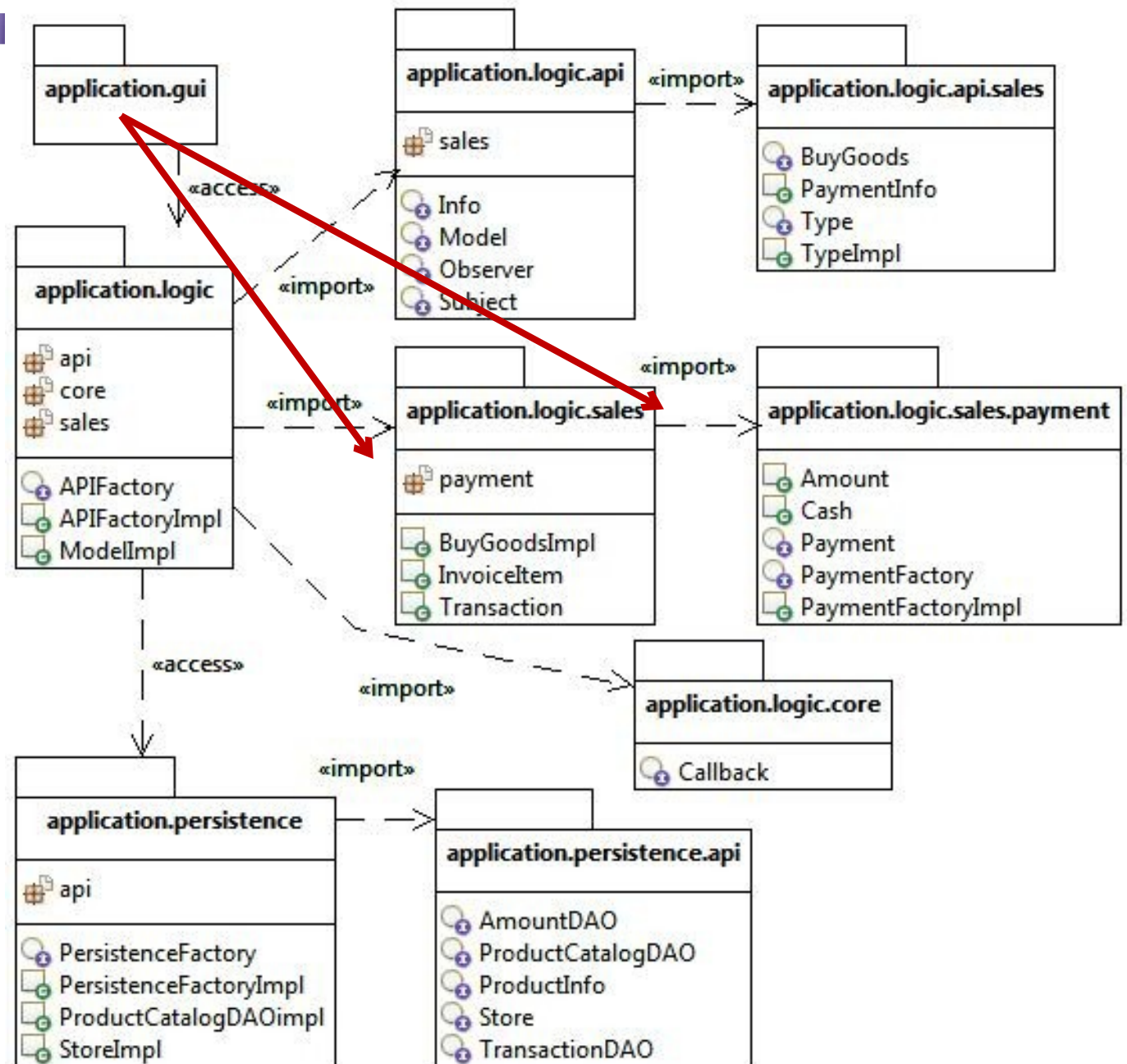
- Keine Komponente erstreckt sich über mehrere Schichten.
- Alle verwendeten Klassen und Operationen in einer Schicht werden entweder vom SDK oder von einer tieferen Schicht bereitgestellt.

- Schnittstellen spezifizieren:

- Ein Blackbox-Ansatz ist stets einem Whitebox-Ansatz vorzuziehen.
- Ein Blackbox-Ansatz ist oftmals nur mit Disziplin erreichbar.

+IKA Layers: Implementierung

Wer verhindert
den Zugriff?



+IKA Layers: Implementierung

- C, C++

**Auf die Schichtung zugeschnittene
Header Files.**

- Java

**Konzepte wie CDI und die Frameworks,
die sie anbieten.**

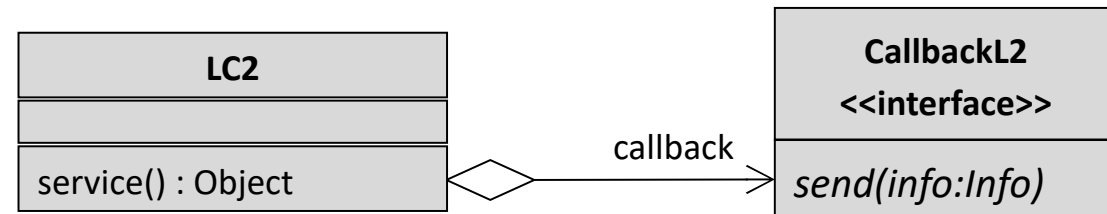
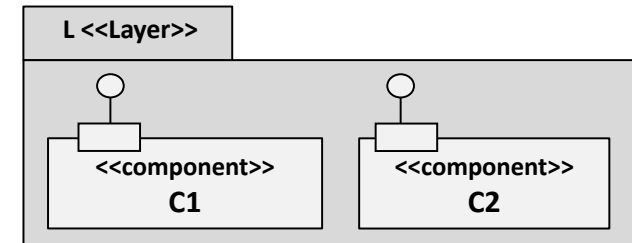
```
public class L3C {  
  
    @Inject  
    private L2I obj;  
  
}
```

```
public interface L2I {  
    // ...  
}
```

```
@ApplicationScoped  
class L2C implements L2I, Serializable{  
    // ...  
}
```

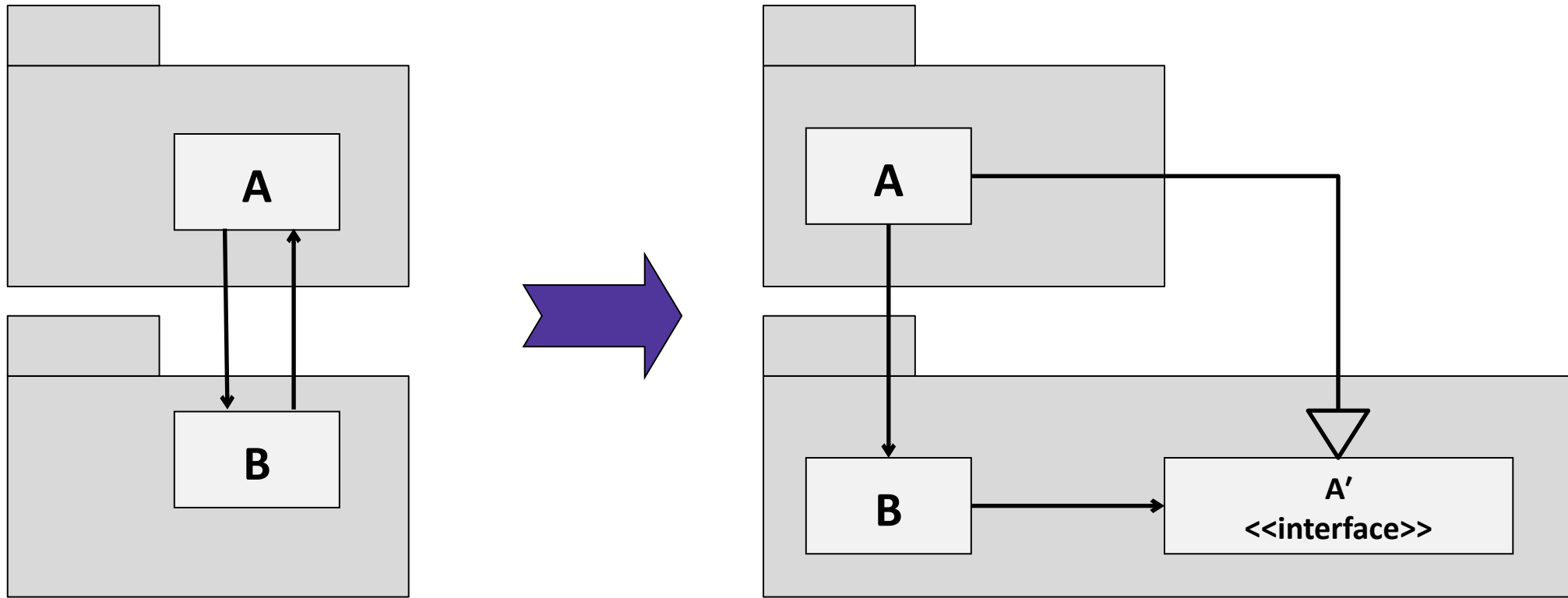

Layers: Implementierung, die nächsten Schritte

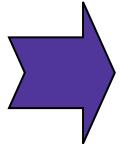
- Die einzelnen Schichten strukturieren und Partitionen bilden.
- Kommunikation zwischen den Schichten N+1 und N definieren:
 - pull: N+1 holt sich die notwendige Information aus N (Methodenaufruf)
 - push: N reicht Informationen an N+1 weiter (Callbacks)



- Schichten entkoppeln:
Schicht N+1 kennt N aber nicht umgekehrt. Schicht N definiert die für die Informationsweitergabe notwendigen Callbacks und Nachrichtenobjekte.
- Fehlerbehandlung:
Fehler sollten in der Schicht behandelt werden in der sie auftreten.

+IK A Layers: Schichten entkoppeln





**Schichten organisieren ein System vertikal,
vergleichbar einem mehrstufigen
Client-Server-System.**

- **Jede Schicht stellt ihre Funktionalität als Schnittstelle der über ihr liegenden Schicht zur Verfügung.**
- **Jede Schicht ist „Client“ und „Server“:**
 - Client aus Sicht der tieferen Schichten
 - Server aus Sicht der höheren Schichten
- **Konnektoren verbinden die einzelnen Schichten:**
 - Vom einfachen Methodenaufruf
 - Über Remote Procedure Calls
 - Bis hinzu komplexen Kommunikationsprotokollen und dedizierten Messaging-Services.

+IKA Layers: Essentials

Vorteile:

- Wiederverwendung von Schichten.
- Änderungen betreffen (im Idealfall) nur die darüber- und die darunterliegende Schicht.
- Schichten werden austauschbar.

Nachteile:

- Die richtige Granularität der Schichtung lässt sich oft nur schwer bestimmen.
- Änderungen bleiben oftmals nicht lokal und eskalieren nach oben.
- Geringere Effizienz:
 - **Oftmals werden Daten zwischen Schichten unnötigerweise kopiert.**
 - **Oftmals führen niedrigere Schichten Arbeiten durch, die keiner braucht.**

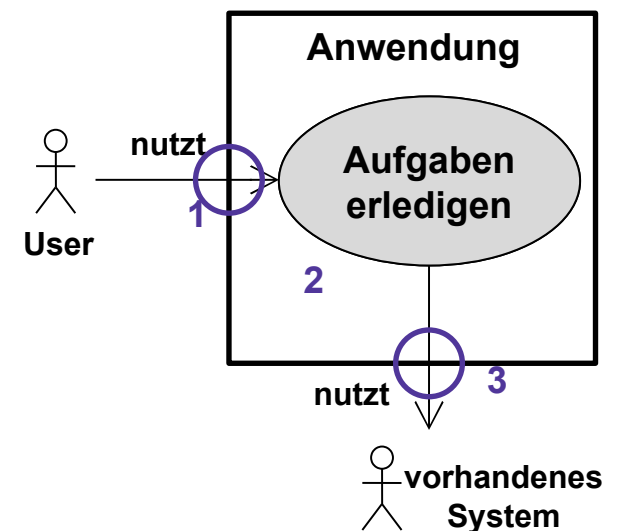
Innerhalb des gleichen Prozessraums kann es von Vorteil sein, trotz verschiedener Layer, auf den gleichen Daten zu arbeiten.

```
typedef TypeL1 TypeL2;
```

+IKA Layers: Drei Schichten

Die folgende Zerlegung ist typisch für fast alle Applikationen – unabhängig davon, ob es sich um ein Client-Server- oder lediglich eine Desktop-Anwendung handelt.

1. **Präsentationsschicht:** Die Präsentationsschicht ermöglicht den Zugriff von außerhalb über Fenster, Dialoge, Web-Seiten, ...
2. **Logikschicht:** Die Logikschicht realisiert die eigentliche Lösung, man spricht von Business-, bzw. Geschäftslogik. Sie wird im Allgemeinen weiter aufgeteilt und strukturiert.
3. **Datenzugriffsschicht / Persistenzschicht:** Die Persistenzschicht realisiert die Anbindung an Drittsysteme, wie Datenbanken, Dateisysteme, Netzwerkdienste, Mail-Systeme usw.



+HKA Layers vs. Tiers

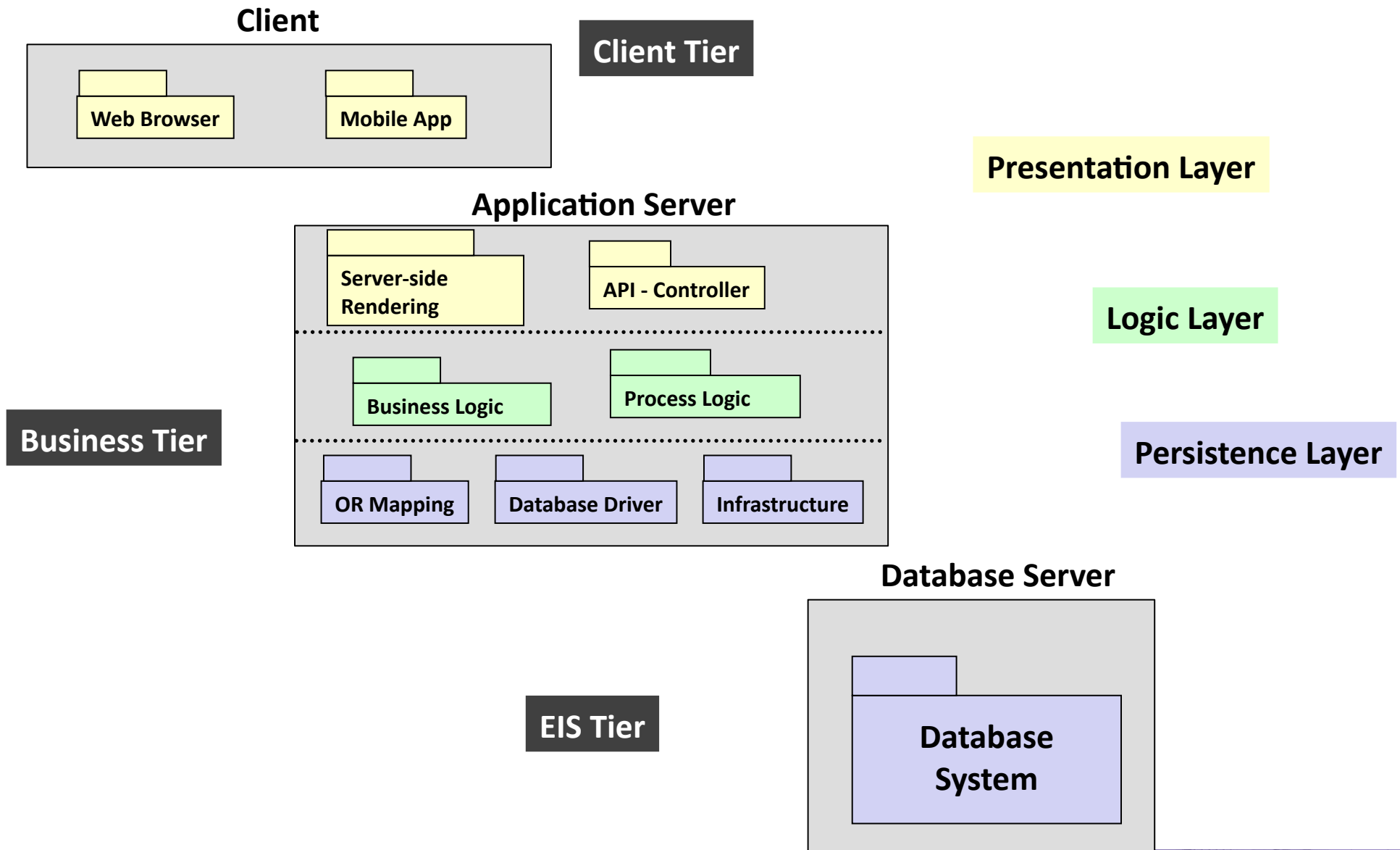
Schichten (Layers) beschreiben die logische Organisation des Systems. Es wird nicht definiert, wo und wie sie sich physisch manifestieren.

Alle Schichten können auf demselben Rechner sein, im selben Prozessraum. Eine Schicht kann aber genauso gut auf mehrere Rechner verteilt werden.

Tiers sind physische Schichten, sie definieren, wo der Code zur Ausführung kommt, sie beziehen sich auf das Deployment.

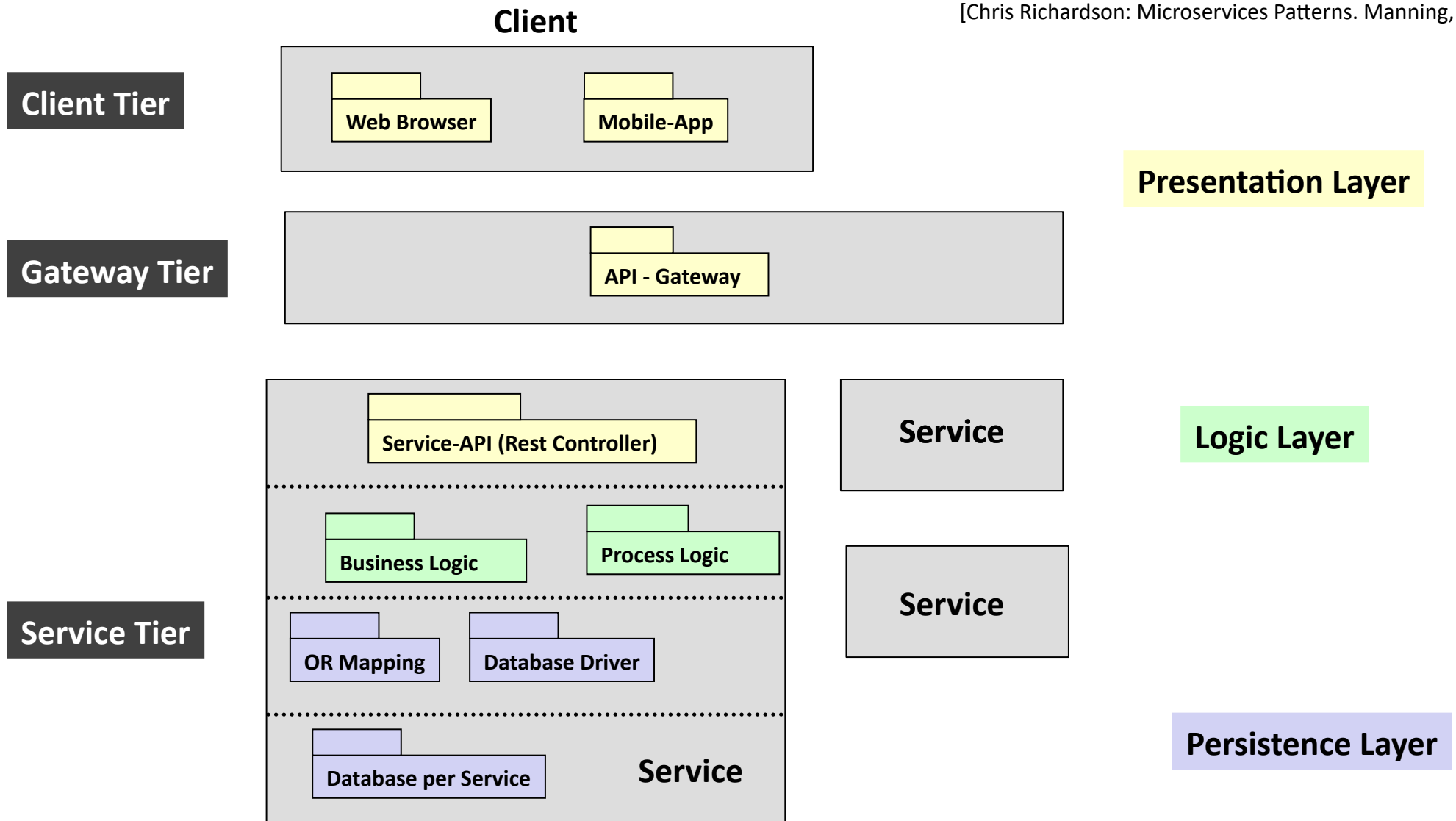
Tiers sind Bestandteile der Physical View bzw. Allocation View.

+IKA Layers vs. Tiers

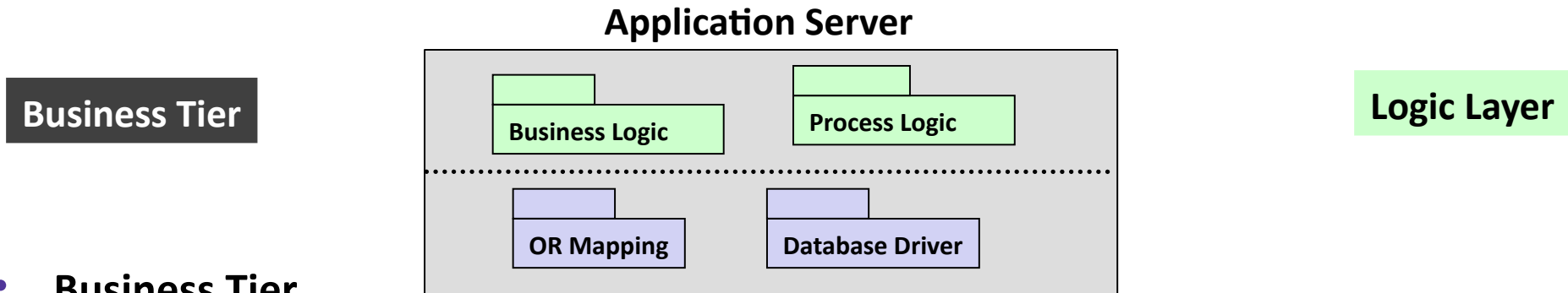


+IKA Layers vs. Tiers

[Chris Richardson: Microservices Patterns. Manning, 2019.]



+IKA Application Server



• Business Tier

- Der physische Kern der Anwendung.
- **Der Application Server ist für diesen Kern verantwortlich.**
 - Verfügt über ein Komponenten- und Container-Modell.
 - Das Container-Modell definiert für die Komponente die Laufzeitumgebung.
 - Die Laufzeitumgebung stellt den Komponenten unterschiedlichste Dienste bereit (von Multi-Threading über Security bis zum Transaktionsmanagement).



Die Anwendungsentwicklung kann sich auf die Anwendung fokussieren.

+IKA Application Server(Eigenschaften)

- **Separation of Concerns:**

Eine klare physische Trennung von Verantwortlichkeitsbereichen.

Client Tier

Business Tier

EIS Tier

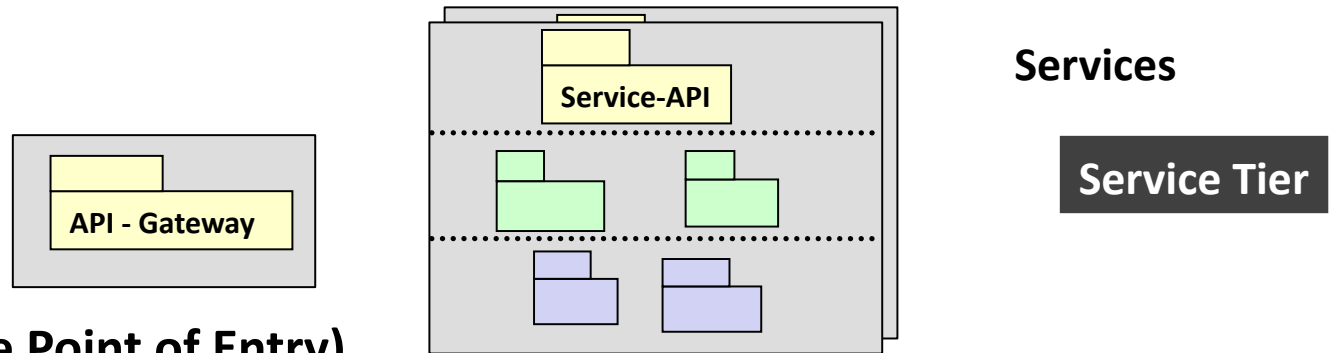
- **Flexibles Deployment:**

Alle Tiers können auf derselben Maschine laufen. Genauso können aber auch alle Tiers auf dedizierten Maschinen laufen. Für die Anwendung macht dies keinen Unterschied.

- **Synchrone Kommunikation**

Die Kommunikation zwischen den einzelnen Tiers erfolgt synchron und ist gerichtet.





- **API-Gateway (Single Point of Entry)**

- Kümmert sich um Security und Routing
- Bietet die Services nach außen an.

- **Microservice (Eine isolierte und klar abgegrenzte Teilaufgabe)**

- Besitzt eigene Logik.
- Besitzt eigene Daten.



Anwendungen bleiben überschaubar. Teilaufgaben können unabhängig entwickelt und gewartet werden.

+HKA Microservices (Eigenschaften)

- **Separation of Concerns:**
Eine klare physische Trennung von Verantwortlichkeitsbereichen (DDD).
- **Flexibles Deployment:**
Services laufen in eigenen Containern und können unabhängig entwickelt und bereitgestellt werden (CI/CD).
- **Herausforderung**
Komplexe Anfragen, die mehrere Services verbinden, müssen in lokale Transaktionen verwandelt werden, die auf lokalen Datenbanken laufen. (Saga, CQRS)

Client Tier

Gateway Tier

Service Tier



+IKA Application Server vs. Microservices

	Application Server	Microservices
Grundidee	eher klassische Schichten	fachliche Trennung (DDD)
Geschäftslogik	zentraler Business Tier	dezentrale Services
Datenhaltung	zentraler EIS Tier	Database per Service
Kopplung	eher stark, aufgrund des zentralen Business Tiers	gering, aufgrund der dezentralen Services
Skalierung	Tier-weise, in der Regel wird der EIS Tier nicht repliziert	Service-weise, eine konsistente Datenhaltung kann komplex werden
Deployment	Tier-weise	Service-spezifisch (CD)
Technologiestack	einheitlich	auch heterogen
Kommunikation	via Netzwerk und lokal innerhalb der Geschäftslogik	via Netzwerk

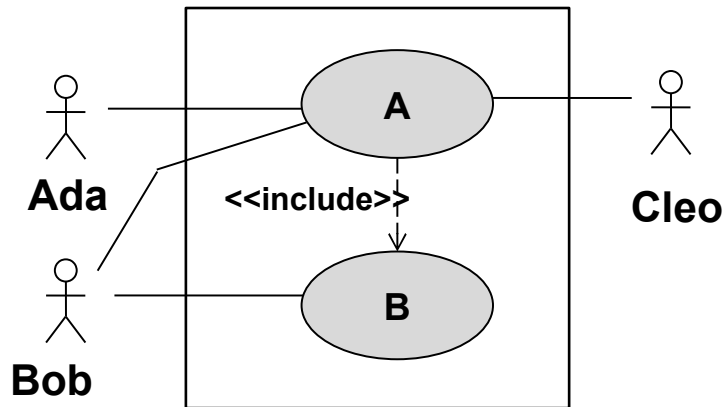
Hexagonal Architecture oder Ports und Adapter

- Komponenten stellen ihre Funktionalität über Ports zur Verfügung und werden mithilfe von Adaptern verbunden.

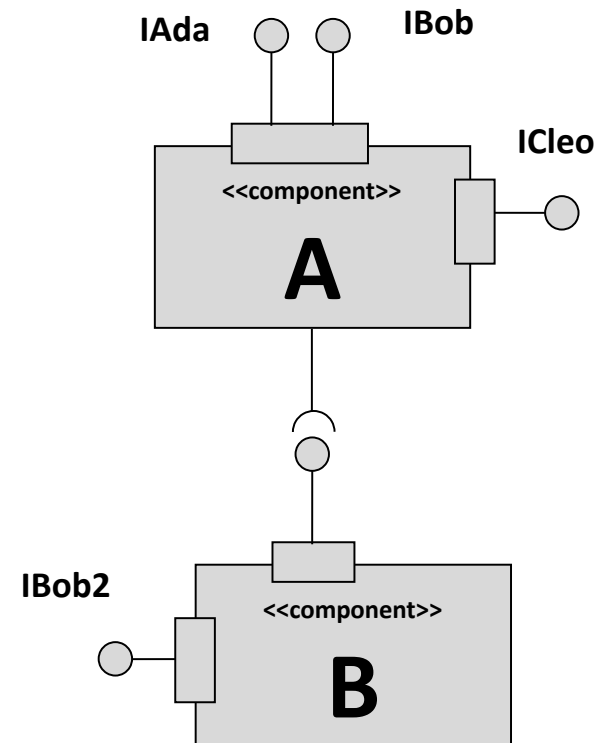
Ziel: Die Aufgaben verbleiben stets in der zuständigen Schicht und werden nicht „verschmiert“!

Keine Business-Logik in der Präsentationsschicht!

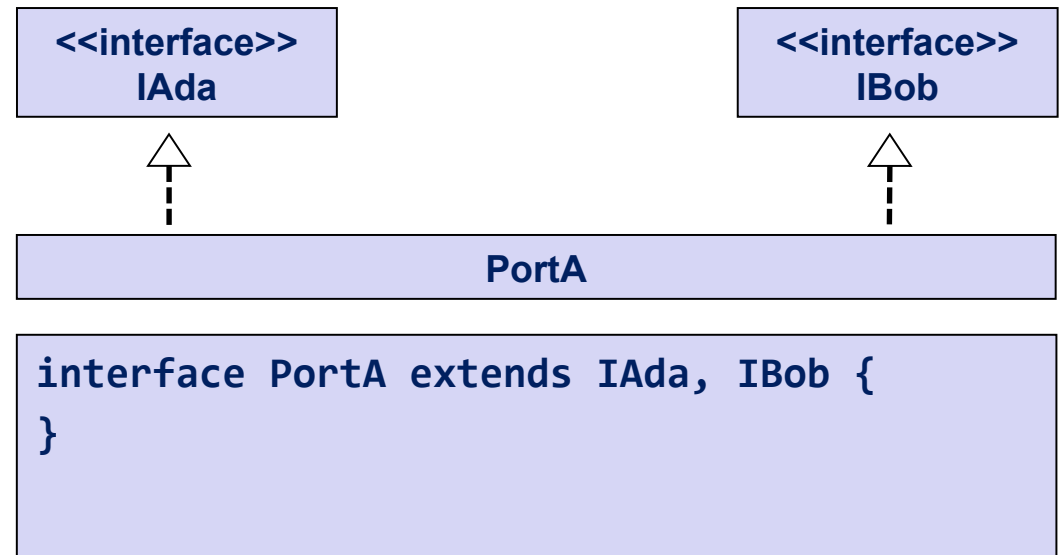
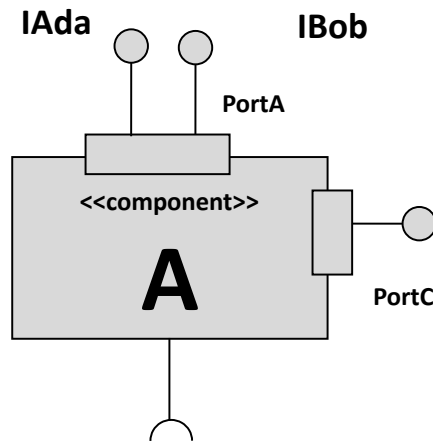
+IK A Ports und Adapter



Um Komponenten zu nutzen, benötigt man Schnittstellen.



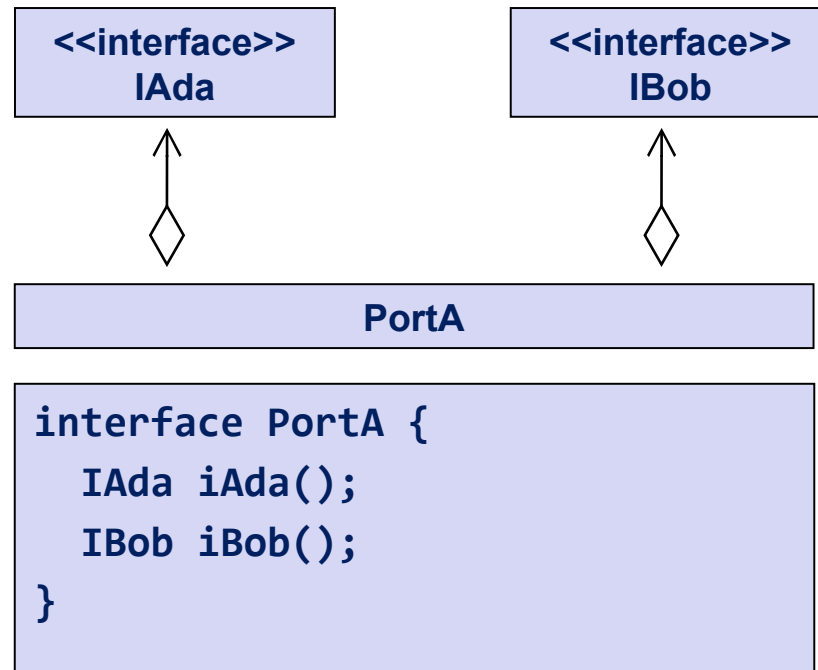
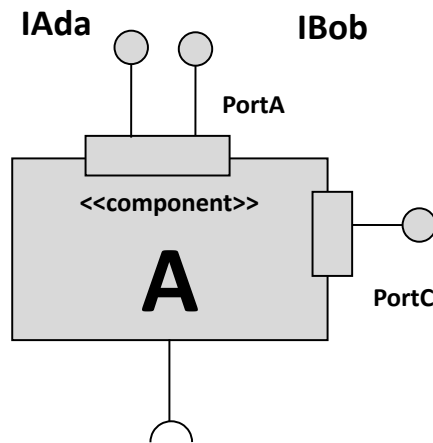
+IKA Ports und Adapter: Realisierung von Ports



Vererbung bedeutet, man legt sich auf eine Implementierung fest!

Der Port ist das Interface!

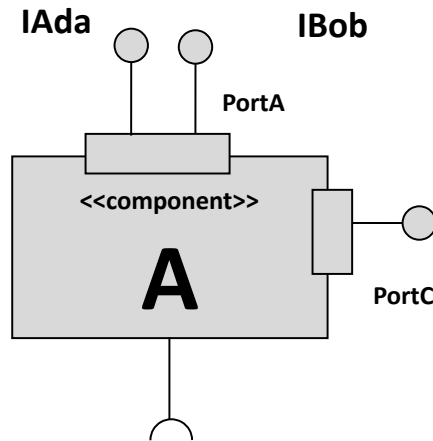
+IK A Ports und Adapter: Realisierung von Ports



Aggregation bedeutet, man nutzt eine definierte Schnittstelle!

Man bezieht das Interface über den Port!

+IK A Ports und Adapter: Fassaden realisieren Ports

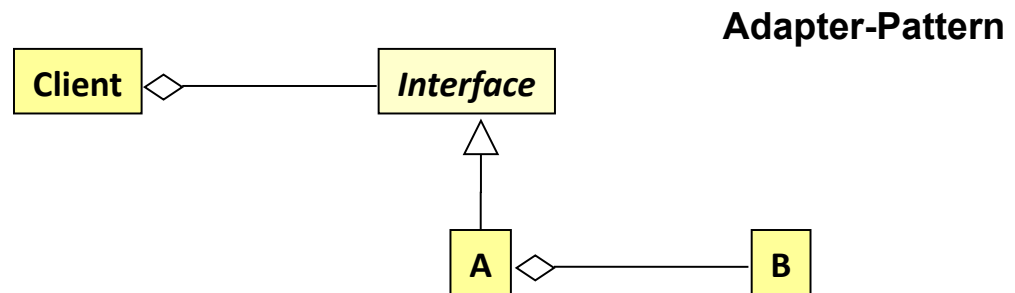


```
interface PortA {  
    IAda iAda();  
    IBob iBob();  
}
```

```
@ApplicationScope  
@Component  
class FacadeA implements PortA {  
  
    @Autowired  
    private PortI portI;  
    ...  
}
```

+IK A Ports und Adapter: Realisierung für „Ada“

Doch wie nutzt Ada den Port?

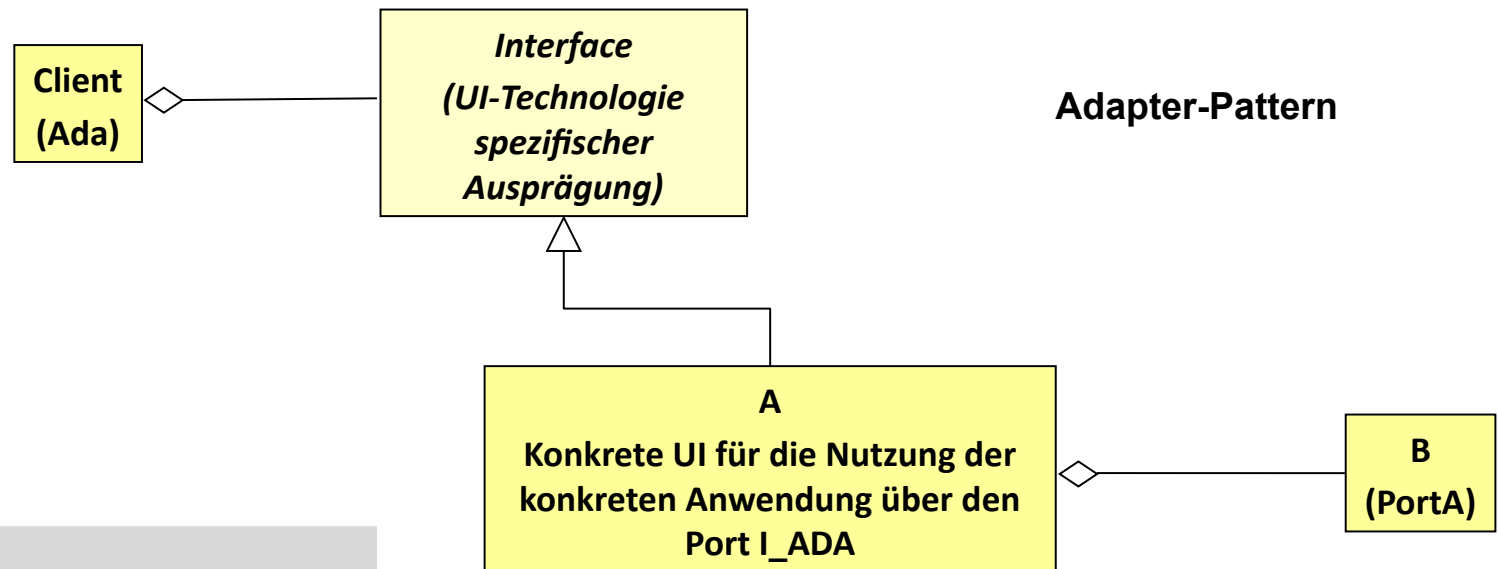


Mithilfe eines Browsers, einer UI, ...

Hierbei handelt es sich um einen spezifischen Adapter, der den Zugriff auf den Port (die Fassade) ermöglicht!

+IK A Ports und Adapter: Realisierung für „Ada“

Doch wie nutzt Ada den Port?



```
@Controller
public class A {
```

```
    @Autowired
    private PortA api;
    private I_Ada ada = api.iAda();
    ...
}
```

```
public class A implements WidgetXY{

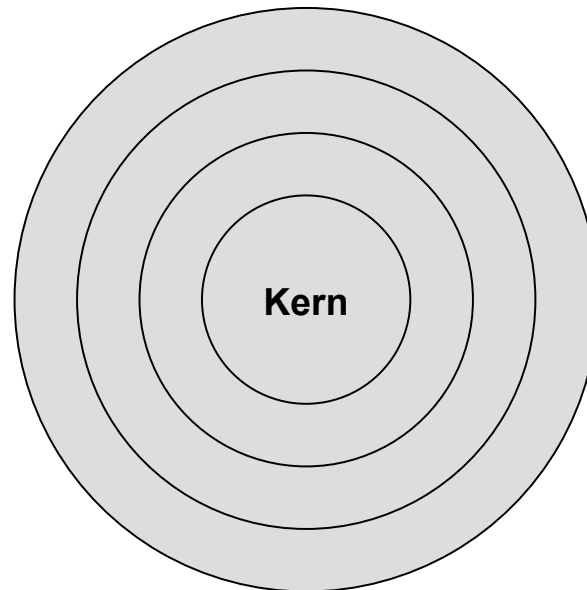
    private PortA api = FactoryB.FACTORY.portI();
    private I_Ada ada = api.iAda();
    ...
}
```

+IKA Layers – Wer nutzt wen?

Stabilität entsteht dadurch, dass man sich nicht immer an neue Gegebenheiten anpassen muss.

Der Kern der Anwendung sollte stabil sein.

Der Rest wird darum gebaut!

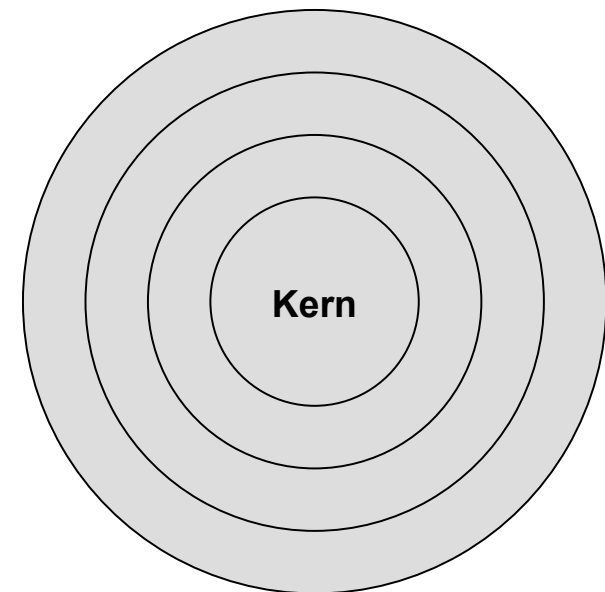
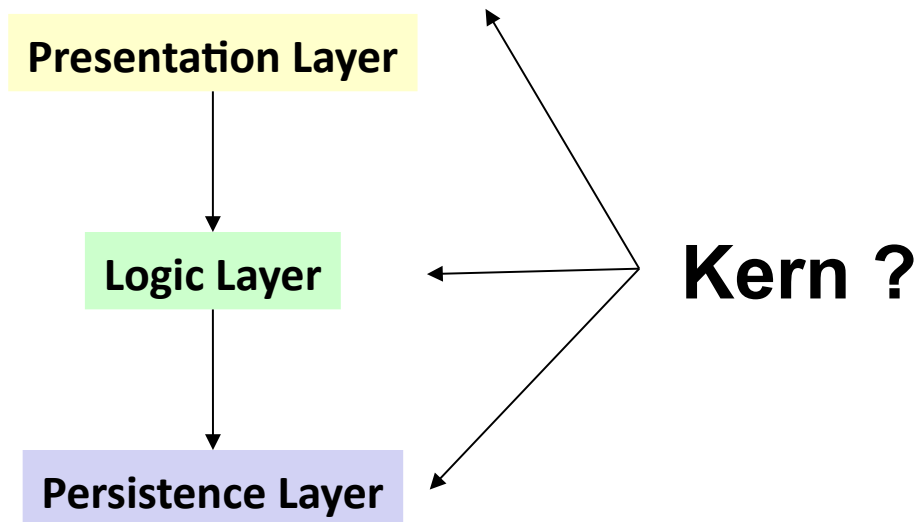


+IKA Layers – Wer nutzt wen?

Stabilität entsteht dadurch, dass man sich nicht immer an neue Gegebenheiten anpassen muss.

Der Kern der Anwendung sollte stabil sein.

Der Rest wird darum gebaut!

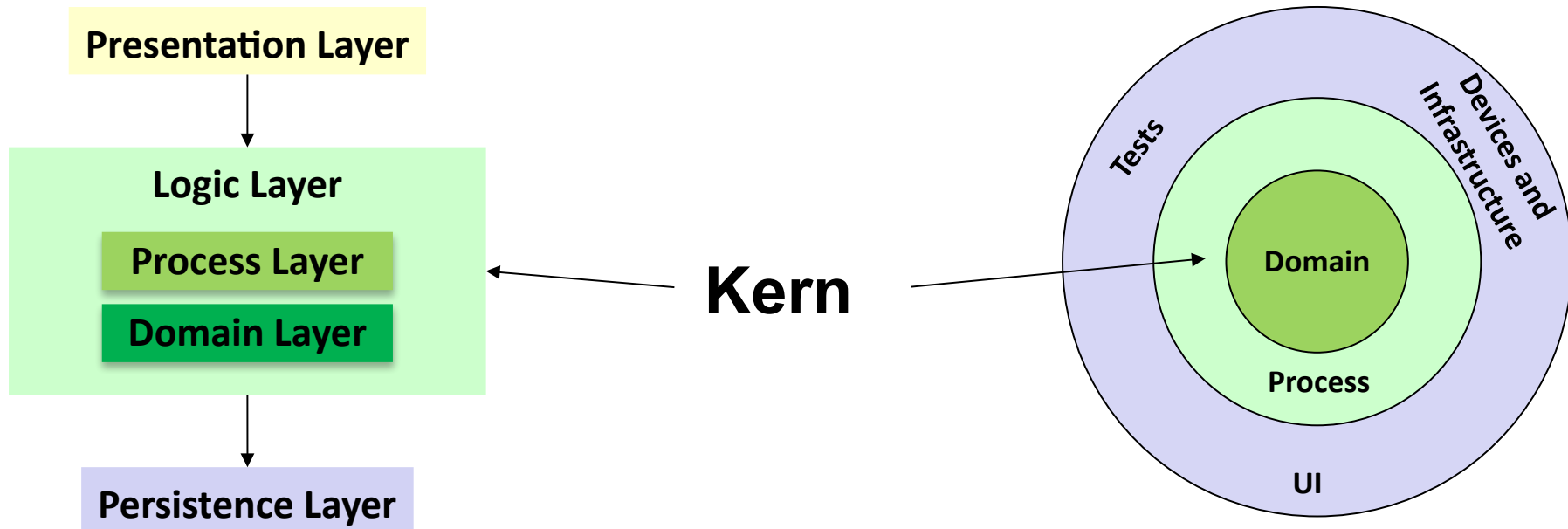


+IKA Layers – Wer nutzt wen?

Stabilität entsteht dadurch, dass man sich nicht immer an neue Gegebenheiten anpassen muss.

Der Kern der Anwendung sollte stabil sein.

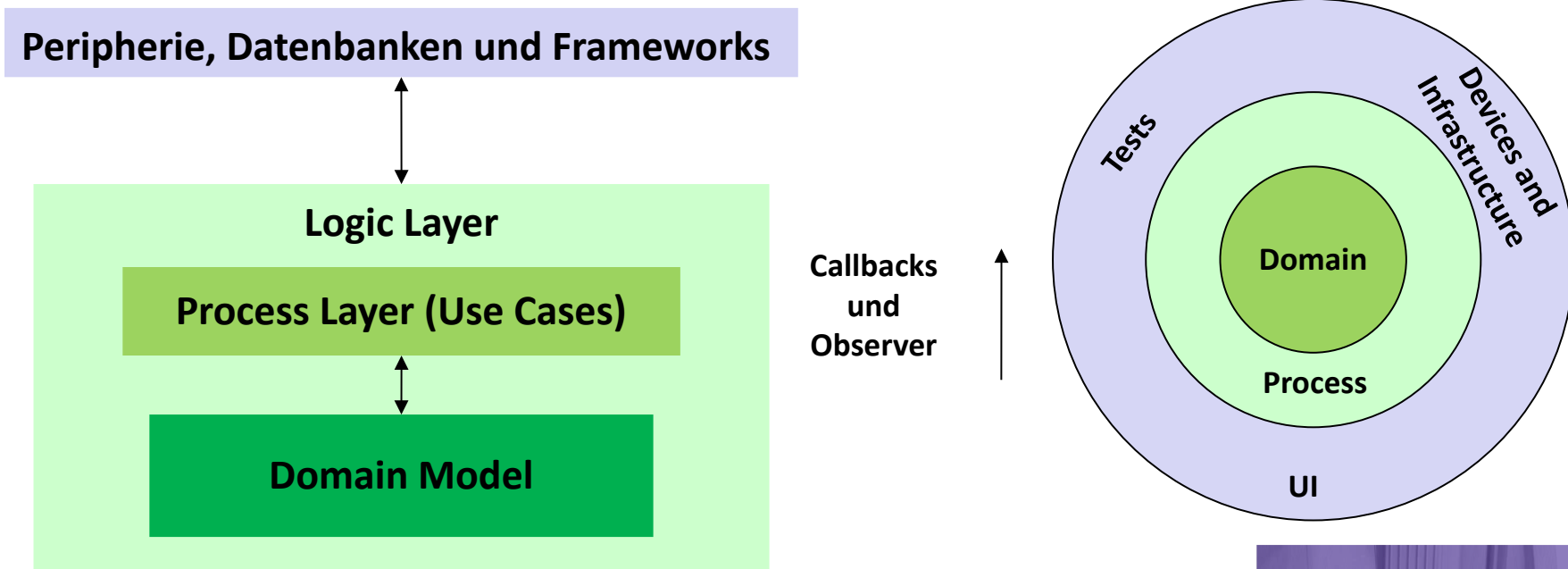
Der Rest wird darum gebaut!



Stabilität entsteht dadurch, dass man sich nicht immer an neue Gegebenheiten anpassen muss.

Der Kern der Anwendung sollte stabil sein.

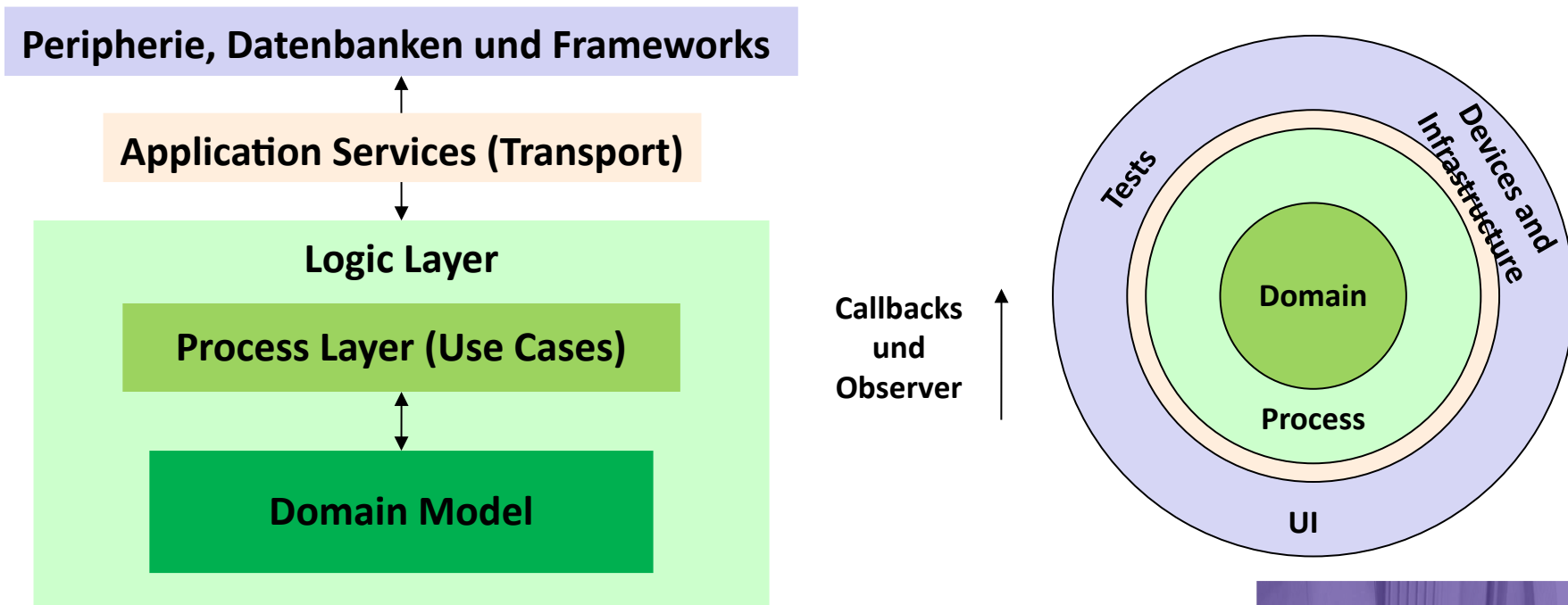
Der Rest wird darum gebaut!



Stabilität entsteht dadurch, dass man sich nicht immer an neue Gegebenheiten anpassen muss.

Der Kern der Anwendung sollte stabil sein.

Der Rest wird darum gebaut!





Software-Architekturen

—

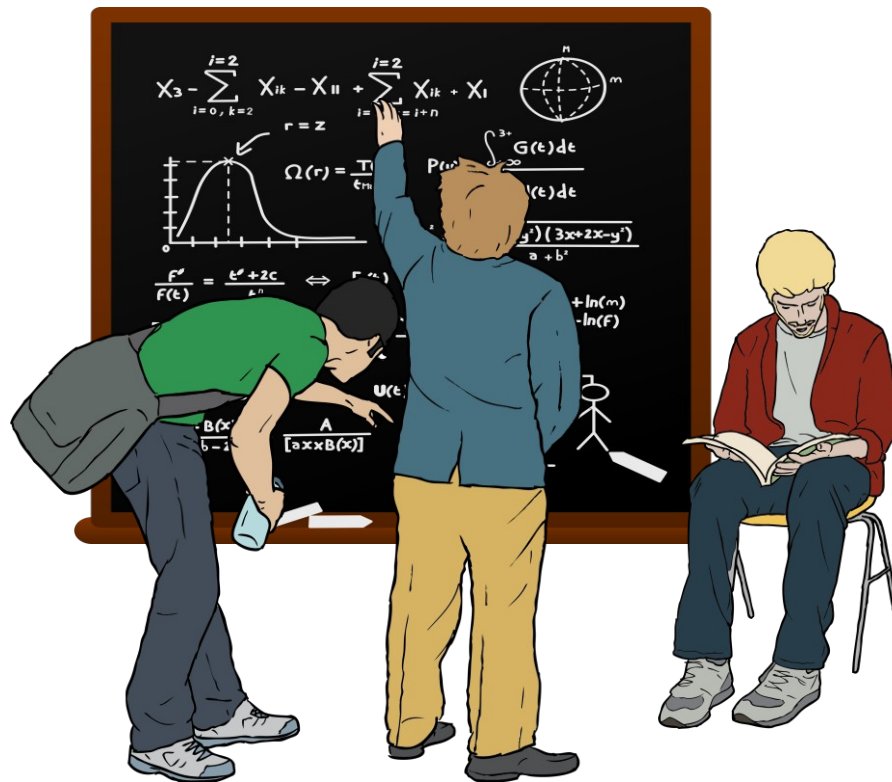
Blackboard und KI

Prof. Dr. Thomas Fuchß

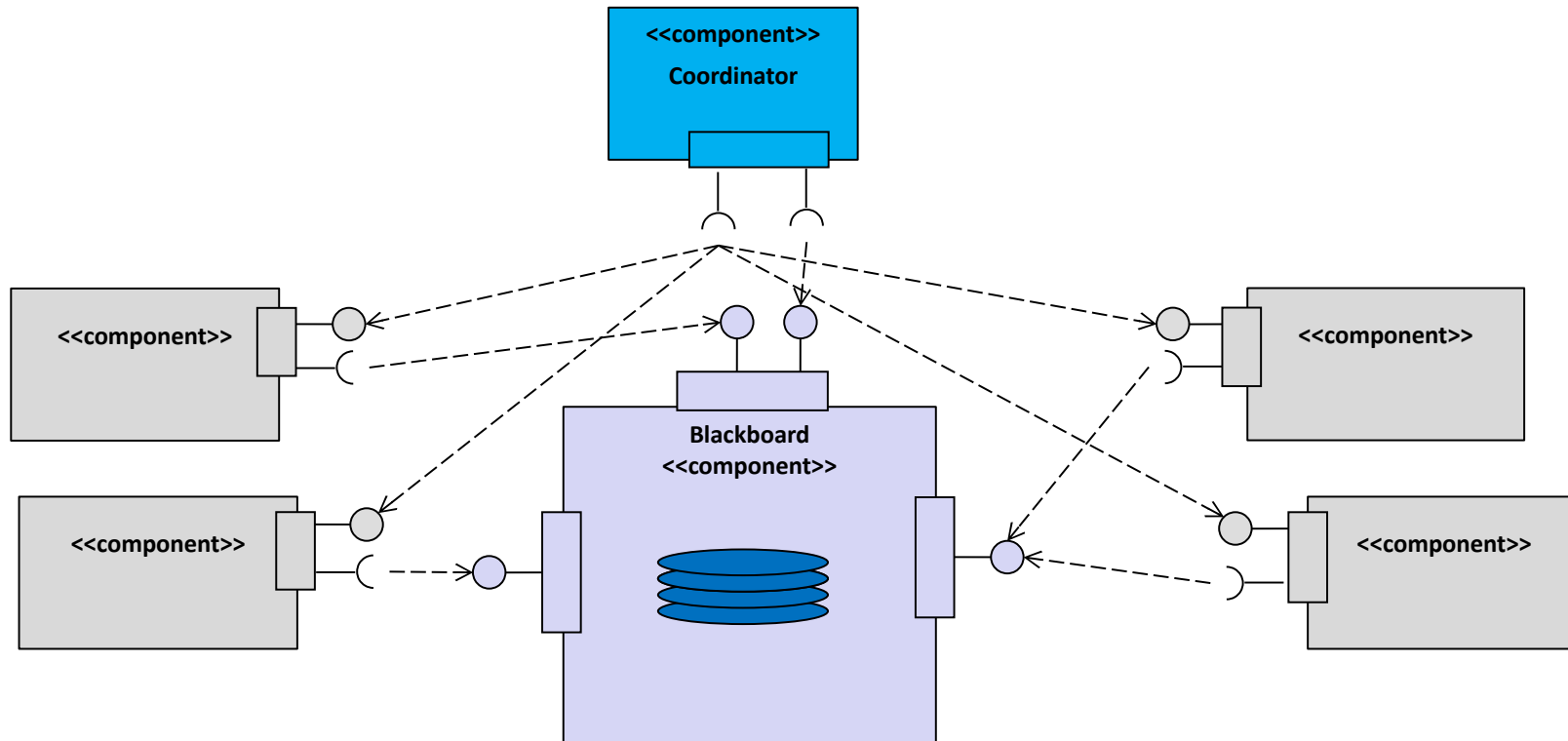
Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
**Informatik und
Wirtschaftsinformatik**

Eigenständige Komponenten erarbeiten gemeinsam und koordiniert eine Lösung auf einer geteilten Datenbasis.



Eigenständige Komponenten erarbeiten gemeinsam und koordiniert eine Lösung auf einer geteilten Datenbasis.



Das Blackboard-Muster zielt auf Systeme, bei denen die Daten im Zentrum stehen. Hochspezialisierte Subsysteme agieren auf diesen Daten und fügen so kontinuierlich „Wissen“ hinzu.

Historisch: Expertensysteme (KI-Anwendungen)

- Probleme zeichnen sich dadurch aus, dass keine deterministische Lösung existiert.
- Es existieren eine Vielzahl **unterschiedlicher Strategien** für Teilprobleme.
- Oftmals reichen bereits suboptimale Lösungen.
- Oft ist es notwendig, mit unscharfem Wissen zu arbeiten.

+IKA Blackboard (Ursprung)

Probleme, für die keine deterministische Lösung existiert.

- Spezialisten bringen ihr Wissen ins Spiel.
- Lieber eine Approximation als gar keine Lösung.
- Auch falsche Wege dürfen beschritten werden.

Vögel können fliegen.

Pinguine sind Vögel.

Pinguine können fliegen.

$$\forall x : V(x) \Rightarrow F(x)$$

$$\forall x : P(x) \Rightarrow V(x)$$

$$\therefore \forall x : P(x) \Rightarrow F(x)$$

+IKA Blackboard (Randbedingungen)

- Ein vollständiges Durchsuchen des Lösungsraums ist inakzeptabel (exponentiell).
- Das Anwendungsgebiet ist noch nicht vollständig verstanden (experimentell).
- Teilprobleme werden durch unterschiedlichste Ansätze gelöst (explorativ).

Zwischenergebnisse werden relevant, Hypothesen müssen aufgestellt, validiert, verworfen und weiterverfolgt werden.



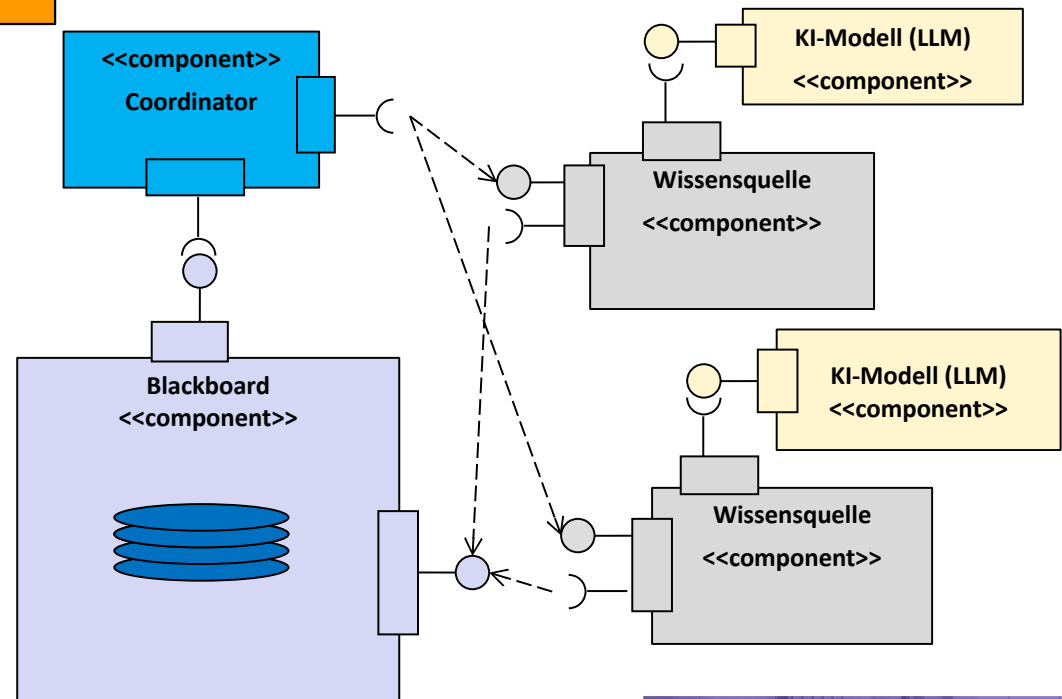
- **Unterschiedliche Lösungsstrategien müssen verfolgt und koordiniert werden.**
- **Ergebnisse benötigen ein einheitliches Format.**

+IKA Blackboard (moderne Sicht)

Das Blackboard-Muster zielt auf Systeme, bei denen die Daten im Zentrum stehen. Hochspezialisierte Subsysteme agieren auf diesen Daten und fügen so kontinuierlich „Wissen“ hinzu.

Modern: KI-Anwendungen

- Wissensquellen als Adapter für KI-Modelle (LLMs).
- Frameworks wie LangChain oder LangGraph bieten erste standardisierte Lösungsbausteine zur Orchestrierung.



+IKA Blackboard (Idee)

Die Grundidee ist eine Sammlung unabhängiger Programme
(Knowledge Sources),
die miteinander zusammenarbeiten.

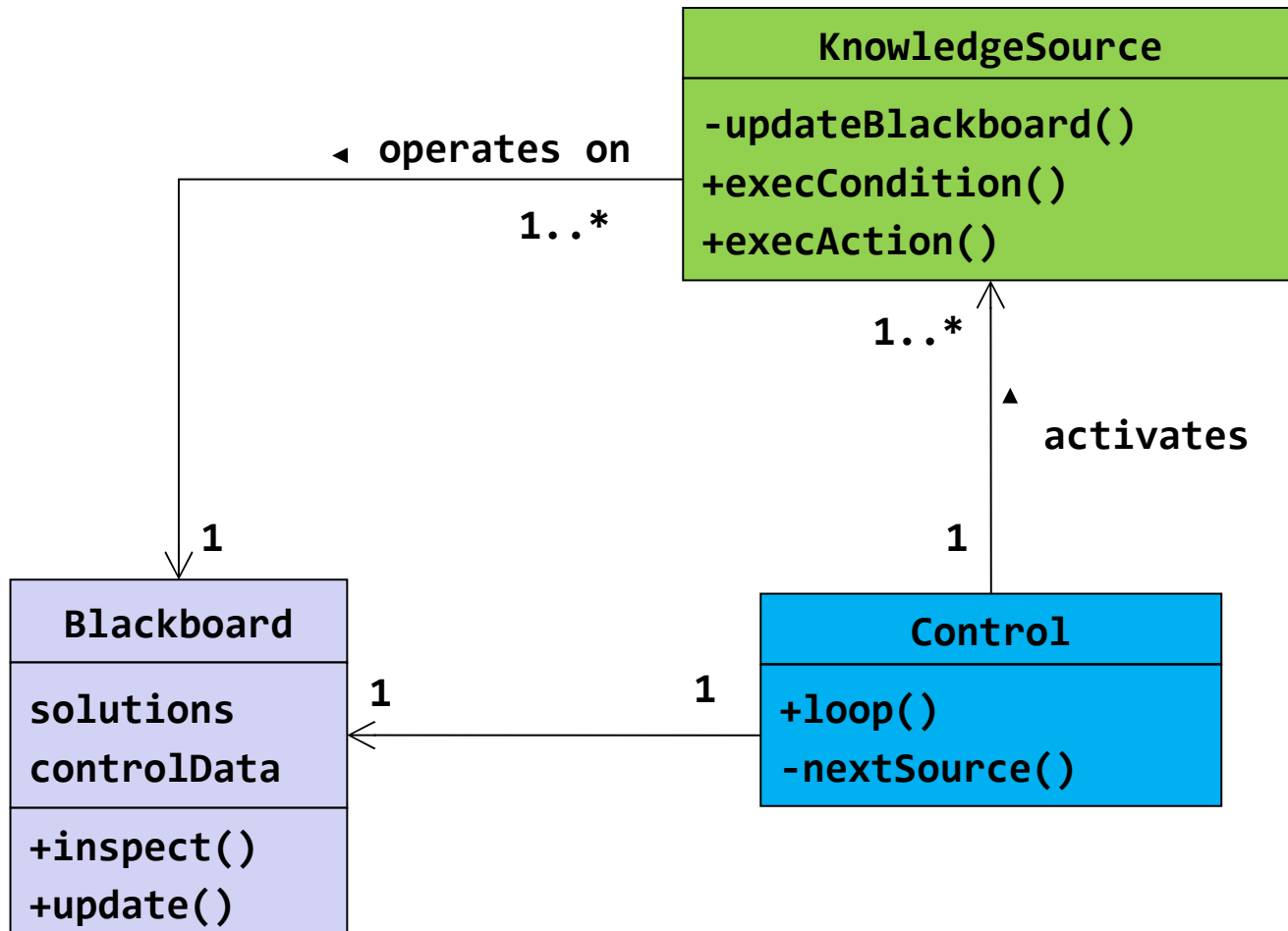
- Jedes Programm ist unabhängig und auf die Lösung eines bestimmten Anteils spezialisiert.
- Eine direkte Zusammenarbeit findet nicht statt.
- Eine zentrale Kontrollkomponente evaluiert den aktuellen Zustand und koordiniert die Programme.
- Deren Teillösungen werden kombiniert, verändert und verworfen.

**Eine Moderator-Komponente bestimmt die
Zusammenarbeit.**

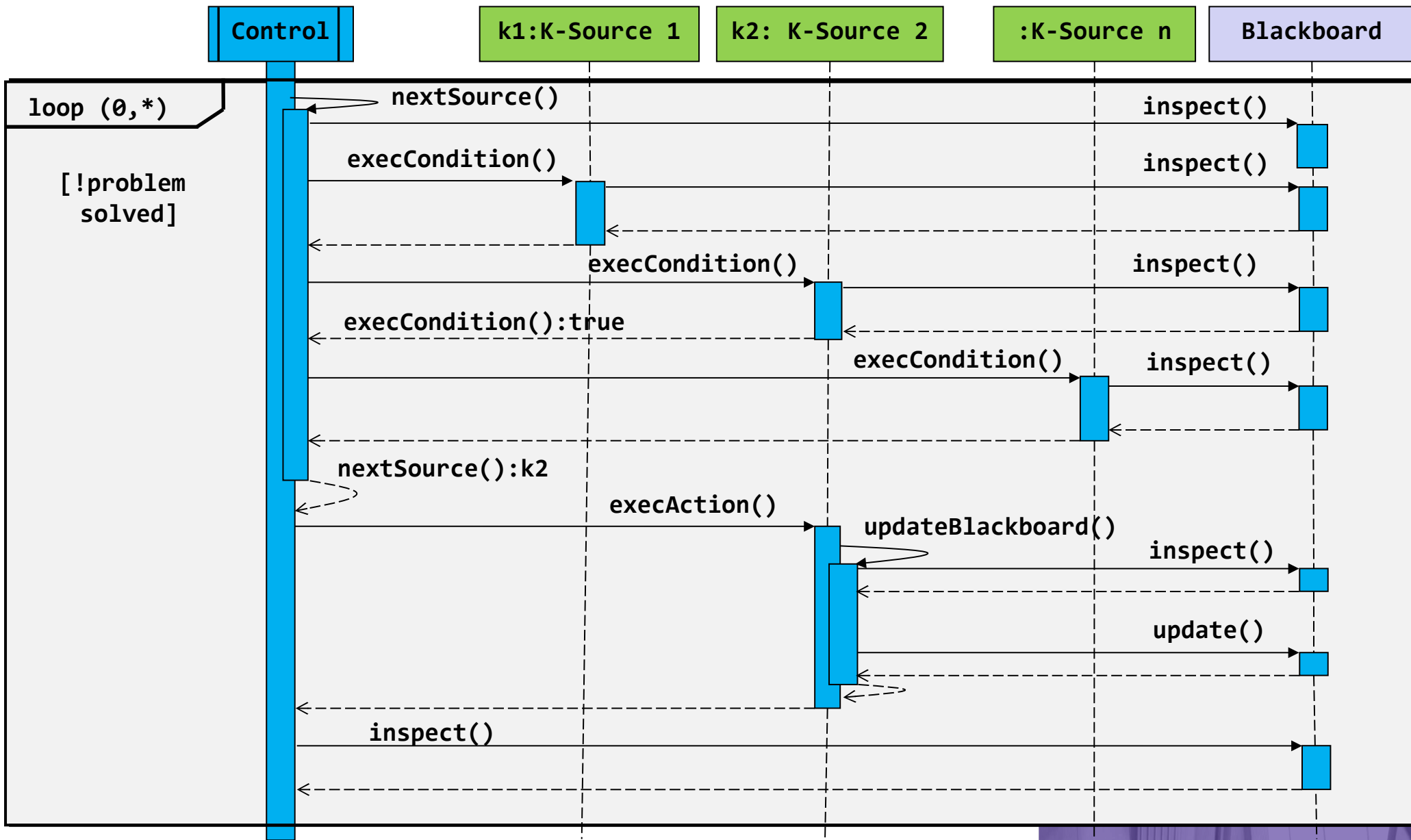
+IKA Blackboard (Struktur)

- **Blackboard**
Es verwaltet alle Daten und Informationen (in einem definierten Format):
 - Elemente des Lösungsraums (Hypothesen) unterschiedlichster Qualität
 - KontrolldatenEs ist das Repository, auf das alle Wissensquellen (Lösungsroutinen) zugreifen.
- **Wissensquellen (Knowledge Sources)**
Sind unabhängige Komponenten, die bestimmte Teile des Gesamtproblems lösen:
 - Evaluieren ihre eigene Anwendbarkeit.
 - Folgen oftmals unterschiedlichen Programmierparadigmen.
 - Berechnen Ergebnisse und aktualisieren das Blackboard.
 - Vorwärtsverkettung: Aus bestehenden Hypothesen werden spezifischere abgeleitet.
 - Rückwärtsverkettung: Bestehende Hypothesen werden validiert oder verworfen.
- **Kontrollkomponente (Control)**
Sie überwacht das Blackboard und bestimmt das Vorgehen, indem Wissensquellen ausgewählt und aktiviert werden.

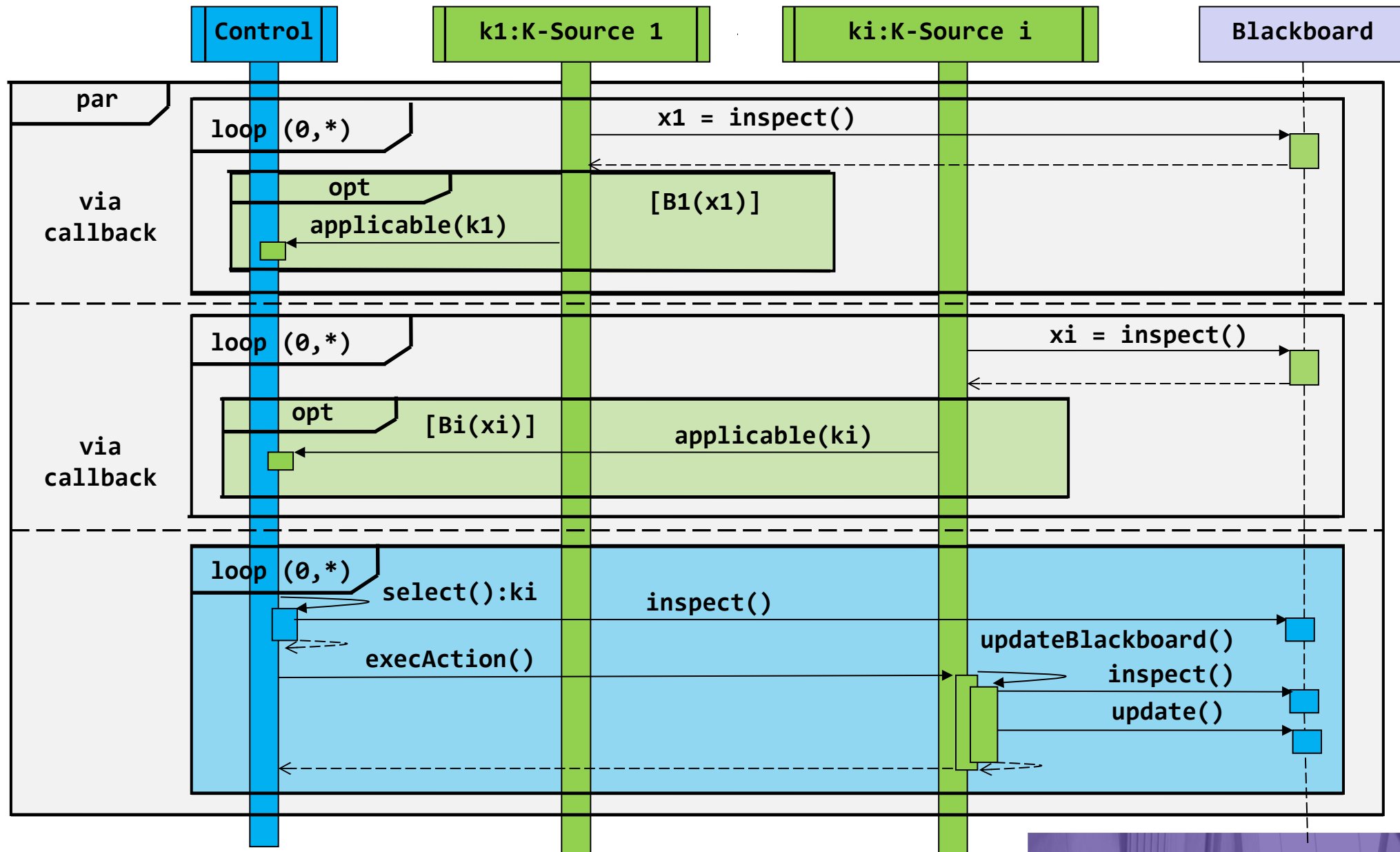
+IKA Blackboard (Struktur)



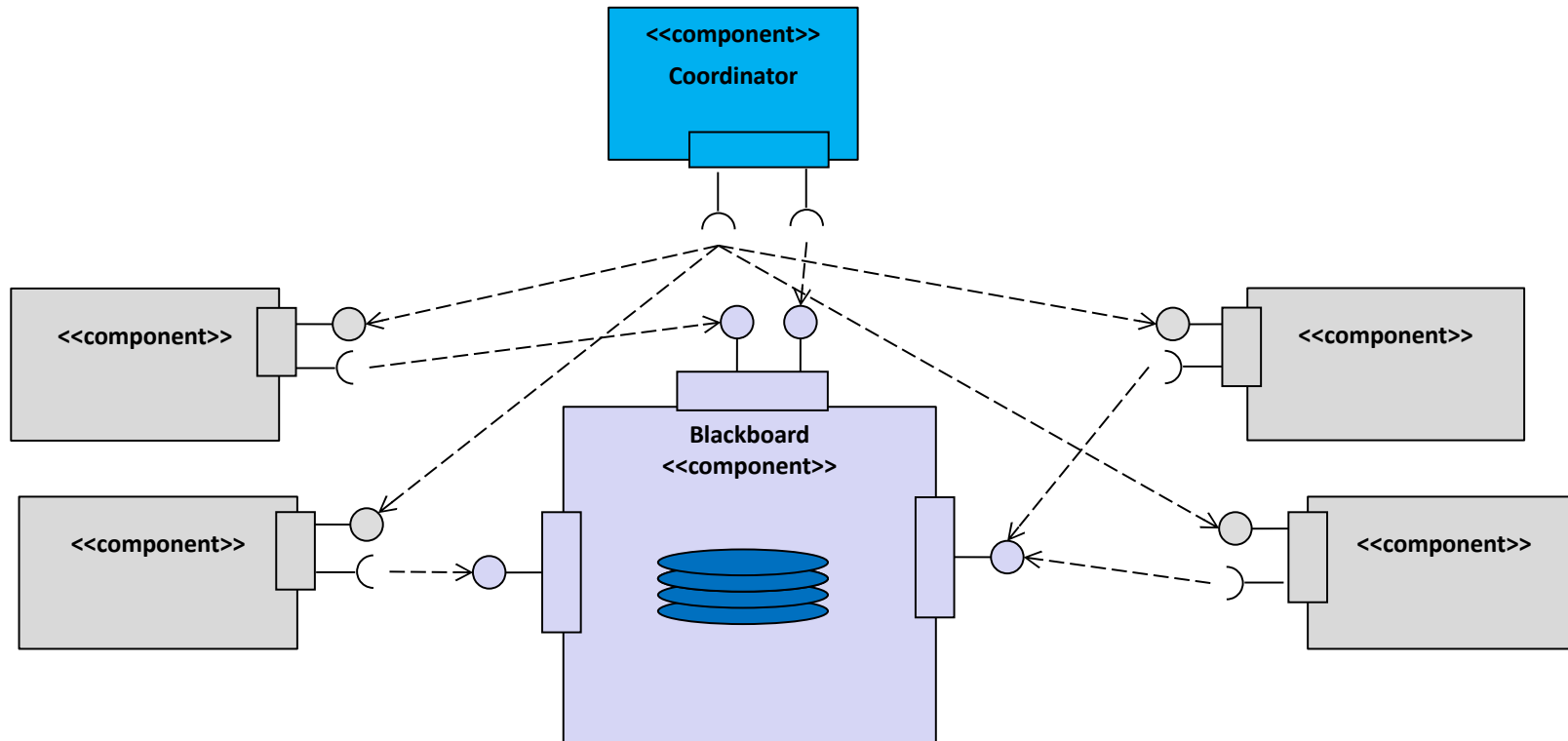
+KA Blackboard (Dynamische Aspekte)



+KA Blackboard (Dynamische Aspekte)

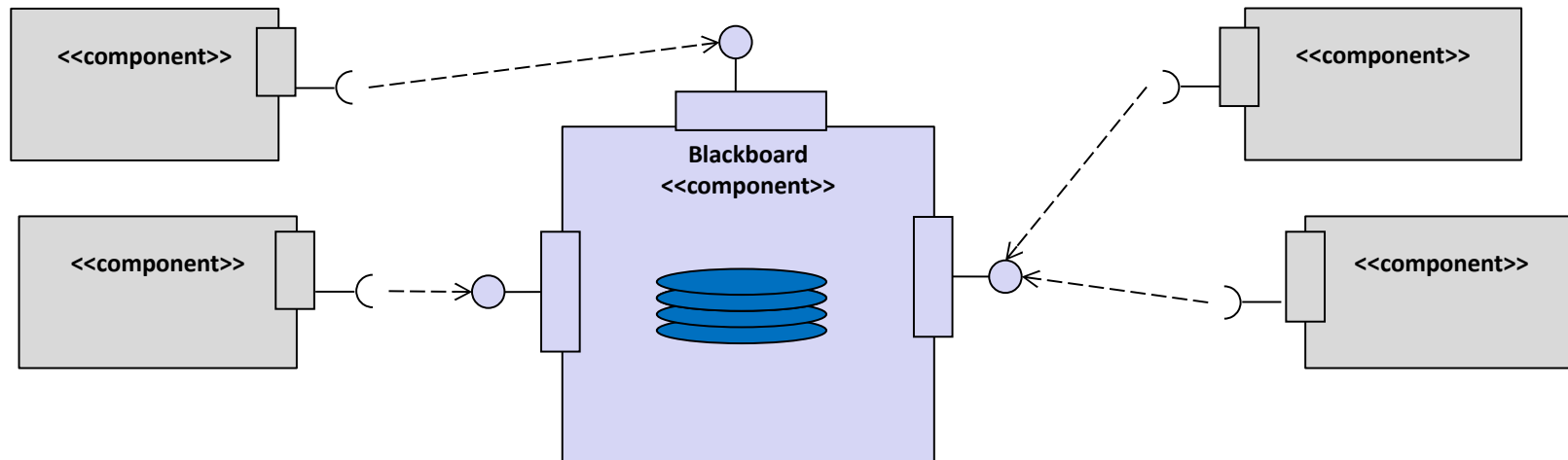


Eigenständige Komponenten erarbeiten gemeinsam eine Lösung auf einer geteilten Datenbasis.



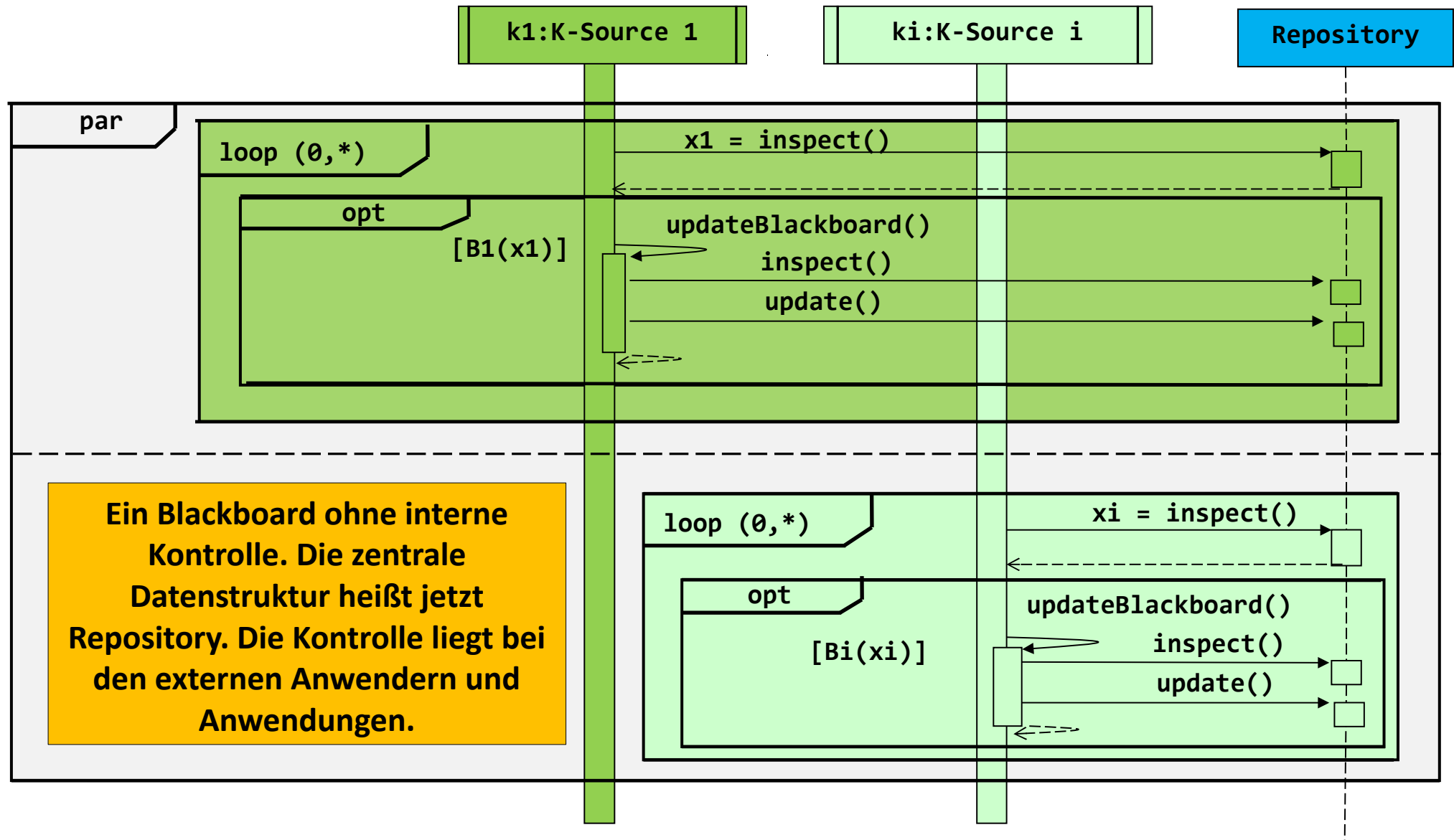
+IK A Blackboard (Repository System)

Eigenständige Komponenten erarbeiten gemeinsam eine Lösung auf einer geteilten Datenbasis.



IDE mit Plugins

+KA Blackboard (Repository System)



+IK A Blackboard (Realisierung)

1. Problem definieren:

- Anwendungsgebiet und Wissensgebiete festlegen.
- Eingaben verstehen, Besonderheiten festlegen, Muster erkennen.
- Ausgaben definieren: Was ist eine „korrekte“ Lösung?

2. Lösungsraum definieren:

- Wie sehen Zwischenlösungen aus?
- Was sind Teillösungen, Gesamtlösungen, ...?

Den Lösungs- und Problemraum strukturieren!
Dies ermöglicht unabhängiges Arbeiten.

3. Den Problemlösungsprozess in Schritte aufspalten:

- Wie werden aus einzelnen Lösungen Gesamtlösungen?
- Wie entstehen Hypothesen?
- Wie können Hypothesen validiert werden?
- Nach welchen Kriterien kann der Lösungsraum eingeschränkt werden?

4. Spezialisierte Wissensquellen definieren:

- Spezialisten für besondere Strategien oder Taktiken.
- Einbindung menschlicher Experten!

5. Vokabular des Blackboards bestimmen:

- Datenstrukturen des Blackboards und der Blackboard-Einträge festlegen.
- Konverter entwerfen (Blackboard $\leftrightarrow$ Wissensquelle).

6. Kontrollkomponente spezifizieren:

Welche Wissensquelle darf wann und wo Änderungen vornehmen?

- Bewertungsstrategie für Hypothesen realisieren.
- Auswahlstrategie für Wissensquellen realisieren.

7. Implementierung der Wissensquellen:

- Aufspalten in Bedingungs- und Aktionsteil.
- Keine Annahmen über andere Wissensquellen verwenden.
- Eventuell kommen verschiedene Technologien und/oder Programmiersprachen zum Einsatz.
 - Regelbasierte Systeme,
 - LLMs, ...

+IKA Blackboard (Vorteile)

- **Man kann experimentieren**
Wichtig, um neue Erkenntnisse zu gewinnen.
- **Änderbarkeit und Wartbarkeit wird unterstützt**
Strikte Trennung zwischen Wissensquellen, Kontrolle und Daten.
- **Wiederverwendbare Wissensquellen**
Wissensquellen sind individuelle Spezialisten.

+IKA Blackboard (Vorteile)

- **Fehlertolerant und stabil**
Ergebnisse sind Hypothesen, Eingabedaten können rauschen, Schlussfolgerungen können falsch sein.
- **Schwierig zu testen**
Berechnungen sind oft nicht nachvollziehbar, da Daten unvollständig und Methoden fragwürdig sind.
- **Keine Lösungsgarantie**
in der Regel wird nur ein gewisser Prozentsatz der Aufgaben gelöst.



Software-Architekturen

—

Pipes and Filters

Prof. Dr. Thomas Fuchß

Hochschule Karlsruhe

University of
Applied Sciences

Fakultät für

**Informatik und
Wirtschaftsinformatik**

+IKA Pipes and Filters

[Frank Buschmann (Pattern-Oriented Software Architecture) Band: 1 A System of Patterns – John Wiley & Sons. 1996.]

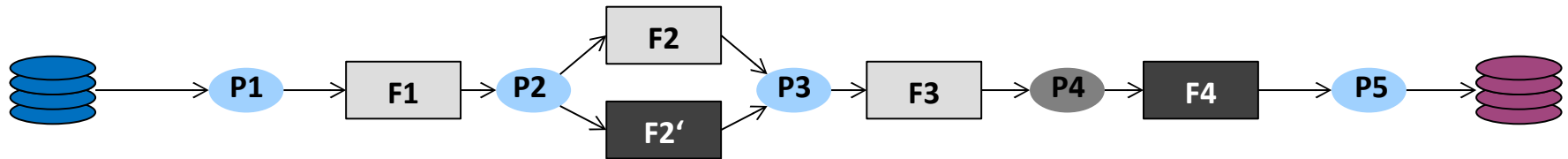
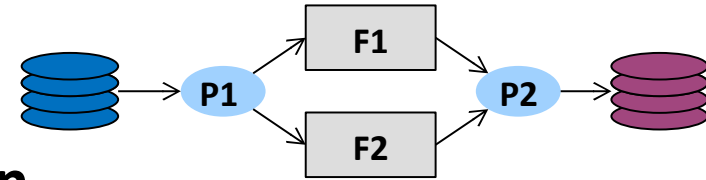
Das Pipes-and-Filters-Muster bietet eine Struktur für Systeme, die Daten Schritt für Schritt verarbeiten.

- **Jeder Bearbeitungsschritt ist in einem „Filter“ gekapselt.**
- **Die Konnektoren zwischen den Filtern werden als „Pipes“ bezeichnet.**

Durch die Standardisierung von Filtern und Konnektoren entstehen leicht adaptierbare und konfigurierbare Systeme.

+IKA Pipes and Filters: Übergeordnete Ziele!

- Ein System soll Daten im Sinne eines Workflows verarbeiten.
- Zukünftige Systemerweiterungen sollen durch den Austausch und das Einfügen neuer Komponenten realisiert werden.



➡ Die verarbeitenden Einheiten müssen eigenständig bleiben.

➡ Der Transport zwischen den Komponenten muss standardisiert erfolgen (festes Protokoll).

➡ Datenfluss und Kontrollfluss sind identisch!

➡ Filter arbeiten seiteneffektfrei!

+IKA Pipes and Filters: Was muss beachtet werden?

- Kleine Komponenten sind leichter wiederverwendbar.
- Filter, die nicht aufeinander folgen, tauschen auch keine Informationen aus.
- Eingaben können aus unterschiedlichen Quellen (Filter) stammen.
- Ausgaben sind für unterschiedlichen Senken (Filter) bestimmt.
- **Eingaben sind oftmals kontinuierlich.**

Filter sollten parallel arbeiten!

Inwieweit eine Aufteilung in Verarbeitungskomponenten durchführbar ist, hängt stark vom Anwendungsgebiet und dem zu lösenden Problem ab.



Wie wird ein interaktives System in parallel arbeitende, unabhängige Komponenten zerlegt?

+IKA Pipes and Filters: Umsetzung (Filter)

Filter sind die verarbeitenden Einheiten, die Daten ergänzen, verfeinern und transformieren.

- Die Aufgaben des Systems werden in mehrere, aufeinanderfolgende, unabhängige Schritte zerlegt.
- Jeder Verarbeitungsschritt wird als eigenständige Komponente realisiert (Filter).
- Die einzelnen Schritte (Filter) werden über den Datenfluss verbunden, das Ergebnis des einen Schritts ist die Grundlage des nächsten.
- Prüfen, ob alle Filter miteinander verbunden sind.

Sicherstellen, dass jeder Filter nur von den Ausgaben des vorherigen abhängt.

+IKA Pipes and Filters: Dynamik

- Daten werden als Ströme aufgefasst, die aus gleichartigen Elementen bestehen.



- Die Latenzzeit wird verringert.
- Ein paralleles Arbeiten wird möglich.

- **Die Ausführung eines Filters wird ausgelöst durch:**

- Das nachfolgende Element, das Daten entnimmt.
- Das vorherige Element, das Daten übergibt.



Passiver Filter

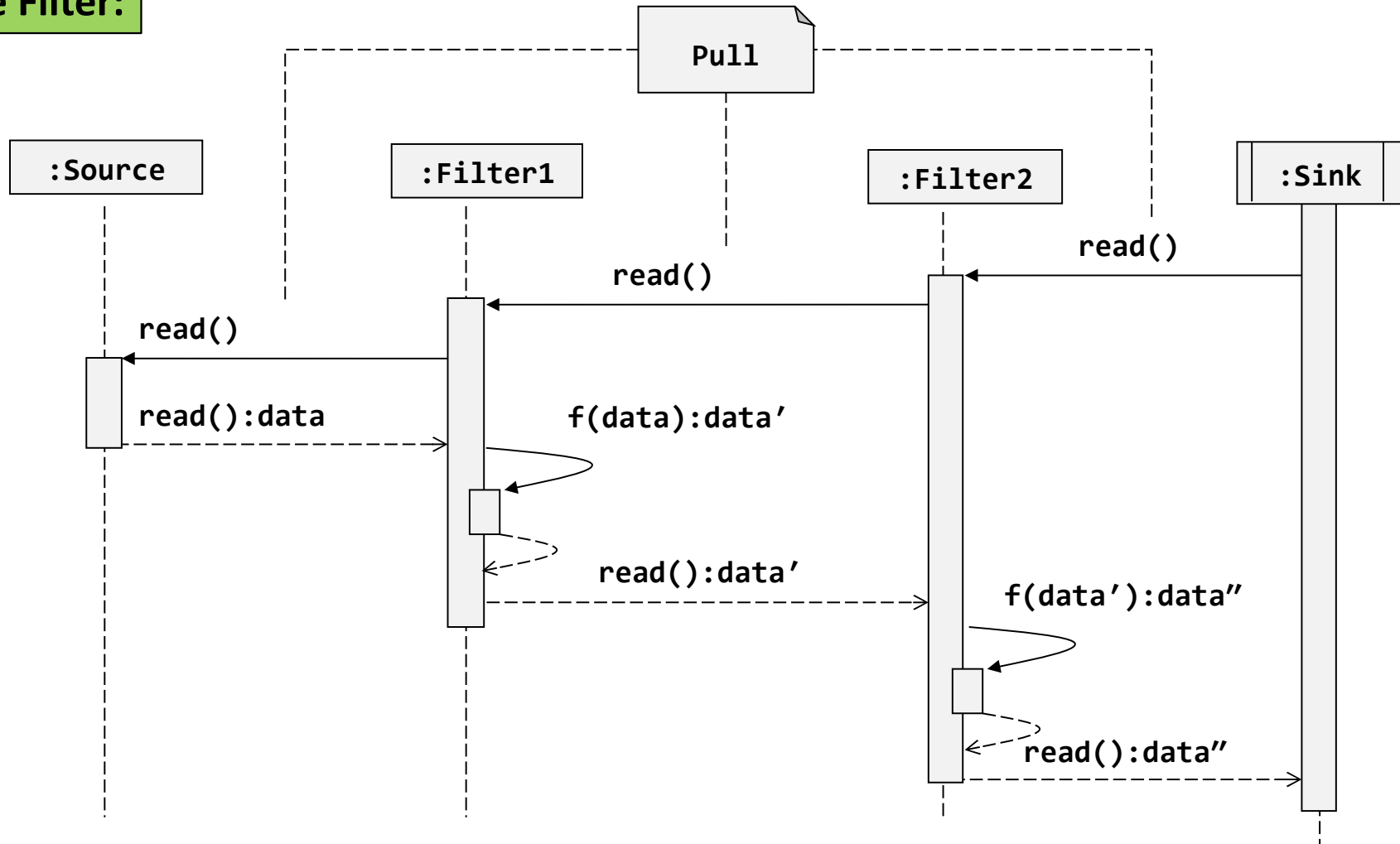
- Durch den Filter, der sich Daten aus der Pipe holt oder in die Pipe schreibt.



Aktiver Filter

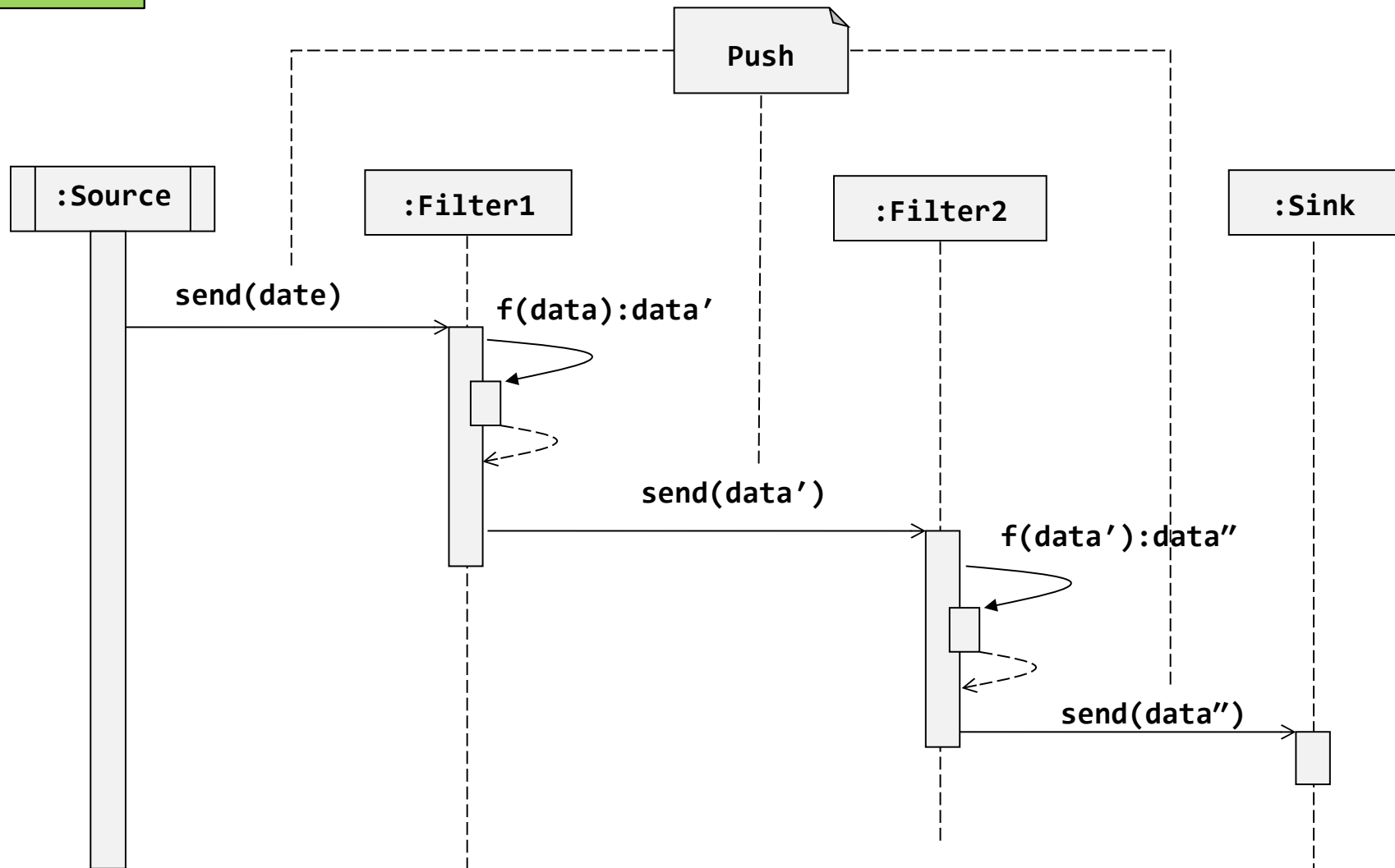
+IK A Pipes and Filters: Dynamik (PULL)

Passive Filter:



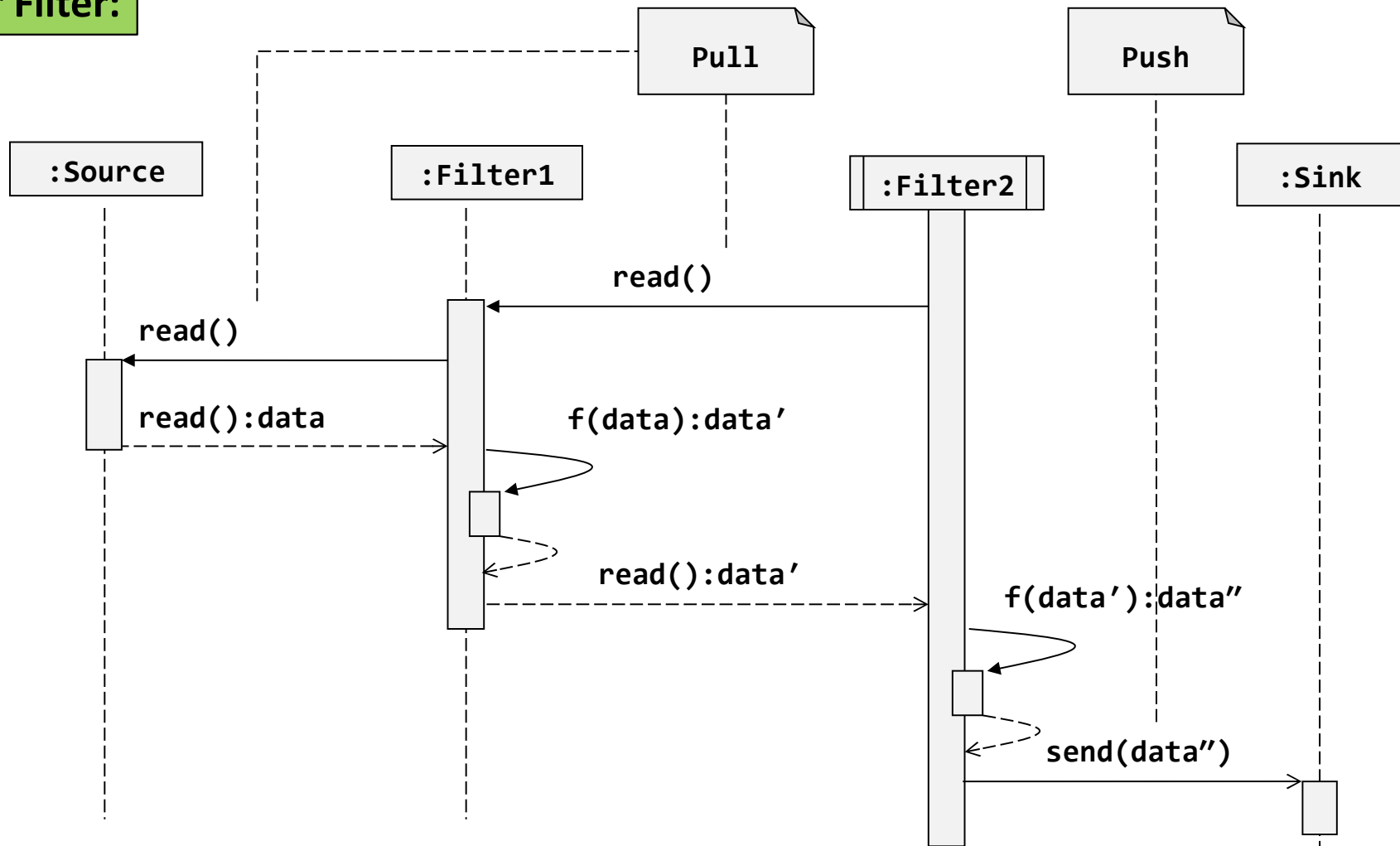
+K A Pipes and Filters: Dynamik (PUSH)

Passive Filter:



+IK A Pipes and Filters: Dynamik (PUSH-PULL)

Aktiver Filter:



+IK A Pipes and Filters: Umsetzung (implizite Pipes)

Konnektoren (Pipes) verbinden und synchronisieren Filter.

- Implizite Konnektoren – die Datenübergabe erfolgt direkt von Filter zu Filter (typischerweise Methodenaufruf / RPC).



Kaum Parallelität!



Flexibilität geht verloren!

+IK A Pipes and Filters: Umsetzung (explizite Pipes)

Konnektoren (Pipes) verbinden und synchronisieren Filter.

- Explizite Konnektoren – Konnektoren werden aktive Komponenten:
 - Bordmittel des Betriebssystems (Message Queues, Shared Memories),
 - Bordmittel der Programmierumgebung, Frameworks (rabbitMQ),
 - Selbst implementierte Komponenten, ...



Parallelität

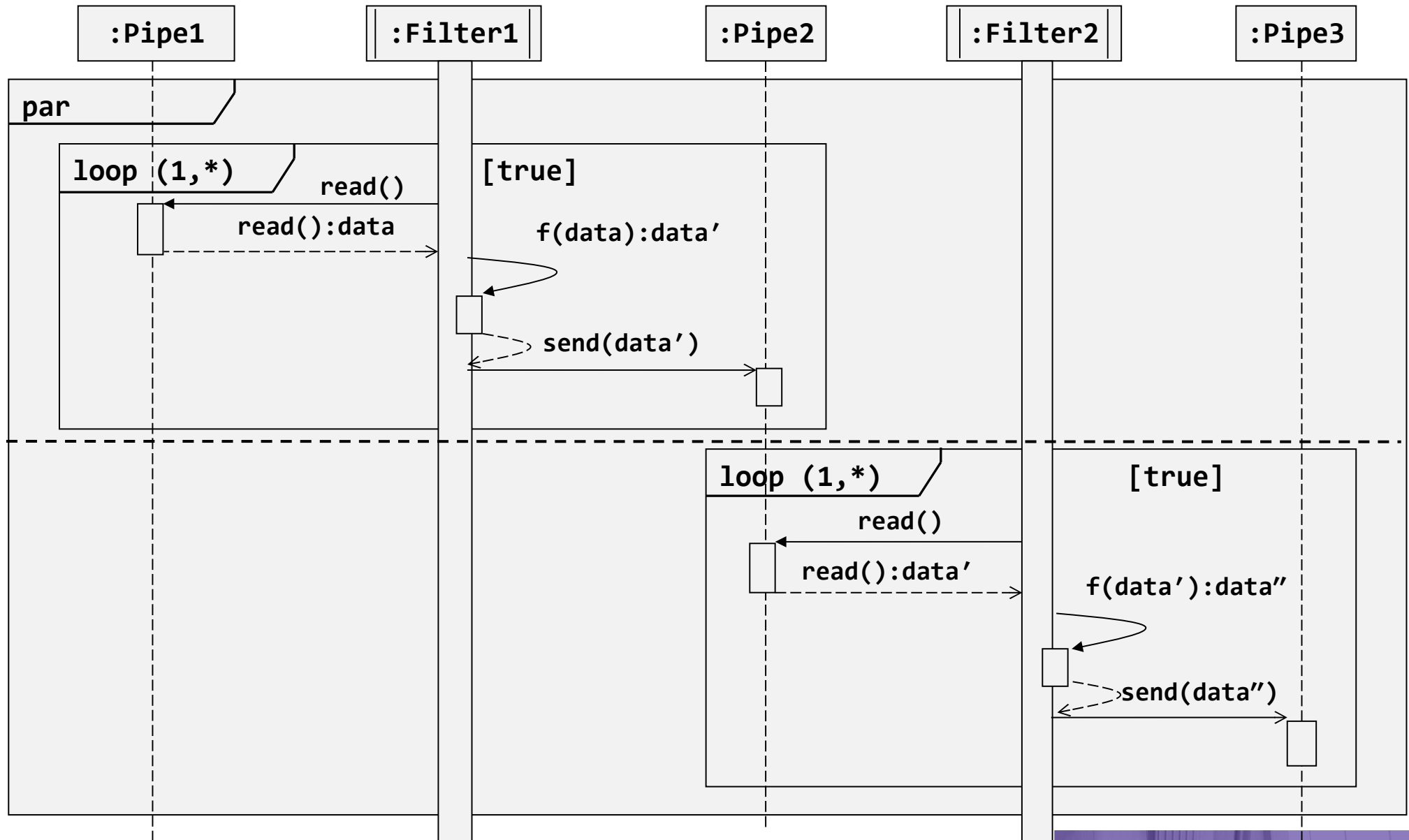


Definierte Austauschformate (JSON, ASN.1, XML)



Hoher Aufwand für die Kommunikation

+KA Pipes and Filters: Dynamik (aktive Filter / explizite Pipes)



+IKA Pipes and Filters: Varianten (Tee and Join)

Tee-and-Join-Pipeline-Systeme:
Statt nur einer Eingabe und einer Ausgabe haben Filter mehrere Ein- (Join) und Ausgaben (Tee).

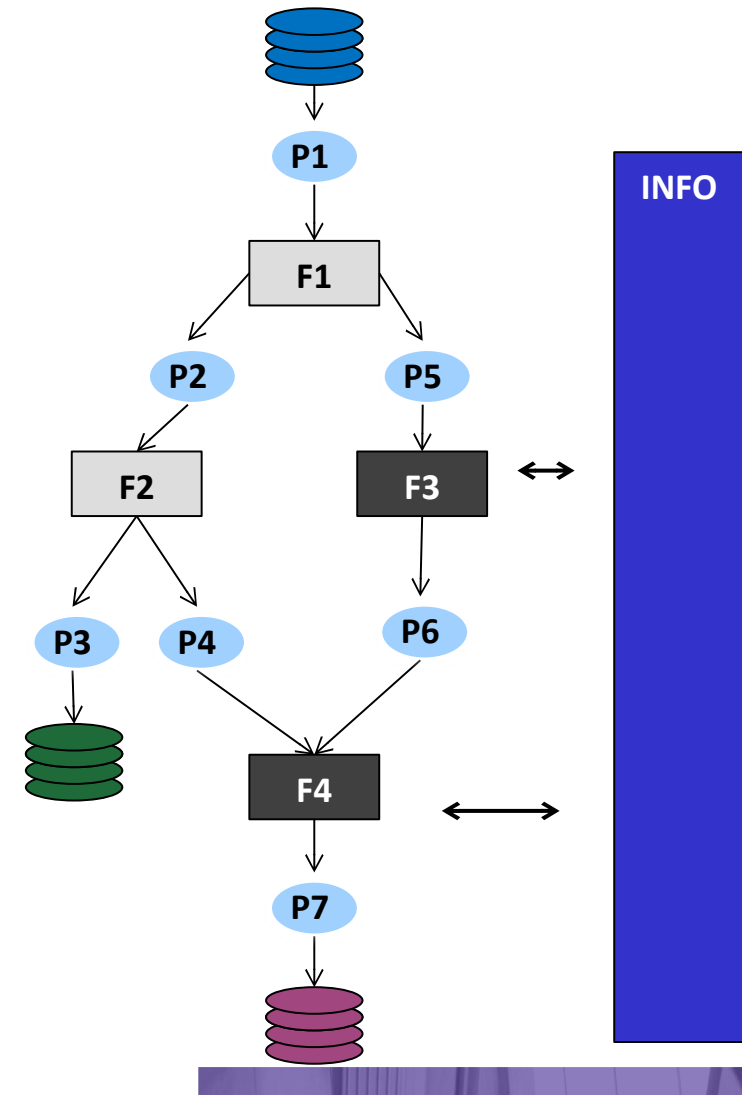
- Die Verarbeitung stellt sich als gerichteter, zyklischer Graph dar.
- Tees ermöglichen die Kontrolle von Berechnungen.
 - Rückkopplungen können realisiert werden.
 - Berechnungen können aufgespalten und zusammengeführt werden.
 - Komplexe Automatisierungssysteme können realisiert werden.

Komplexität steigt, Synchronisierung wird notwendig.

+IKA Pipes and Filters: Varianten (globale Daten)

Gemeinsame Datenstrukturen

Statt Daten immer durchzureichen, kann es aus Effizienzgründen notwendig sein, gemeinsame Daten-Container zu nutzen und lediglich Referenzen durchzureichen.



+IKA Pipes and Filters: Auswirkungen

Vorteile:

- Zwischendateien sind nicht notwendig (aber möglich, da sie bei kommunizierenden Prozessen Kanäle sparen).
- Tees ermöglichen die Kontrolle von Berechnungen.
- Flexibilität:
 - Austausch von Filtern,
 - Neuordnung von Filtern,
 - Wiederverwendung von Filtern, ...

**Effizienz durch Parallelität aktiver
Filterkomponenten!**

+IKA Pipes and Filters: Auswirkungen

Nachteile:

- **Gemeinsame Nutzung von Zustandsinformation ist teuer:**
 - Der Zustand muss durchgereicht werden.
 - Fehlt Zustandsinformation, dann wird eine Fehlerbehandlung schwierig.
- **Effizienzgewinn durch Parallelität ist oft geringer als vermutet:**
 - Die Datenübertragung ist teuer und aufwendig.
 - Eventuell muss die Datenverarbeitung doch in einem großen Block erfolgen.
 - Die Synchronisation zwischen Kanälen und Filtern ist nicht trivial.



Ein Fehlerkanal sollte eingerichtet werden.



Überflüssige Transformationen vermeiden und ggf. Filter zusammenfassen.

[Richards, Mark. Software Architecture Patterns – O'Reilly Media, 2015.]

Pipes and Filters in Event-Driven Architectures

Eine ereignisgesteuerte Architektur zielt auf hochskalierbare, anpassungsfähige Anwendungen ab.

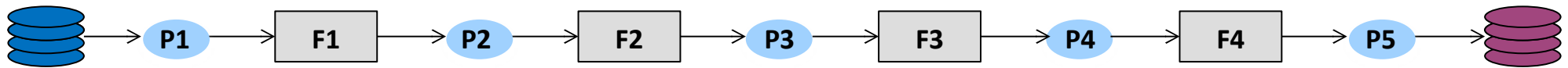


- Die einzelnen Komponenten müssen voneinander entkoppelt sein.
- Ereignisse müssen asynchron empfangen und verarbeitet werden.

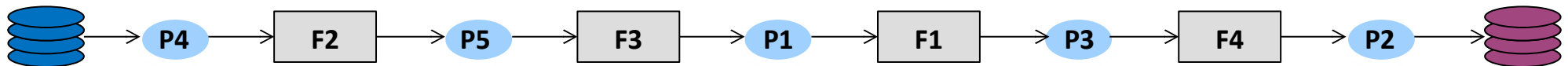
Ein Tee-and-Join-Pipeline-System mit expliziten Pipes.

Komplexe Events werden durch den Mediator aufgespalten und in eigenständige, asynchron bearbeitbare Aufgaben verwandelt.

+IKA Mediator: Idee

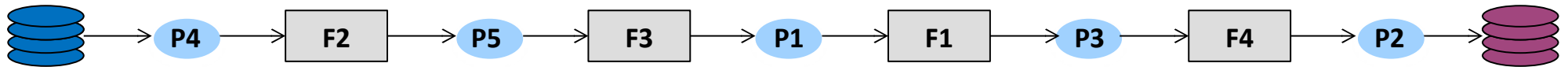


Voraussetzung: Die Bearbeitungsreihenfolge ist frei.

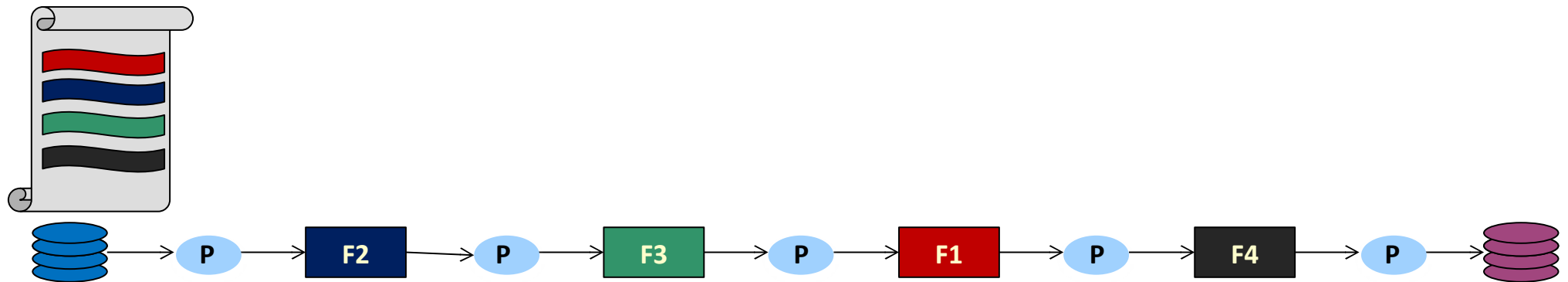


Filter und Pipes sind austauschbar.

+IKA Mediator: Idee

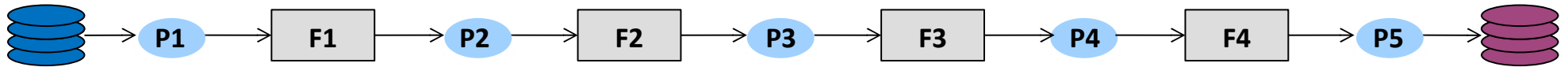


**Nicht das Produkt,
sondern die Aktionen stehen im Vordergrund.**

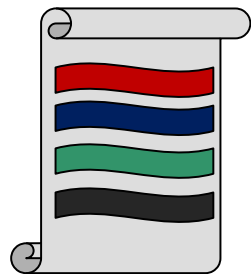


+IKA Mediator: Idee

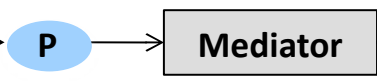
Statt:



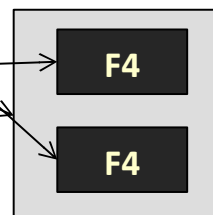
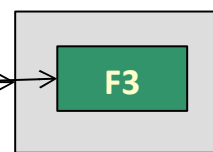
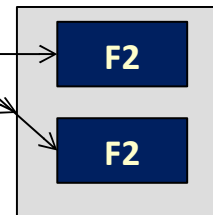
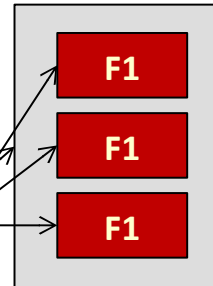
Jetzt:



Event
Queue



Event
Channel



Event Processor

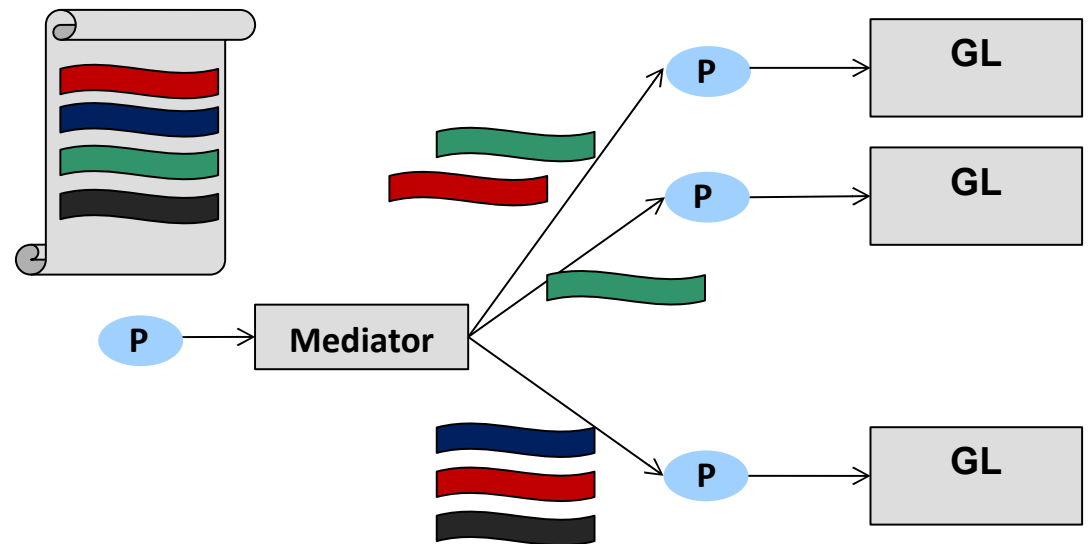
Verarbeitende Filter
(Komponenten) werden zu
Event-Prozessoren
zusammengefasst.

Event
Processor

+IKA Mediator: Aufgaben

- Der Mediator ist für die Verteilung der Aufgaben zuständig.
- Jeder Arbeitsschritt wird als Processing Event über spezifische Pipes (Kanäle) an einen oder mehrere Event-Prozessoren geschickt.
- Die Komponenten (Filter) innerhalb der Event-Prozessoren kapseln die Geschäftslogik.

Eine Verteilung nach dem Publish-Subscribe-Prinzip ist möglich!



+IKA Mediator: Auswirkungen

Vorteile:

- Sehr anpassungsfähig: Komponenten sind unabhängig und austauschbar.
- Gute Performance durch asynchrone und parallele Verarbeitung.
- Sehr gut und individuell skalierbar – jeder Event-Prozessor kann einzeln skaliert werden.

Nachteile:

- Abhängigkeiten zwischen Mediator und Prozessor lassen sich kaum vermeiden.
- Neue Prozessoren erfordern im Allgemeinen eine Anpassung des Mediators.

**Reale Implementierungen nutzen typischerweise
vorgefertigte Middleware-Konzepte wie
Message Broker
oder
Business Process Orchestration Tools.**



Software-Architekturen

—

Middleware-Konzepte zwischen gestern, heute und morgen

Prof. Dr. Thomas Fuchß

Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
**Informatik und
Wirtschaftsinformatik**

Die wenigsten Systeme werden auf der grünen Wiese entwickelt oder „from scratch“.

Meistens handelt es sich um Korrekturen, Adaptionen oder Erweiterungen bestehender Systeme



Die Entwicklung muss in eine definierte „Umgebung“ eingebettet werden.

Die Middleware ist das Framework für diese Umgebung, sie ist die Abstraktionsebene zwischen der eigentlichen Anwendung und der realen Welt (Hardware, Betriebssystem, Cloud-Anbieter), in der die Anwendung abläuft.



Das Rückgrat jeder Anwendung ist die richtige Middleware.

Eine Middleware stellt die elementare Infrastruktur für ein System bereit.

- Eine Middleware ist verantwortlich für die Topologie (1-1, 1-n, n-m).
- Eine Middleware bietet einheitliche Mechanismen für die Kommunikation zwischen den Komponenten.
- Eine Middleware bietet einheitliche Mechanismen für den Datenaustausch zwischen Komponenten.
- Eine Middleware bietet eine verallgemeinerte Laufzeitumgebung.
- Eine Middleware ist für den Anwender transparent.

Eine Middleware ist unabhängig von der Applikation.

+IKA Middleware: Abstraktionsebenen (gestern)

**Business Process
Orchestration Tools**

BPEL, BIZTALK,...

Message Broker

ESB, Mule, WebSphere, ...

Application Server

AOM, JEE, Tomcat, Geronimo, ...

Transport Mechanism

OOM (RPC), MOM, CORBA, .NET Remote, ...

+IKA Middleware (gestern – Einordnung)

- **Transport Mechanism:**
Basiskonnektoren zum Senden von Nachrichten und zum Austausch von Daten sowie für den Zugriff auf Objekte.

Vereinheitlichen und vereinfachen die Kommunikation!

- **Application Server:**
Erweitern die Transportebenen um eine definierte Ausführungsumgebung, Komponentenmodelle, Transaktionsverwaltung, Verzeichnisdienste und Security.

Vereinheitlichen das Programmiermodell!

+IKA Middleware (gestern – Einordnung)

- **Message Broker (ESB):**
Erweitern Application Server um eine Kommunikationsinfrastruktur, zum schnellen Austausch von Daten über Anwendungsgrenzen hinweg. Bieten Transformations-, Integrations- und Routingdienste.

Vereinheitlichen und integrieren Anwendungen!

- **Business Process Orchestration Tools:**
Erweitern Message Broker um Workflow-Engines. Sie verwalten und organisieren ganze Geschäftsabläufe. Managen deren Zustände und verteilen die Aufgaben und überwachen die Ausführung.

Vereinheitlichen Abläufe!

+IKA Middleware: Abstraktionsebenen (heute)

Orchestration

Kubernetes

Container

Docker

Services und Runtime

Spring Boot, Node.js, ...

Transport und Networking

Http, gRPC, Kafka, ... MOM

- **Transport und Networking Tools:**
Basiskonnektoren zum Senden von Nachrichten und zum Austausch von Daten sowie für den Zugriff auf Objekte.

Vereinheitlichen und vereinfachen die Kommunikation!

- **Services und Runtime Tools (nur noch leichtgewichtige Application Server):**
Erweitern die Transportebenen um eine definierte Ausführungsumgebung und Komponentenmodelle. Bieten nur noch eine leichtgewichtige Transaktionsverwaltung, aber keine Verzeichnisdienste mehr.

Ermöglichen eine Aufteilung der Logik in unabhängig ausführbare Services!

- **Container Tools:**

Kapseln Laufzeitumgebung und Anwendung in portablen, isolierten Einheiten.

Bieten Lösungen zur portablen und reproduzierbaren Bereitstellung der Services!

- **Orchestration Tools:**

Verwalten, skalieren und überwachen die Container auch über mehrere Knoten hinweg. Kümern sich um das Service Discovery.

Ermöglichen den verteilten und ausfallsicheren Betrieb der Anwendung!

+IKA Was hat sich geändert?

Die klassischen, oft schweren Middleware-Frameworks sind verschwunden.

Die Features aber nicht – sie sind lediglich in spezialisierte Komponenten gewandert.

Statt Magie: mehr Transparenz und Übersichtlichkeit!

Statt einer Lösung: mehr Flexibilität und Kombinierbarkeit!

Klassisch: OOM

- **Ein zustandsbasiertes Ausführungsmodell.**
- **Rein objektorientiertes Programmiermodell.**
- **Im Allgemeinen synchrone Methodenaufrufe.**
- **Hoch komplexe Konfigurationsmethoden (IDL, ORB).**
- **Hohe Transferkosten für Parameter und Resultate.**

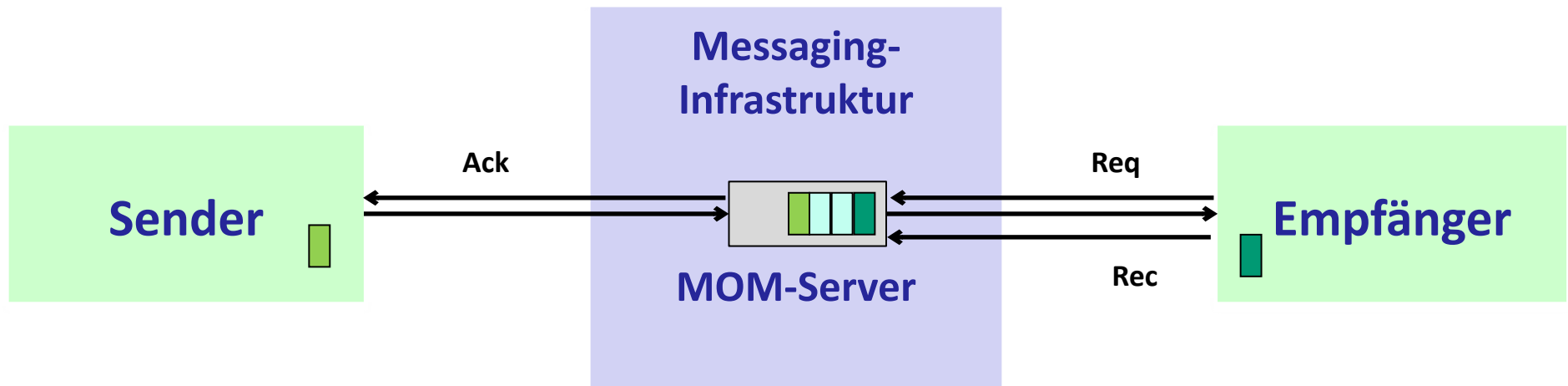
Modern: REST und gRPC

- **Ein zustandsloses Ausführungsmodell (klassisch funktionale Sichtweise).**
- **Service- und ressourcenorientiertes Programmiermodell.**
- **Ob asynchron oder synchron, hängt vom Anwendungsszenario ab.**
- **Statt hoch komplexer Mechanismen: einfache Protokolle und Konventionen (REST/JSON).**
- **Transferkosten für Parameter und Resultate bleiben.**

+IKA MOM: Entkopplung durch Nachrichten

- Anwendungen sind autonom.
- Die Kommunikation erfolgt über Nachrichten.
- Nachrichten werden in Warteschlangen verwaltet.

Die Messaging-Infrastruktur entkoppelt Sender und Empfänger.



+IKA MOM: Entkopplung durch Nachrichten

Typische Aufgaben eines MOM-Servers:

- Nachrichten entgegennehmen und
 - in die richtige Queue einsortieren,
 - Eine Bestätigung an den Absender schicken,
 - solange vorhalten, bis die Nachricht abgeholt wird.
- Auf Anfrage Nachrichten ausliefern.



**Queues werden mit ihren Empfängern
assoziiert.**

+IKA MOM: Entkopplung durch Nachrichten

Typische Anwendungsszenarien:

- Ein Sender möchte Informationen verteilen, benötigt aber keine Bestätigung durch den Empfänger. (Werbung)



Fire-And-Forget

- Ein Sender möchte Informationen verteilen, benötigt aber keine **direkte** Bestätigung durch den Empfänger (kann weiter arbeiten).
- Sender, Empfänger und Netzwerk sind geprägt durch hohe Ausfallzeiten (mobile Anwendungen) .



**MOM-Infrastruktur speichert Nachrichten, bis sie ausgeliefert wurden und überwacht die Auslieferung.
(SMS , WhatsApp, RabbitMQ)**

+IKA MOM: Entkopplung durch Nachrichten

Concerns im Umfeld von Auslieferungsprozessen:

- Reliability:
Welche Probleme verursacht ein Nachrichtenverlust?
- Performance:
Wie hoch ist der Aufwand, um eine Nachricht zu übermitteln?



**Reliability und Performance sind entgegengesetzte Kräfte:
Je mehr Reliability, desto weniger Performance und
umgekehrt.**

QoS-Abstufungen

- **Best Effort (hohe Performance):**
Die MOM-Schicht versucht ihr Bestes garantiert aber nichts. Nachrichten werden lediglich im RAM des Servers gehalten und nach einer definierten Zeit verworfen.
- **Persistent (geringe Performance):**
Die MOM-Schicht garantiert die Auslieferung einer Nachricht auch über den Systemausfall hinaus. Nachrichten werden hierzu nicht flüchtig gesichert.
- **Transactional (noch etwas geringere Performance):**
Mehrere Nachrichten können zu einer Transaktion mit ACID-Eigenschaften zusammengefasst werden.

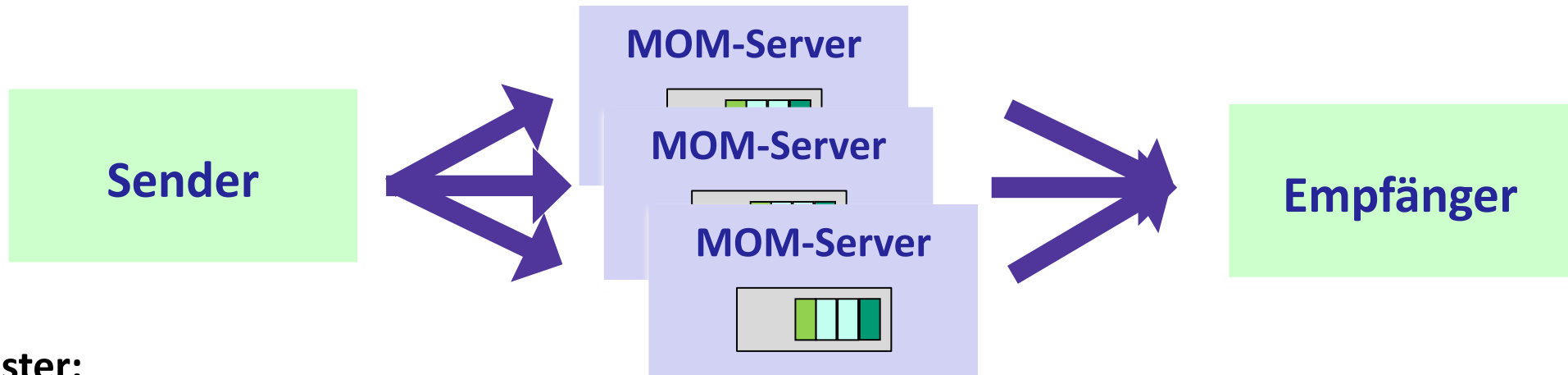
+IKA MOM: Entkopplung durch Nachrichten

Eine MOM-Infrastruktur steht und fällt mit der Verfügbarkeit des MOM-Servers.



Redundanz schafft Ausfallsicherheit.

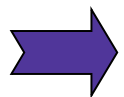
+IKA MOM (Clustering)



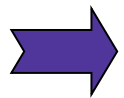
- **Cluster:**

Mehrere Server werden transparent zusammengefasst.

- Jeder MOM-Server verfügt über dieselben Queues.
- Ein Sender (Client) wird entweder statisch oder dynamisch einem MOM-Server zugeteilt.



- **Beim Ausfall eines Servers können Zuordnungen neu vergeben werden, die Infrastruktur bleibt aktiv.**



- **Eine Lastverteilung ist möglich, jeder MOM-Server sieht nur eine bestimmte Anzahl von Clients.**

+IKA MOM: Entkopplung durch Nachrichten

Publish-Subscribe

Aus n-1 wird n-m



Aus Queues werden Topics!

- Empfänger werden zu Subscribern, die sich für bestimmte Topics registrieren.
- Sender werden zu Publishern, die zu bestimmten Topics Nachrichten liefern.
- **Aufgabe des Servers ist es nun, die Nachricht an alle Subscriber auszuliefern.**

Aufgabe des MOM-Servers ist es Subscriber und Topics zu verwalten.

+IKA MOM: Entkopplung durch Nachrichten

Aus Sicht einer Software-Architektur sind folgende Punkte von besonderer Bedeutung:

- Sender und Empfänger sind völlig unabhängig.
- Verbindungen sind gerichtet und typischerweise 1-1

**Ein Duplex-Kanal muss über den Nachrichteninhalt definiert werden.
Der Sender übermittelt seine eigene Empfänger-Queue.**

- Messaging Services gibt es wie Sand am Meer.
- Auch Legacy-Systeme lassen sich über Adapter einfach ansprechen.
- Feste Nachrichtenformate sind nicht vorgeschrieben .
- QoS ist konfigurierbar.
- Nachrichtenübertragung kostet Zeit.

+IKA MOM: Entkopplung durch Nachrichten

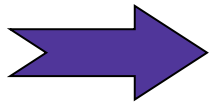
Quality Attributes	
Verfügbarkeit	Physische Queues können repliziert werden. Dadurch entsteht Ausfallsicherheit.
Modifizierbarkeit	Queues trennen Sender und Empfänger vollständig. Bis auf das Austauschformat besteht keinerlei Abhängigkeit.
Performance	QoS-Abstufungen beeinflussen die Performance. Das Netzwerk muss auf die zu erwartende Nachrichtengröße und das zu erwartende Aufkommen abgestimmt sein.
Skalierbarkeit	Queues können auf dedizierten Maschinen betrieben werden, diese sind leicht replizierbar und skalieren damit beliebig. Ein Engpass stellt höchstens das darunterliegende Netzwerk dar.

Herausforderungen bleiben:

- **Die Bauteile müssen „korrekt“ verwendet werden.**
 - Der Architekt muss entscheiden „was“ und sich auf den Entwickler verlassen, dieser entscheidet „wie“.
- **Große Systeme haben viele unterschiedliche Anforderungen.**
 - Architekturen und Standards müssen kombiniert werden.
 - Es ist niemals nur eine Architektur und eine Technologie, die zum Einsatz kommt.

Der Architekt muss abwägen und entscheiden!

Objektorientierung: Die Lösung eines Problems liegt in der „richtigen“ Zusammenarbeit seiner Teile!



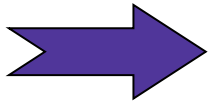
Objekte und Komponenten müssen sich kennen und müssen miteinander kommunizieren.

Eine geeignete Middleware abstrahiert von vielen Problemen:

- Wie findet man ein Objekt?
- Wie schickt man eine Nachricht an ein Objekt bzw. Ressource (Methodenaufruf, Service-Aufruf, Parameterübergabe)?
- Wie kommt man an die richtige Antwort (Übermittlung von Rückgabewerten, ...)?

+IKA Gedanken zur Verteilung

Eine Middleware reduziert nicht den Aufwand für die Kommunikation!



Ein verteiltes System ist mehr als die Verteilung von Objekten!

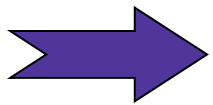
Schnittstellen müssen nicht nur für die Aufgaben, sondern auch für das Ausführungsmodell geeignet sein.

Lokales Interface (feingranular)

- viele kleine und hoch spezifische Methoden
- individuelle Getter / Setter
- ...

Remote Interface (grobgranular)

- wenige Methoden und Funktionen
- Daten werden kombiniert und komprimiert
- ...



Kommunikationsaufwand gilt es zu minimieren – die Flexibilität darf leiden!

**„First Law of Distributed Object Desing:
Don't distribute your objects!“**

[Fowler, M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2003.]

**Die Verteilung von Aufgaben
hat stets eine Grund!**



Software-Architekturen

—

Patterns für den Business Layer

Prof. Dr. Thomas Fuchß

Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
**Informatik und
Wirtschaftsinformatik**

+IKA Patterns für den Business Layer

[Fowler, Martin. Patterns of Enterprise Application Architecture. Addison-Wesley, 2003.]

**Es gilt die Objekte des Anwendungsgebiets zu organisieren und mit den Anforderungen an Performance, Flexibilität, Skalierbarkeit u.ä. in Einklang zu bringen.
Komponenten, die Verwaltungs- und Hilfsfunktionalität bereitstellen, müssen integriert werden.**

+IKA Patterns für den Business Layer

- **Transaction Script:**
Die Geschäftslogik besteht aus definierten Methoden, die aus der Präsentationsschicht über Benutzerinteraktionen aufgerufen werden und die Arbeit erledigen.
- **Domain Model:**
Zentrale Teile des Problemraums werden in den Lösungsraum integriert. Dies schafft Übersicht. Aufgaben werden dort erledigt, wo man es erwartet.
- **Table Module:**
Die zentralen Entitäten des Problemraums werden in Einheiten verwaltet. Statt individueller Repräsentanten nutzt man Objekte, die Mengen von Entitäten repräsentieren (Tabellen und Spalten).

+IKA Das Herzstück des Designs (GRASP)

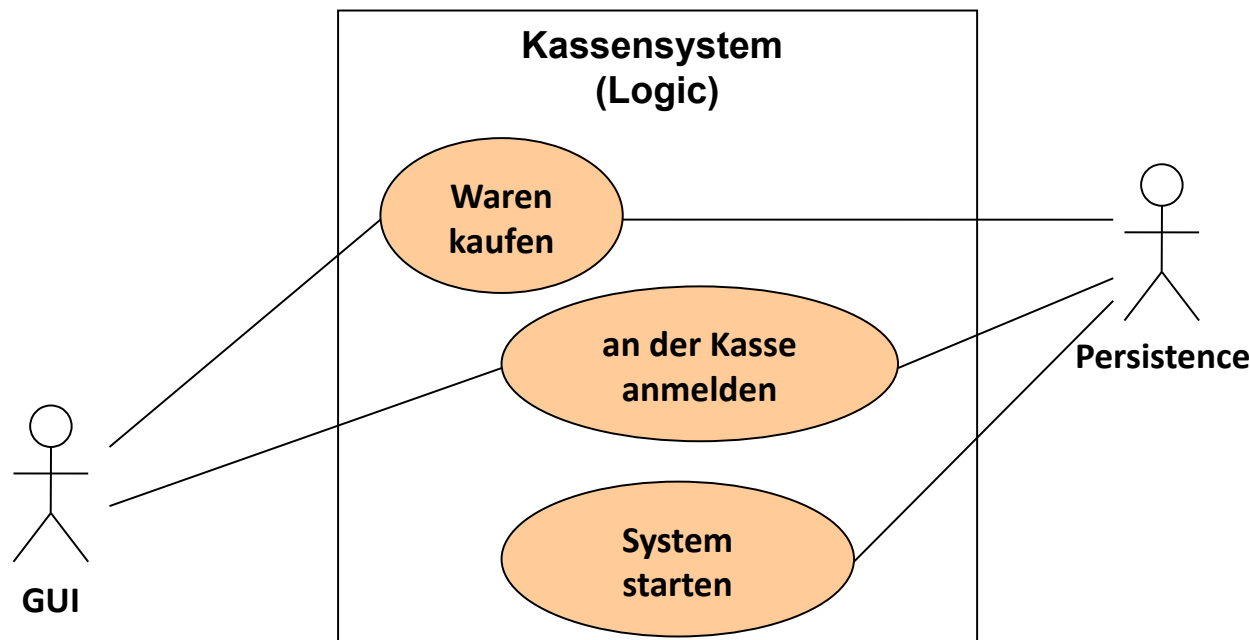
[Larman, Craig: Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, 2. ed. Prentice Hall, 2002.]

- **Verantwortlichkeiten, etwas zu tun**
 - etwas selbst tun
 - jemanden auffordern, etwas zu tun (delegieren)
- **Verantwortlichkeiten, etwas zu wissen**
 - private Informationen kennen
 - abgeleitete oder berechnete Information kennen
 - zugeordnete Objekte kennen

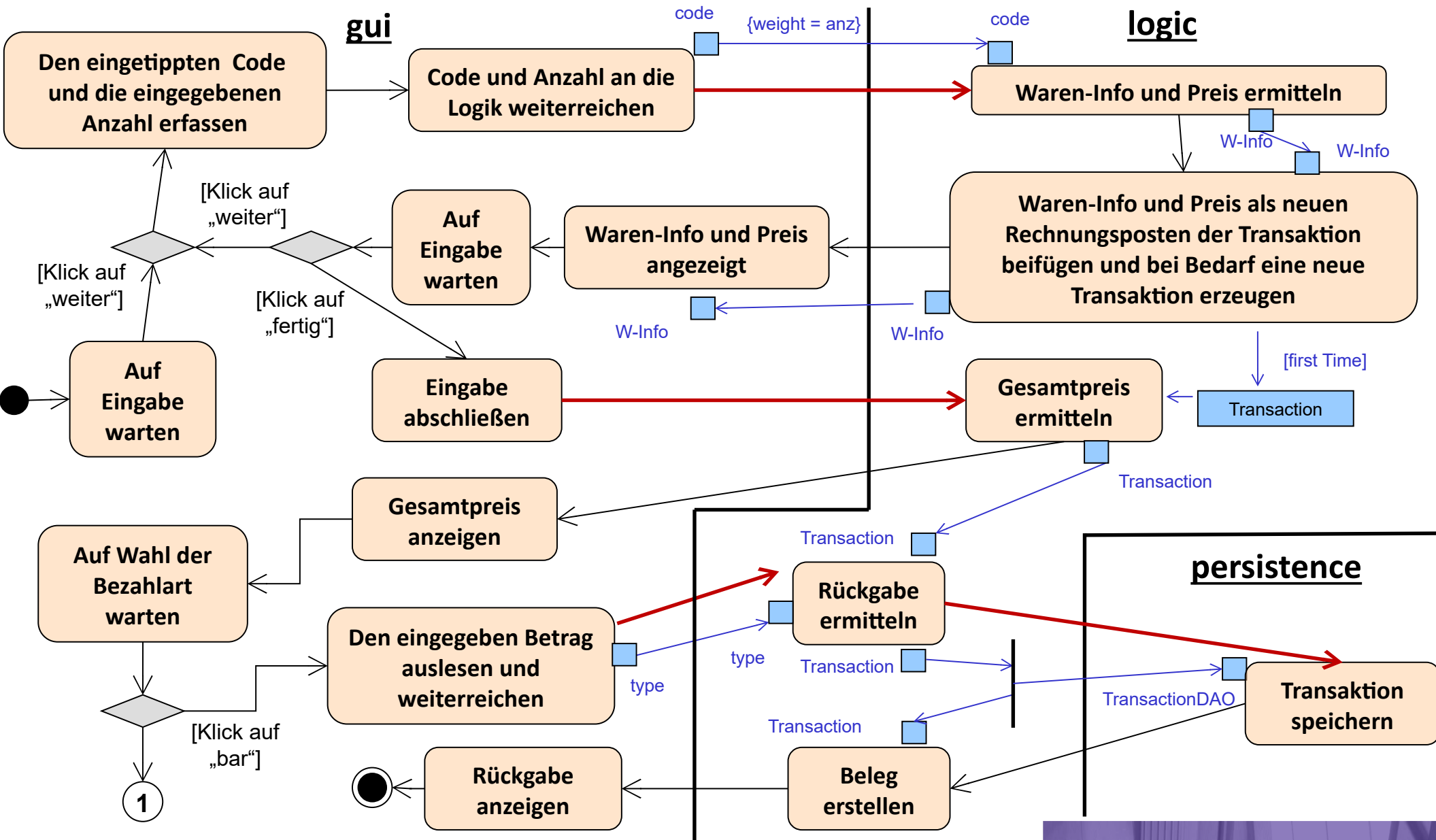
Die Verantwortlichkeiten werden den Objekten während des Designs zugeordnet.

+HKA Use Cases organisieren das Design

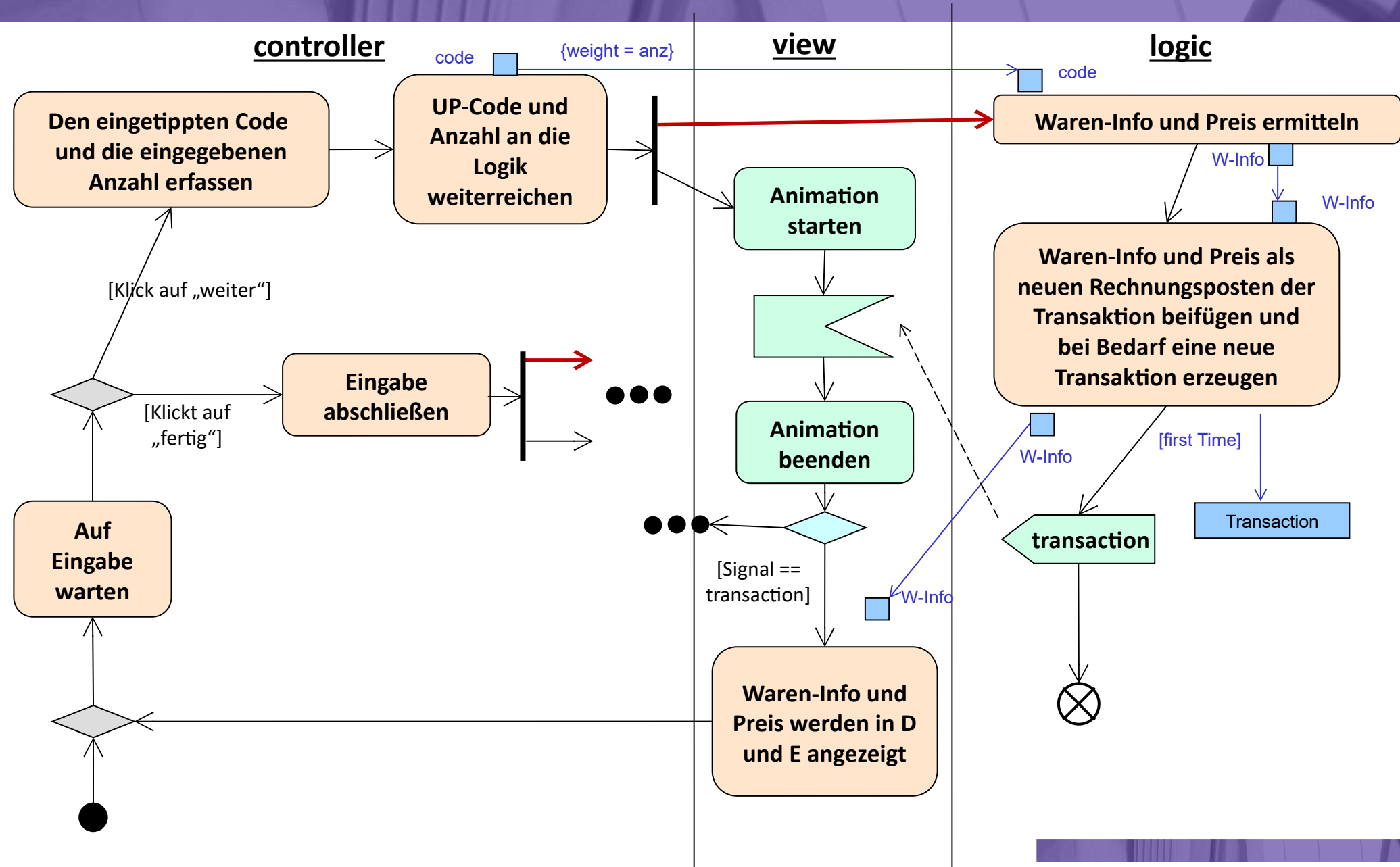
- Für die umzusetzenden Use Cases werden auf Basis der Analyseergebnisse Mockups erstellt
- Aus den Use Cases der Analyse werden konkrete System-Use-Cases als Grundlage für das Design
- Im Rahmen der Designphase werden die Use Cases iterativ realisiert



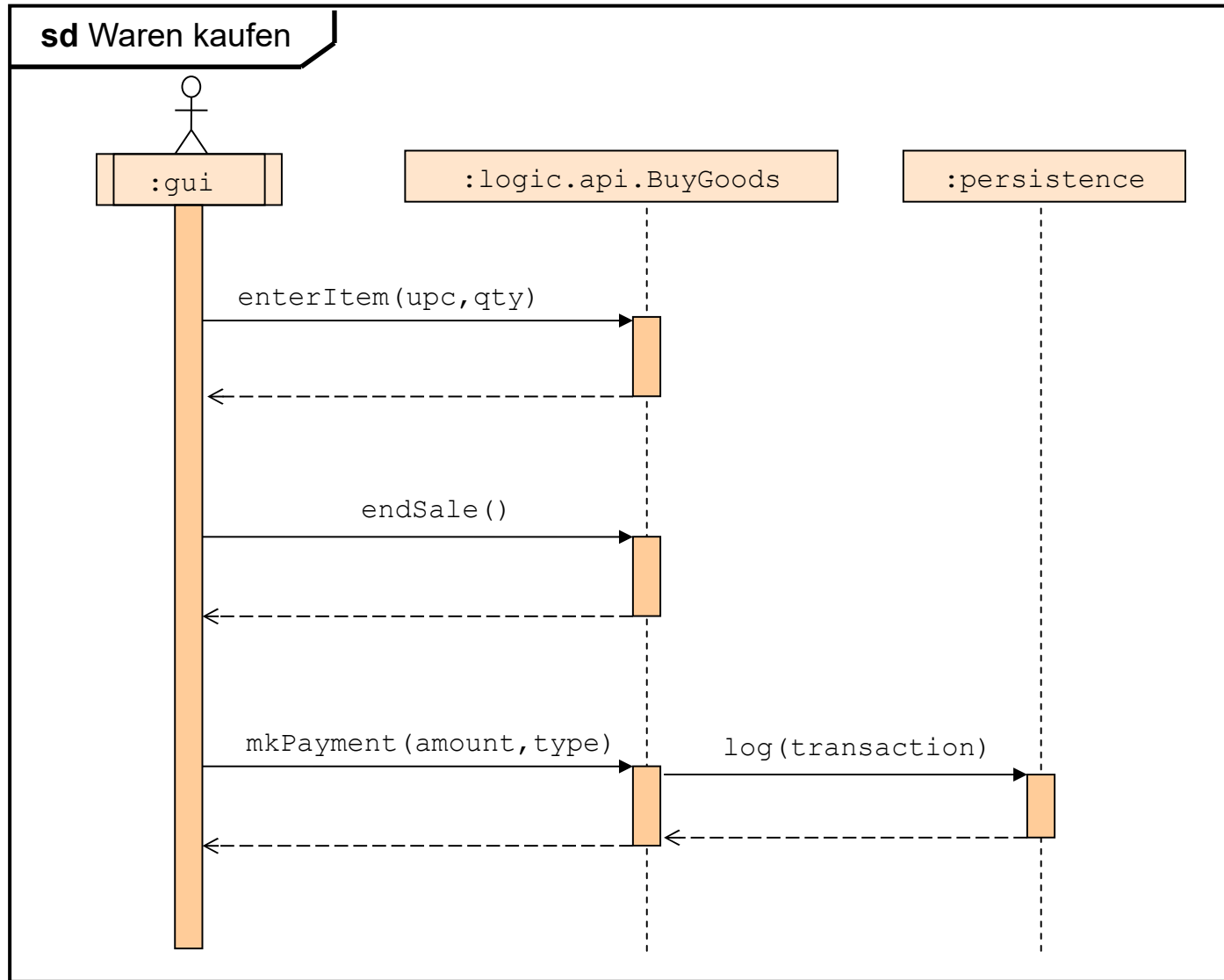
+K A Der Use Case „Waren kaufen“



+K A Der Use Case „Waren kaufen“ (Alternative)



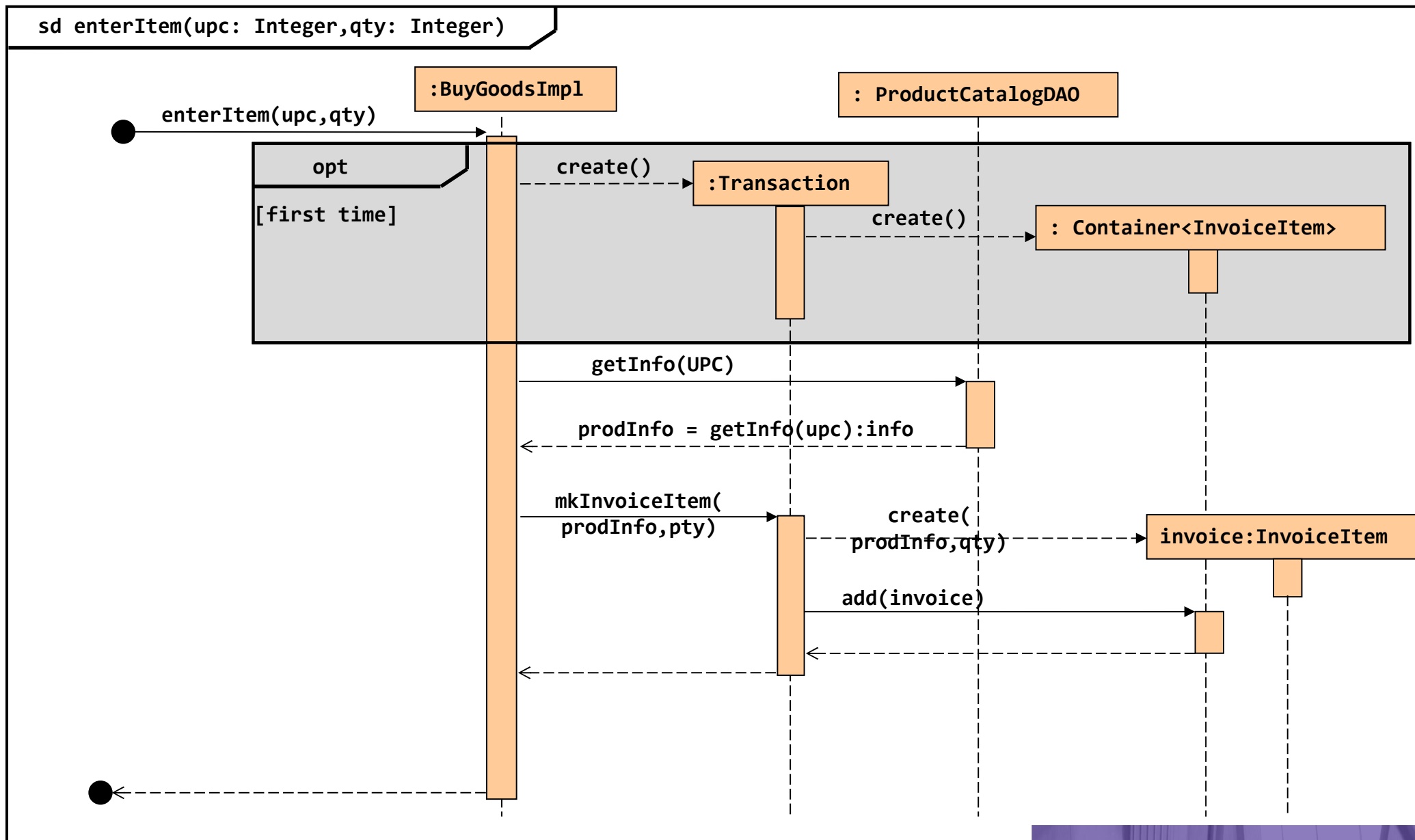
+HKA Die Schnittstelle



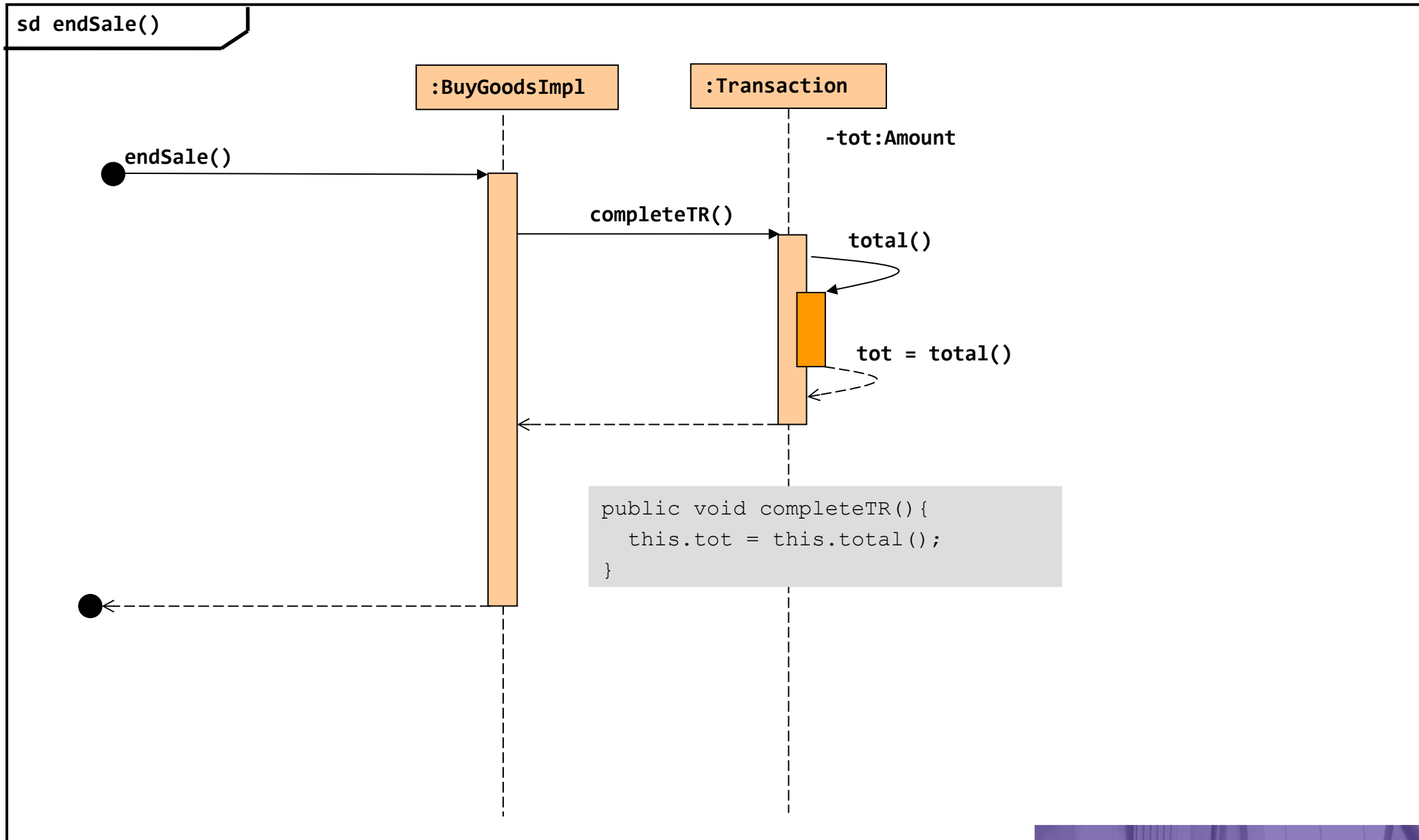
+IKA Domain Model



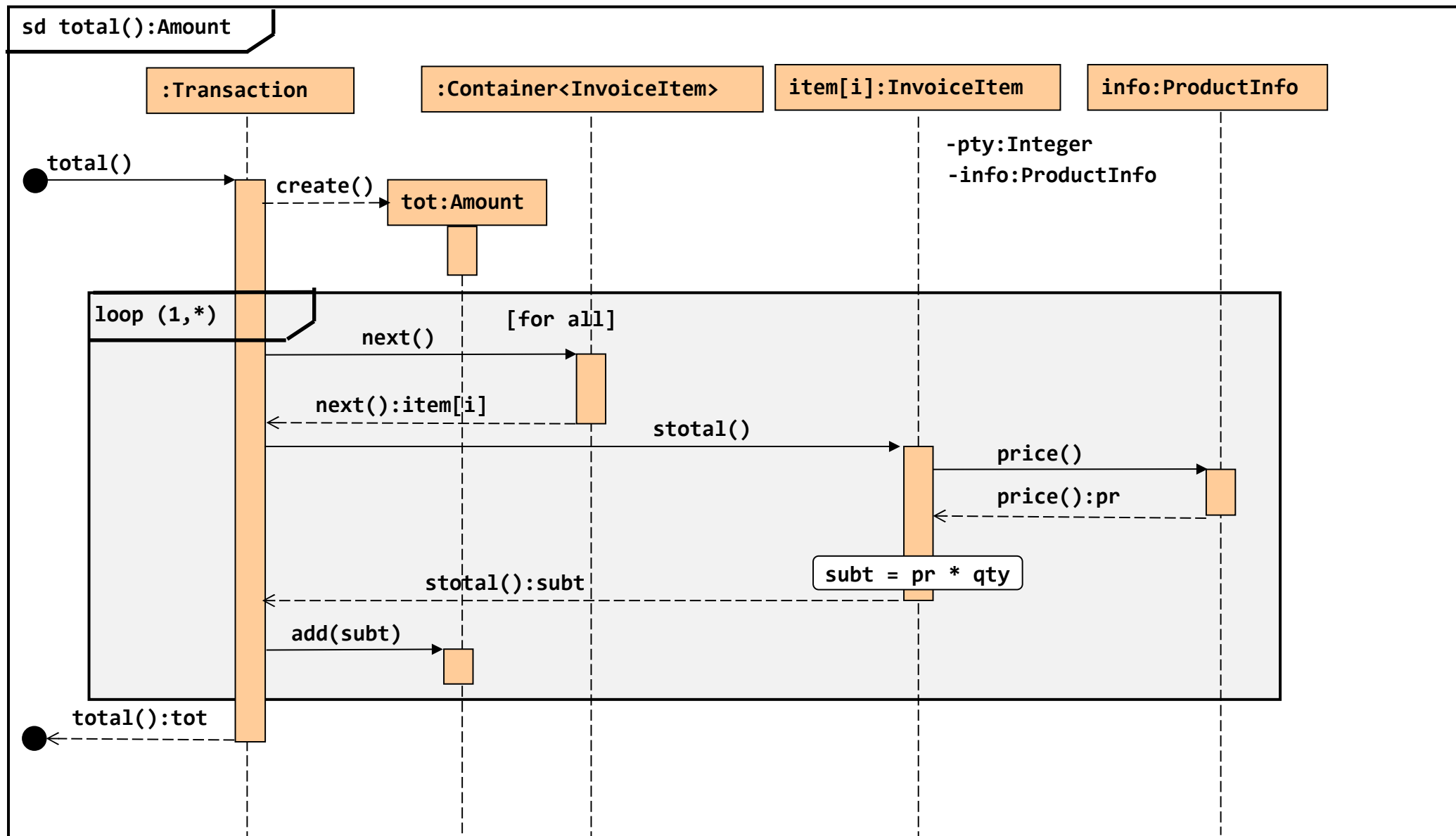
+K A enterItem() (Domain Model)



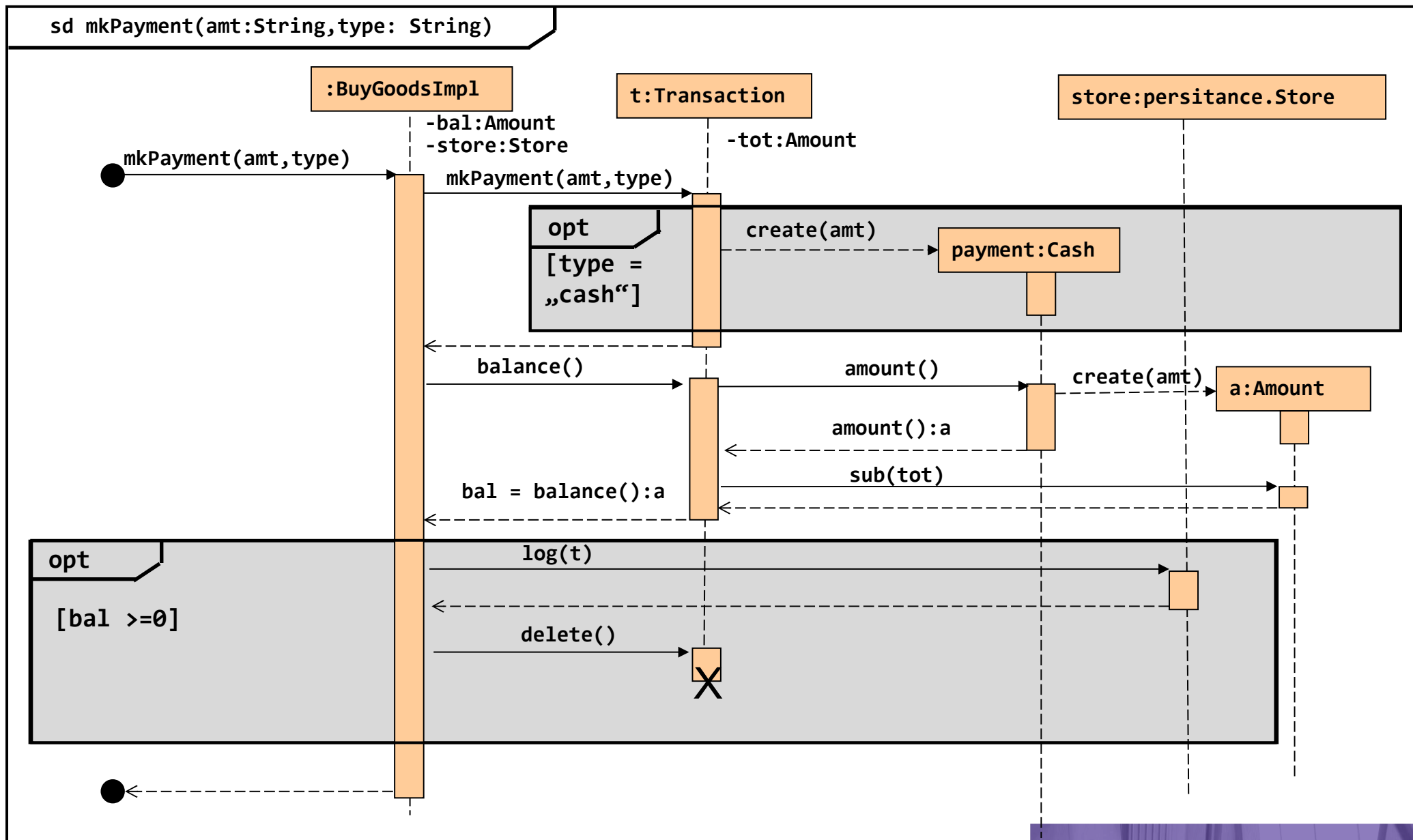
+K A endSale() (Domain Model)



+K A total () (Domain Model)

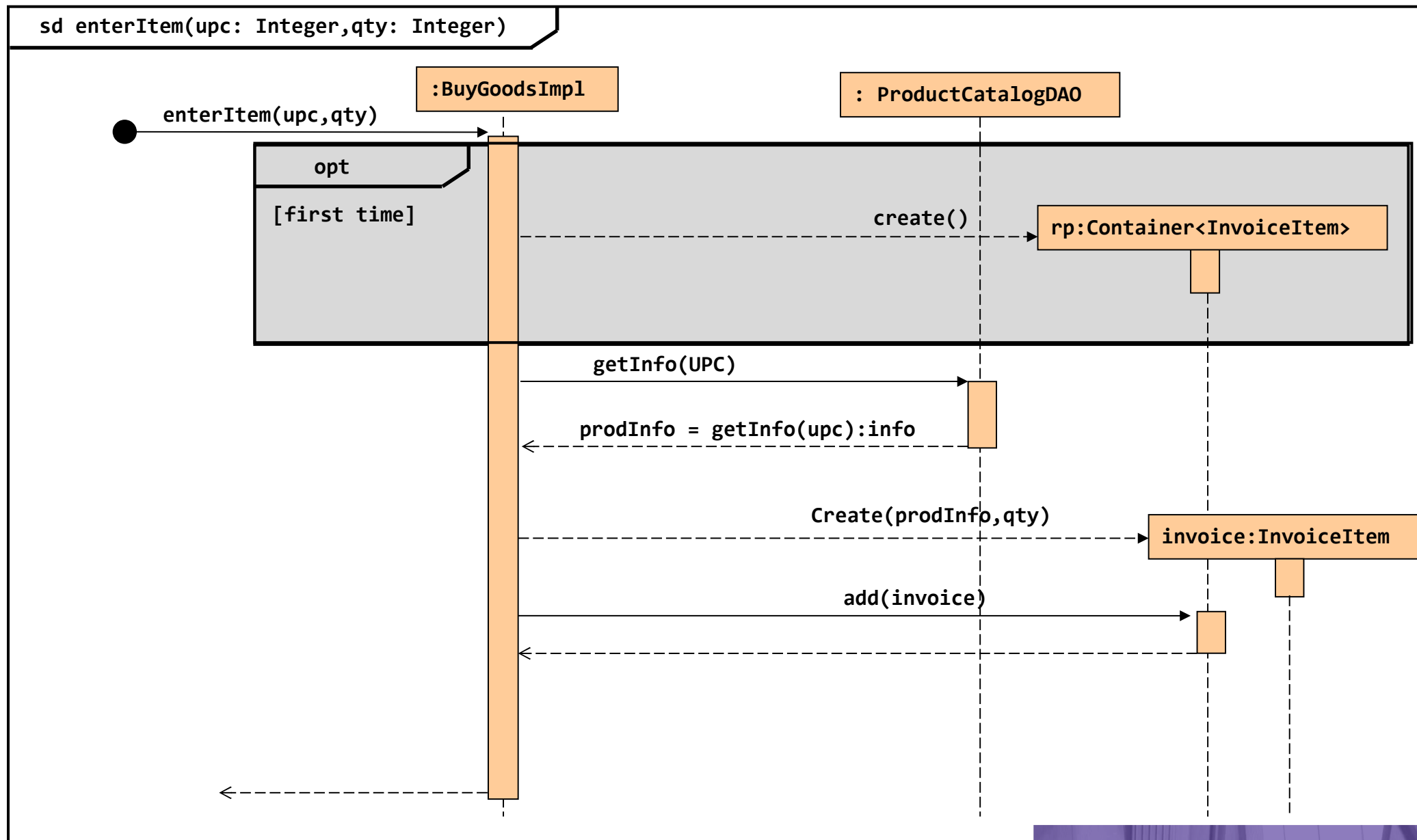


+K A mkPayment () (Domain Model)

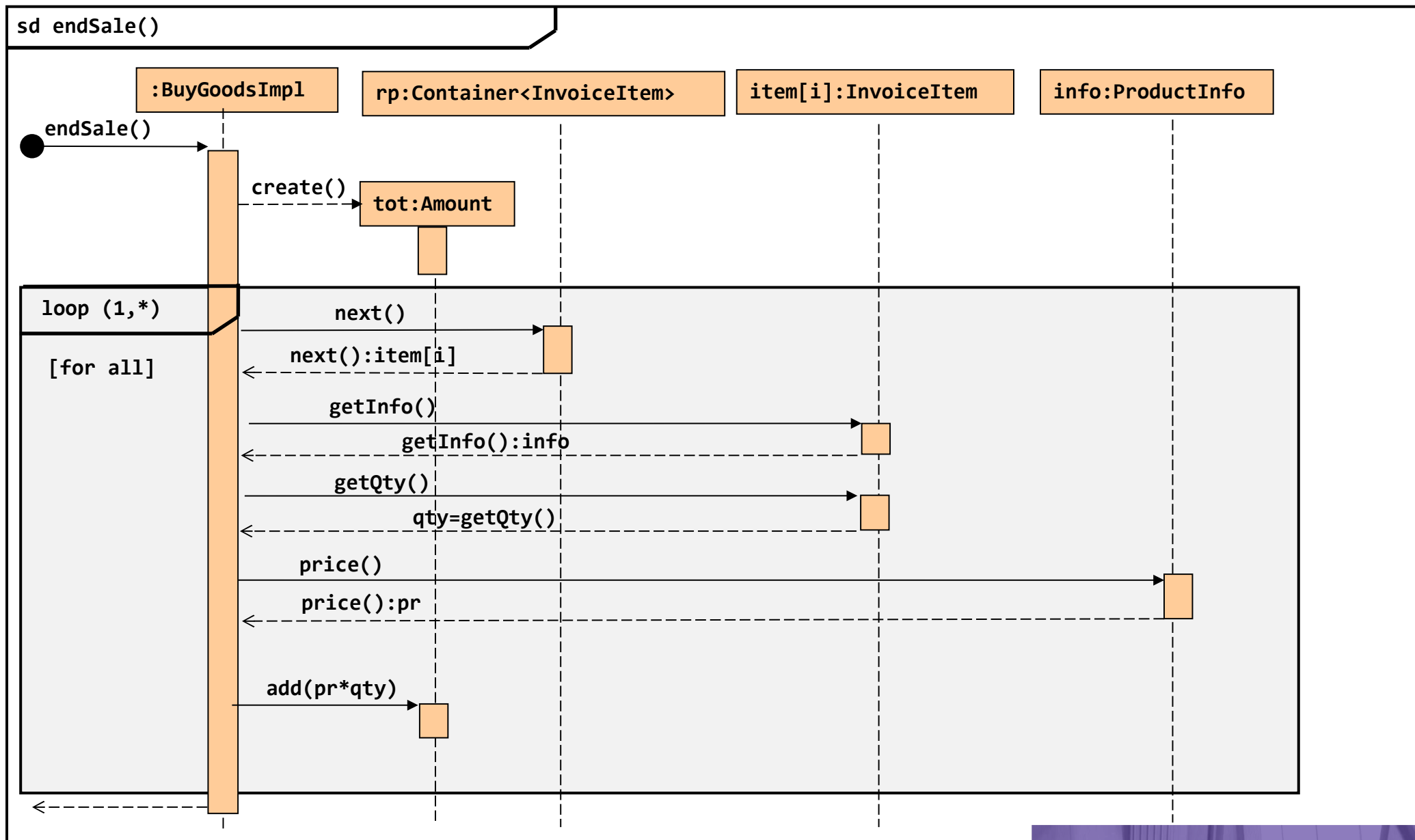


+HKA Transaction Script

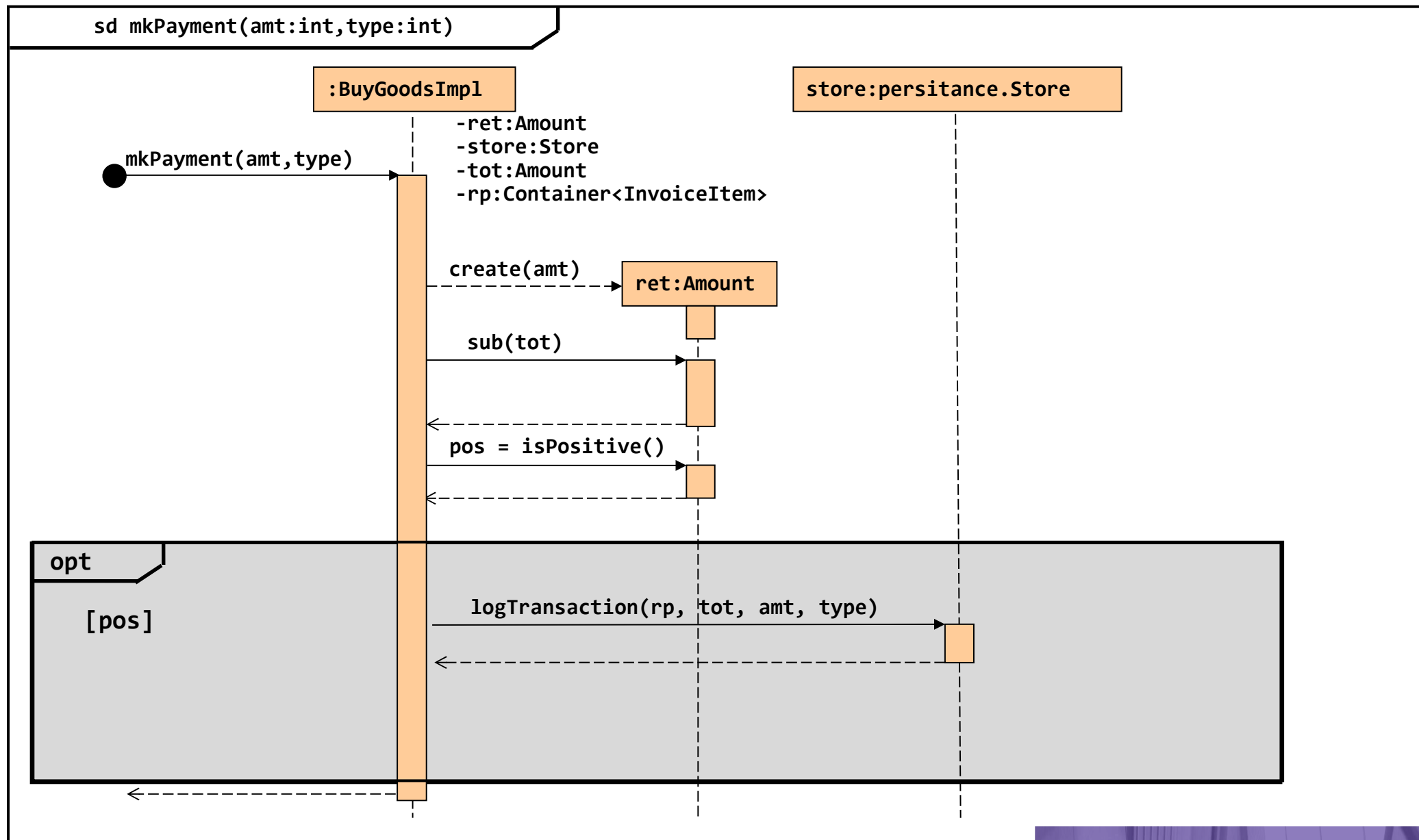
+K A enterItem() (Transaction Script)



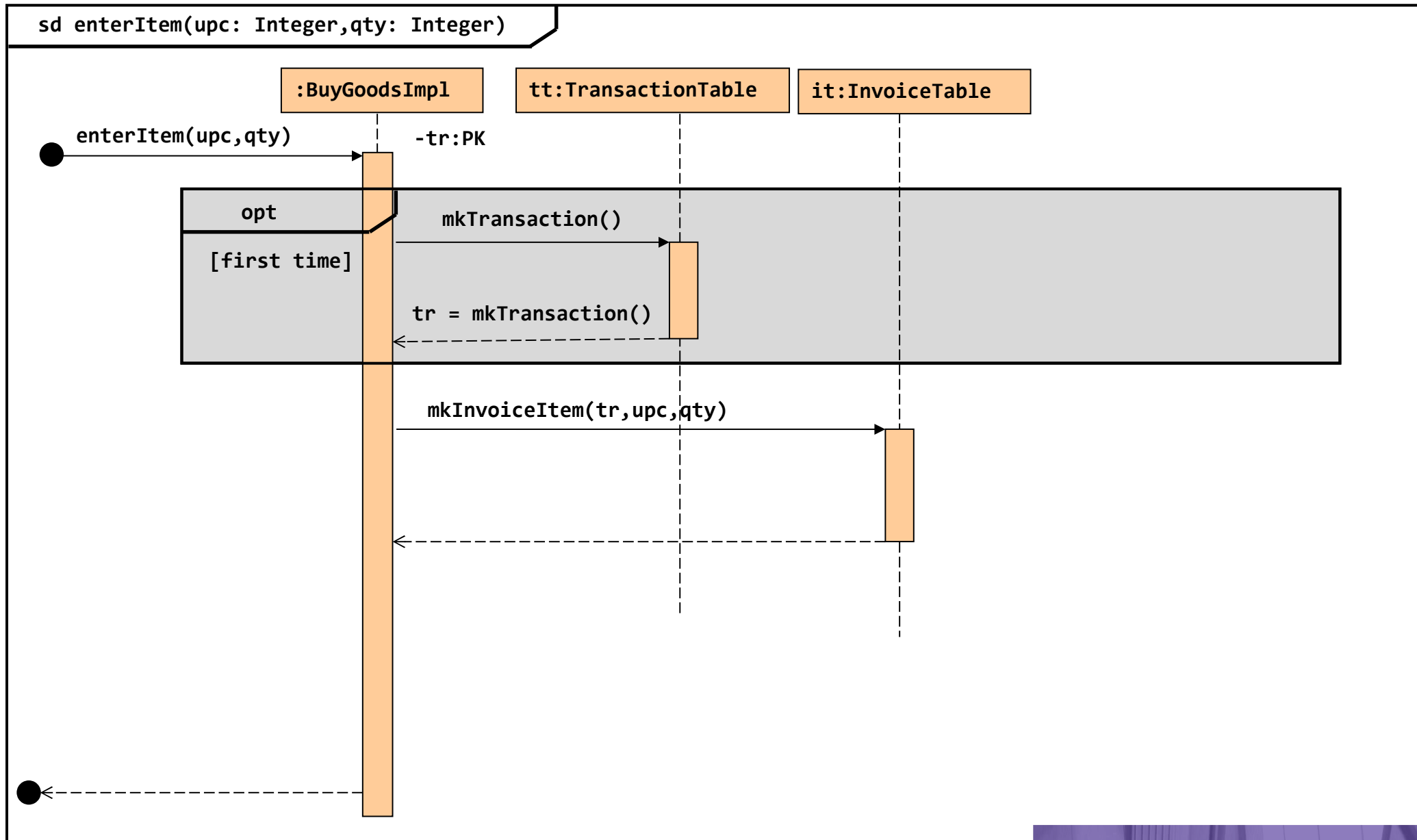
+K A endSale () (Transaction Script)



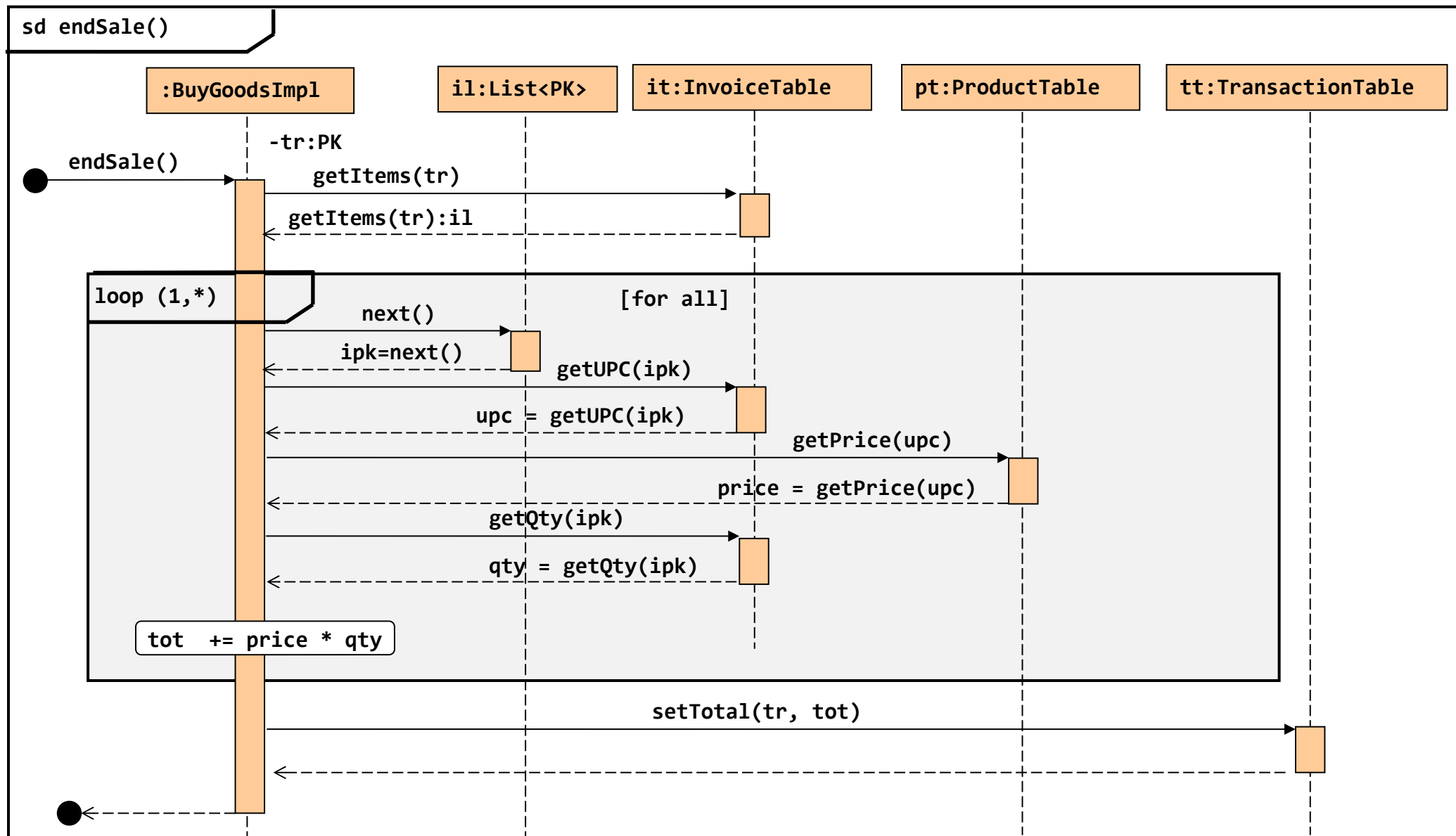
+K A mkPayment () (Transaction Script)



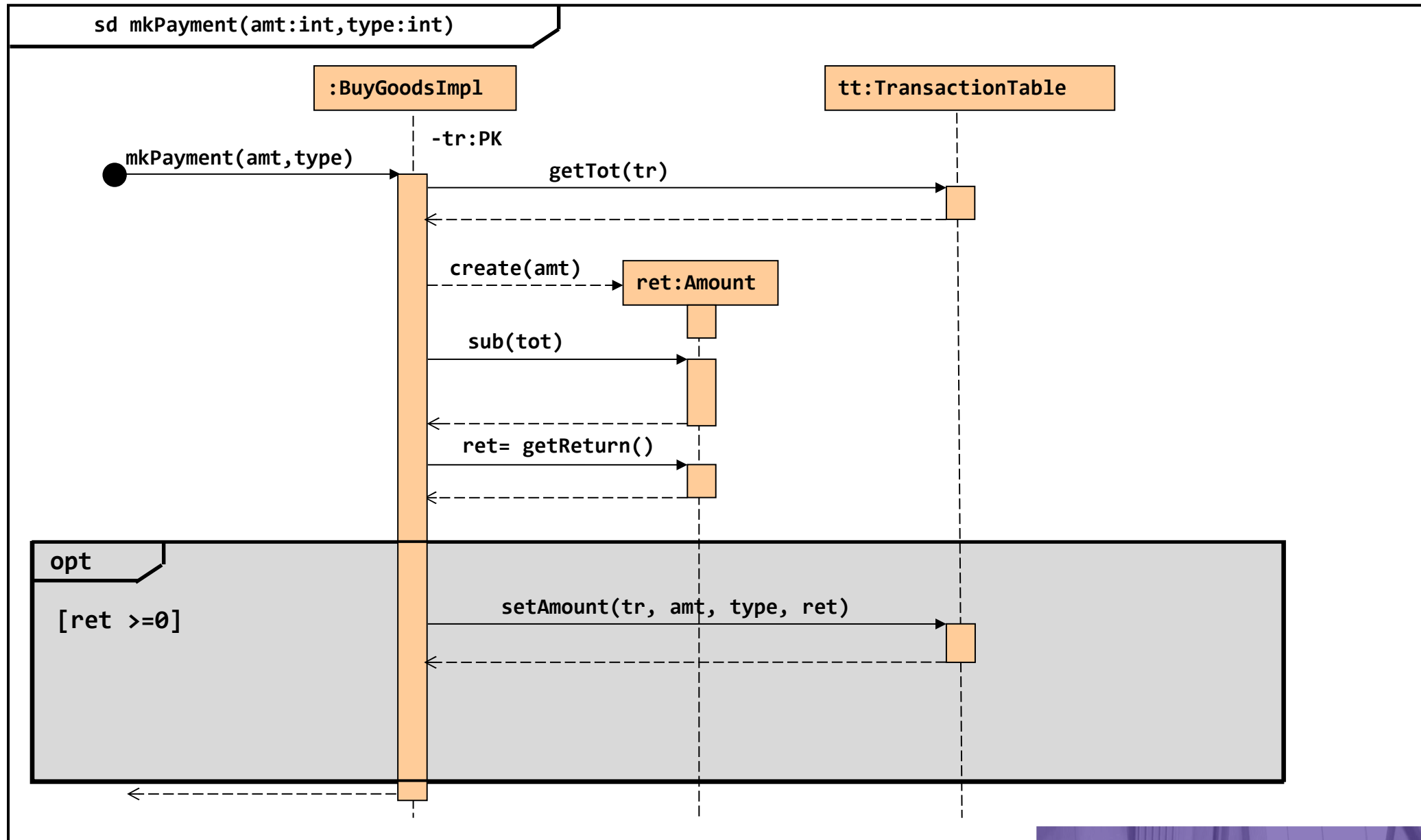
+K A enterItem() (Table Module)



+K A total () (Table Module)



+K A mkPayment () (Table Module)



+IKA Transaction Script

Allgemeines:

- Einfach und überschaubar:
Für jeden Request (Transaktion) gibt es genau eine Methode.
- Die Logik entspricht den durchzuführenden Transaktionen:
Aufgaben, die im Rahmen einer Transaktion durchgeführt werden müssen, werden alle von derselben Methode gekapselt.
- Oftmals erfolgt ein direkter Zugriff auf die Datenbank.

**Ideal für einfache Anwendungen
mit wenig Logik!**

+IKA Transaction Script

Umsetzung:

- Methoden auf Klassen verteilen, sodass zusammengehörende Funktionalität in einer gemeinsamen Klasse liegt.

Klassen realisieren Use Cases!

BuyGoodsImpl
enterItem(upc:int,qty:int) endSale() mkPayment(amt:int,type:int)

Vorteile:

- einfach, simpel, klar
- kaum Overhead

Nachteile:

- Nimmt die Komplexität zu, geht die Übersicht schnell verloren.
- Code-Verdopplung ist an der Tagesordnung.

**Use-Case-übergreifende, gemeinsame Funktionalität kann nur schwer ausgelagert werden
(kaum verfügbare Klassen).**

Allgemeines:

- Zentrale Teile des Problemraums werden in den Lösungsraum überführt.
- Aufgaben werden von den Objekten bearbeitet, denen sie zugeordnet sind.
- Das Problem wird gelöst durch „Simulation der Realität“.

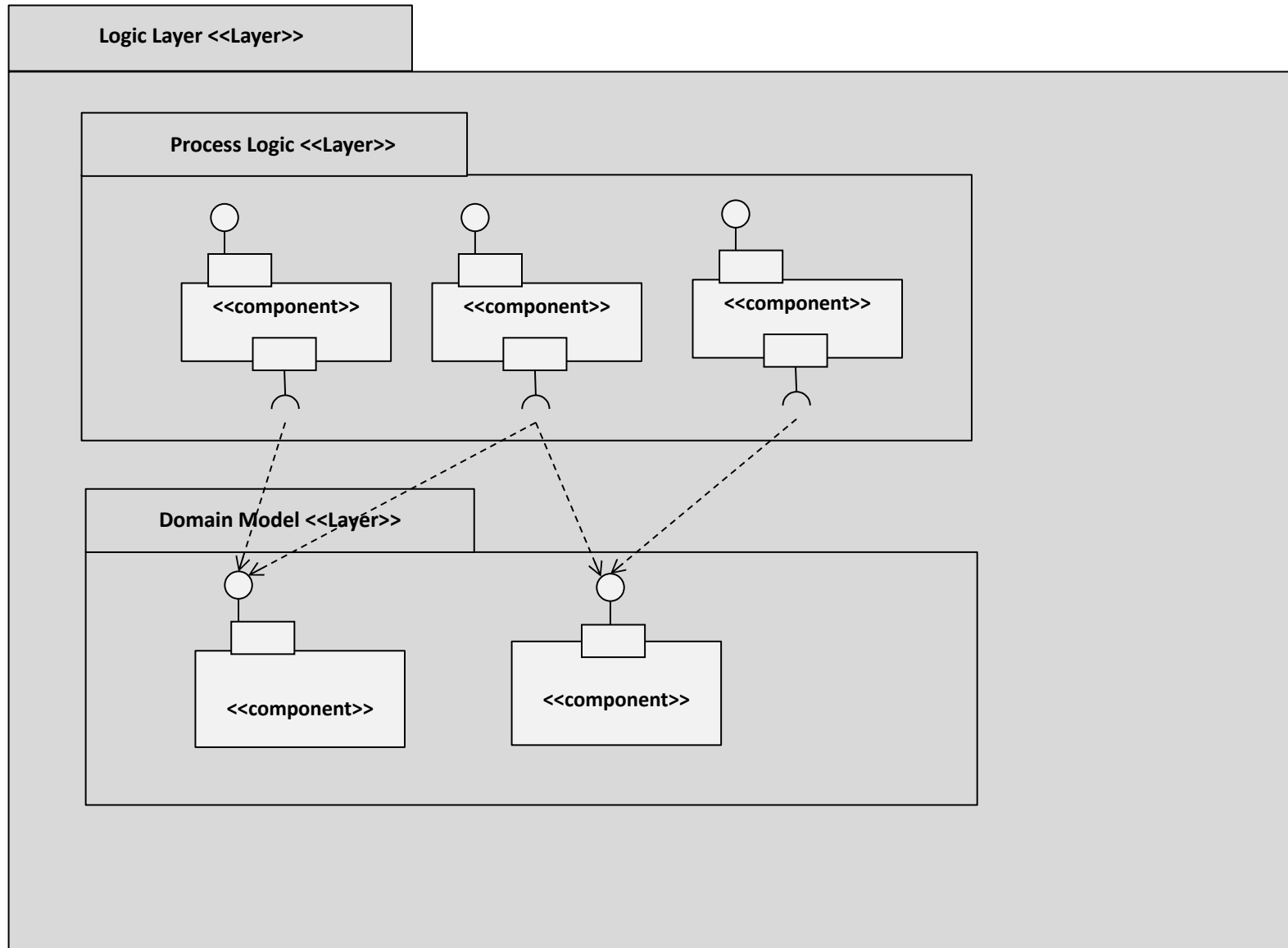
**Das Programm ist Abbild der realen Welt.
Die Zerlegung sollte intuitiv und offensichtlich sein.**

Umsetzung:

- Es empfiehlt sich einen eigenen Layer für die Domainobjekte zu erstellen.
- Daten und Funktionalität gilt es geeignet zu verbinden:
 - **Experten-Regel:** Verantwortlichkeit dem Informationsexperten geben. Der Klasse, die alle notwendigen Informationen hat.
 - **Lose Kopplung:** Verantwortlichkeiten so zuweisen, dass nicht jeder jeden kennt.
 - **Hohe Kohäsion:** Verantwortlichkeiten so zuweisen, dass nicht einer alles macht.

In einfachen Domain-Modellen repräsentiert eine Klasse oftmals genau eine Datenbanktabelle.

+IKA Layers: Umsetzung



Nachteile:

- Die Entscheidung, wann ein Objekt erzeugt wird und wann es wieder gelöscht werden kann, ist nicht trivial:
 - Session-Information ist notwendig.
 - Session-Information muss eventuell gespeichert werden.
- Die Verbindung von Daten und Funktionalität führt oftmals zu überladenen Klassen.

Funktionalität, die nur für einen Use Case relevant ist, muss nicht in Domänenobjekten gekapselt werden!

Vorteile:

- Lösungen bleiben überschaubar und verständlich.
- Aufgaben werden dort durchgeführt, wo sie anfallen.
- ...

Durch die Bereitstellung unterschiedlicher Interfaces. Lassen sich Aufgaben kapseln und Use-Case-spezifisch strukturieren.

Allgemeines:

- Direkte Abbildung der relationalen Datensicht in die Geschäftslogik.
- Für Datenbanktabellen, Views und Anfragen gibt es eigene Klassen.
- Jede dieser Klassen kapselt ihre eigene Funktionalität.

Die Funktionalität ist immer noch spezifisch.

Vorteile:

- Der Zugriff auf die relationale Datenbank wird erleichtert.

**Objekte repräsentieren
Datenbankanfragen.**

- Funktionalität und Daten bleiben gekapselt.
- Tabellen können an die Bedürfnisse der Anwendung angepasst werden (Views).

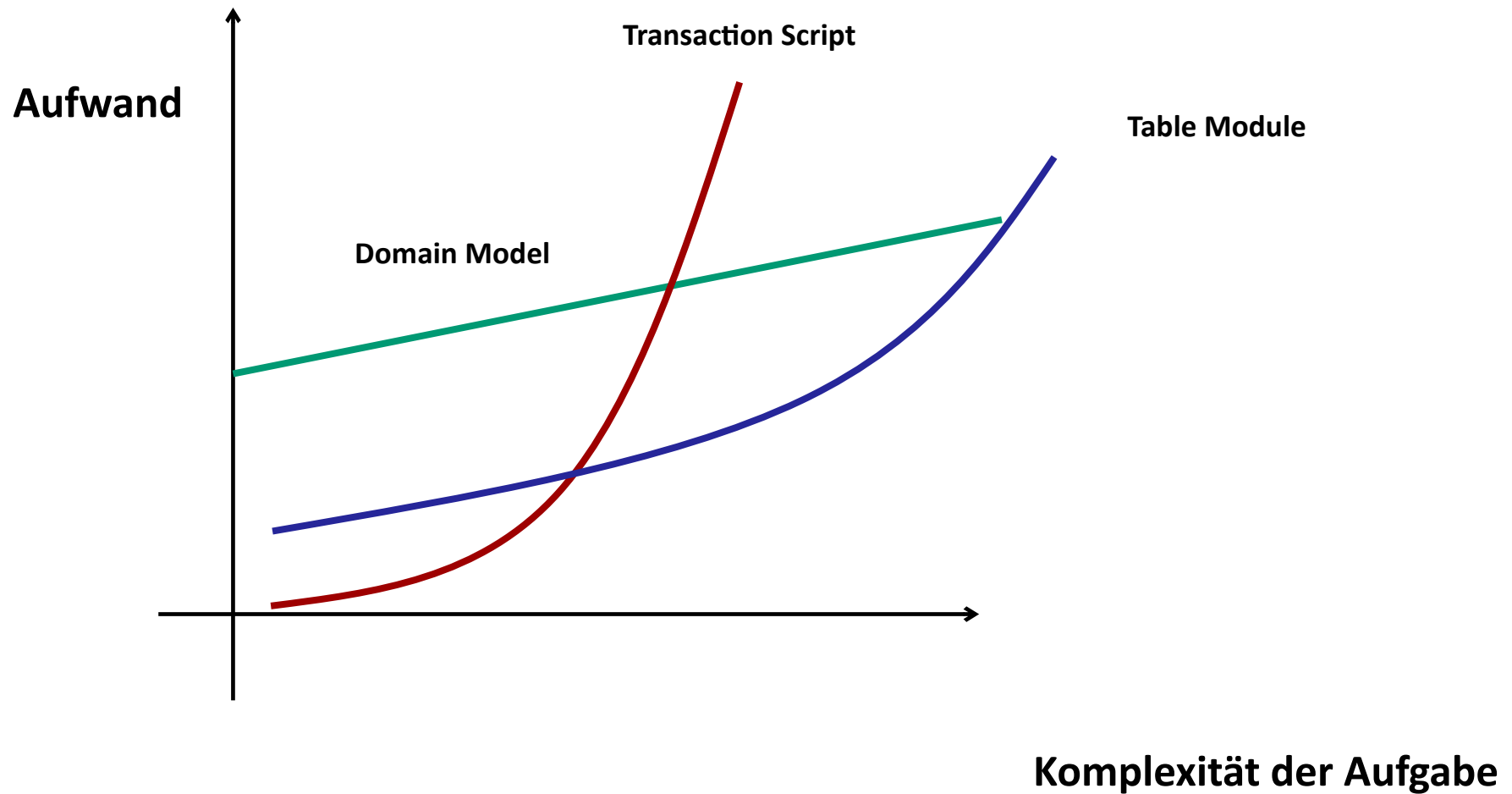
**Ideal für kleine, schlanke,
datenbankorientierte Anwendungen!**

Nachteile:

- Eine komplexe Geschäftslogik lässt sich nur schwer auf Tabellen verteilen.
- Das gleichzeitige Halten von Views und/oder Assoziationstabellen führen leicht zu Inkonsistenzen.

**Die Verbindung zur Datenbank muss
gut geplant sein!**

+IK A Aufwand im Verhältnis zur Komplexität



Nach [Fowler, Martin. Patterns of Enterprise Application Architecture. Addison-Wesley, 2003.]

Tipp: Viele reale Probleme liegen auf der Skala links!



Software-Architekturen

—

Zwischen Architektur und Design: Interfaces und ihre Ausprägungen

Prof. Dr. Thomas Fuchß

Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
Informatik und
Wirtschaftsinformatik

**Wie alle Patterns sind auch Entwurfsmuster bewährte
Lösungen für immer wiederkehrende Probleme.**

**Entwurfsmuster sind sowohl im Kleinen als auch im Großen
gewinnbringend einsetzbar.**

**Sie ermöglichen es nicht-funktionale
Requirements auf Code-Ebene
umzusetzen.**

Ein gelungener Entwurf zeichnet sich meistens durch folgende Punkte aus:

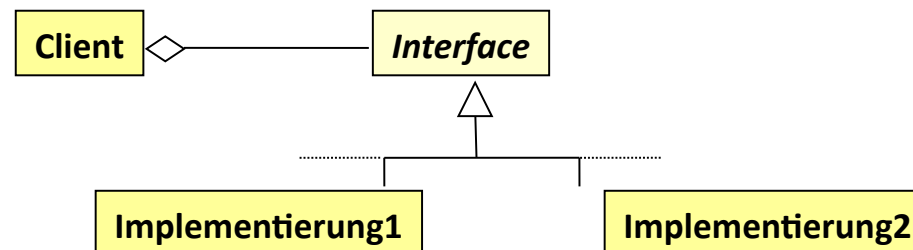
- Die Trennung zwischen Interface und Implementierung.
- Die Trennung zwischen Allgemeingültigem und Spezifischem:
 - **Allgemeingültig bedeutet stabil.**
 - **Spezifisch bedeutet variabel.**
- Das Verbergen einer spezifischen Implementierung hinter einem allgemeingültigen Interface:
 - **Spezifisches wird austauschbar.**

Die Trennung eines Problems in allgemein und spezifisch ist immer abhängig vom Problem, selten vollständig und dient nie dem Selbstzweck.

+HKA Zentrale Gedanken (Gama et. al.)

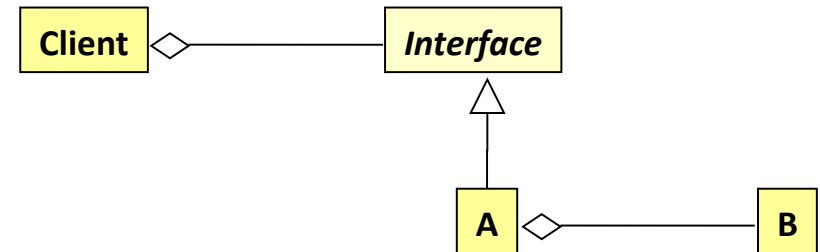
**Man programmiert auf eine Schnittstelle hin,
nicht auf eine Implementierung.**

- Klienten wissen nichts über die konkreten Klassen der Objekte, die sie benutzen.
- Klienten kennen nur deren Schnittstellen.

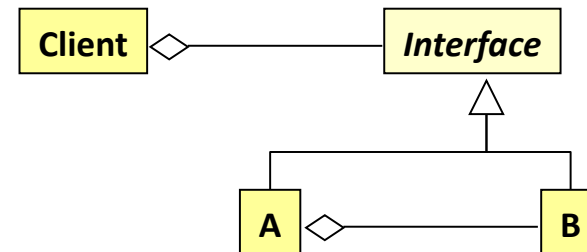


+IKA Zentrale Gedanken

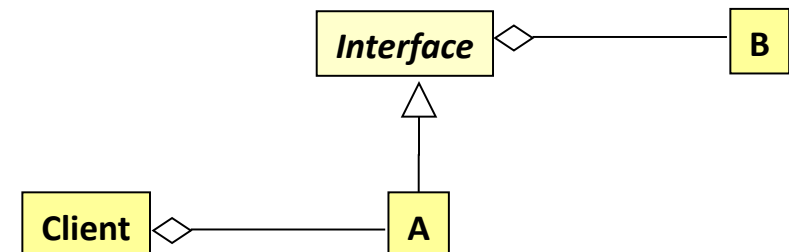
- Schnittstellen müssen angepasst werden.



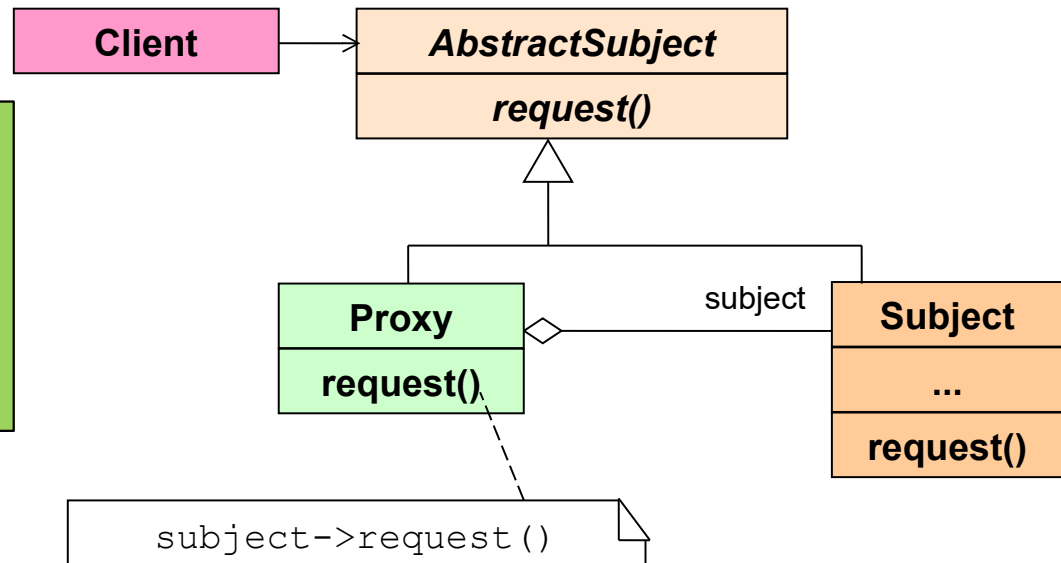
- Schnittstellen müssen zugänglich gemacht werden.



- Schnittstellen müssen verkörpert werden.

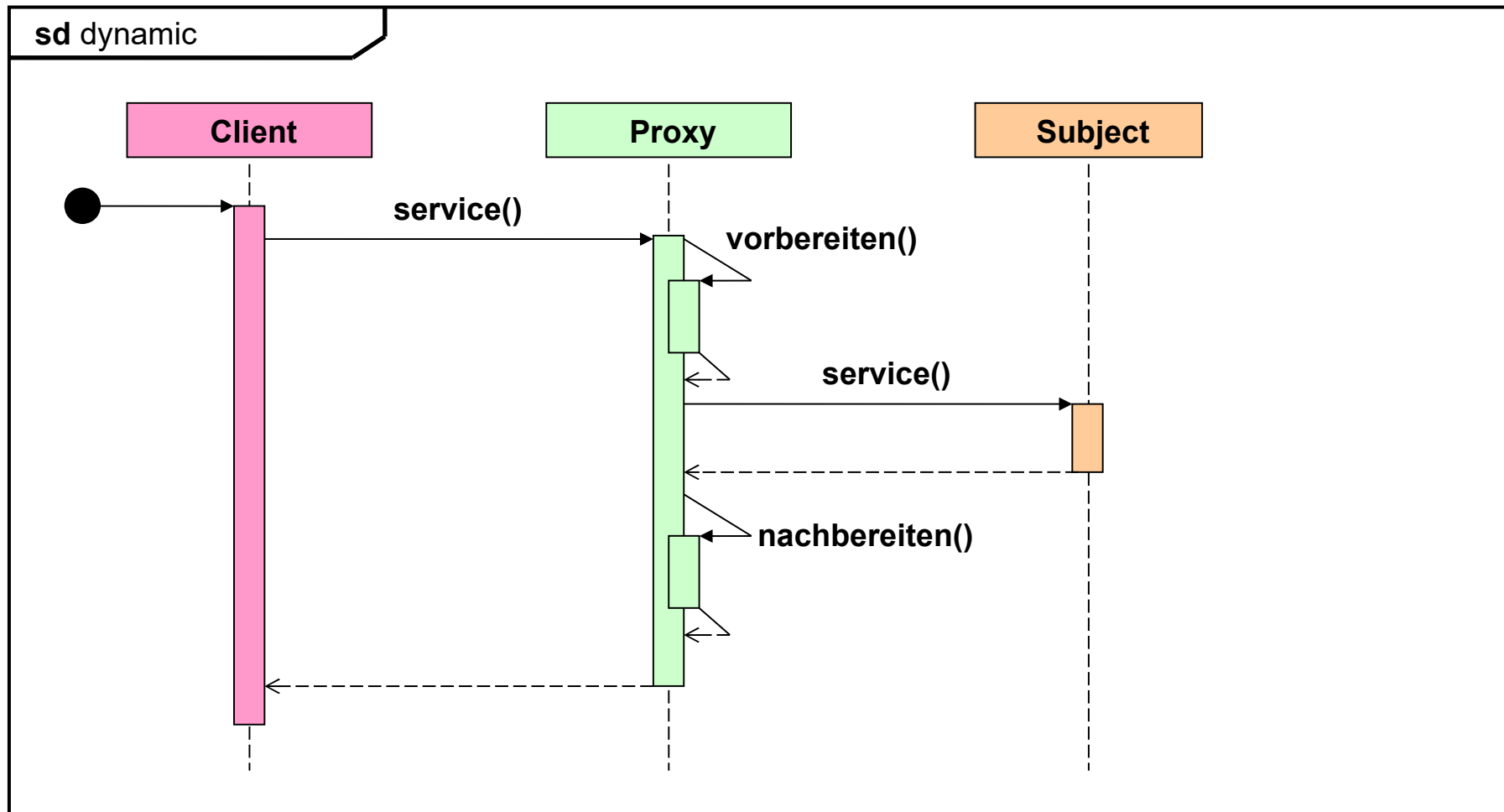


Ein Proxy fungiert als
Platzhalter für ein echtes
Objekt und kontrolliert den
Zugriff!



- Proxy (Stellvertreter) und Original (Subject) haben dasselbe Interface.
- Im Allgemeinen fügt ein Proxy keine neue Funktionalität hinzu. Ein Proxy erleichtert bzw. verbessert lediglich den Zugriff auf das Original.

+IK A Dynamisches Verhalten



- **Virtueller Proxy (Lazy Evaluation):**

Ein virtueller Proxy erzeugt die Objekte erst dann, wenn sie gebraucht werden. Typisches Anwendungsszenario: Initialisierung von Komponenten

```
T f() {return ((this.y==null) ? this.y = new YImpl() : this.y).f();}
```

- **Schutz-Proxy:**

Ein Schutz-Proxy überwacht den Zugriff.

```
T f() { return (this.state.equals(S.allowed)) ? this.y.f() : null;}
```

- **Cache-Proxy:**

Ein Cache-Proxy vermeidet unnötige Kommunikation.

```
T get(Key k) {  
    T res = this.cache.get(k);  
    if (res == null) this.cache.put(k, res = this.y.get(k));  
    return res;}  

```

Synchronisierung:

Im Rahmen einer interaktiven Interpreter-Anwendung, wird zur Datentransformation ein Parser eingebunden, dessen Funktionalität dem Anwender direkt an der Oberfläche angeboten werden soll.

```
interface IParser{  
    ...  
    public setInput(String txt);  
    public void parse();  
    public typeCheck();  
    public void optimize(int level);  
    public void makeCode();  
}
```

Problem:

Parse-Vorgang, Typüberprüfung, Code-Erzeugung, usw. sind zeitintensive Vorgänge, die einer asynchronen Anbindung an die Oberfläche bedürfen. D. h., es muss sichergestellt werden, dass der Parser „thread-safe“ ist.

Lösung: Ein Proxy mit Monitor!

```
class ParserProxy implements IParser{  
    Parser parser;  
    Object monitor;  
  
    public ParserProxy( Parser p) {  
        this.parser = p;  
        this.monitor = new Object();  
    }  
}
```

Original



```
public void parse(){
    synchronized ( this.monitor ) { this.parser.parse(); }
}
public void setInput(String txt){
    synchronized ( this.monitor ) { this.parser.setInput(txt); }
}
public void typeCheck() {
    synchronized ( this.monitor ) { this.parser.typeCheck(); }
}
public void optimize(int level) {
    synchronized ( this.monitor ){this.parser.optimize(level);}
}
public void makeCode() {
    synchronized ( this.monitor ) { this.parser.makeCode(); }
}
```

Konsequenzen

Das Proxy-Muster führt eine zusätzliche Indirektionsstufe (Zwischenschicht) ein. Diese Zwischenschicht ermöglicht Anpassungen, die den Umgang mit dem Original erleichtern, z. B. **Thread-Sicherheit**.

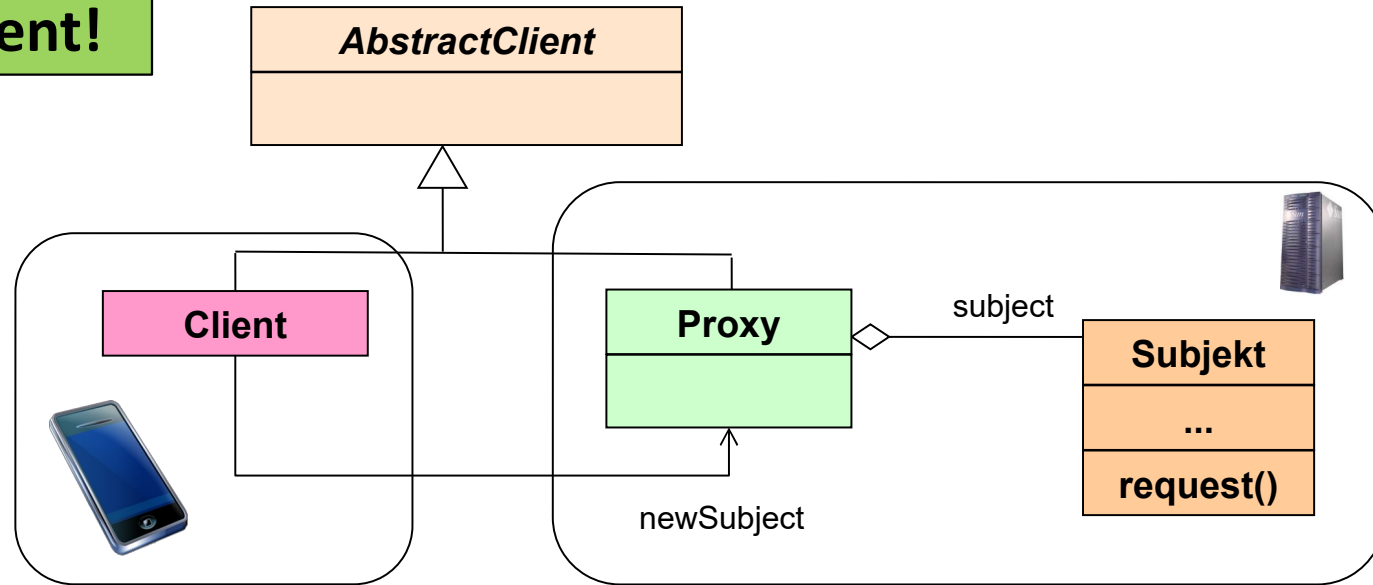
Anwendbarkeit

Das Proxy-Muster sollte immer dann eine Option sein, wenn Bedarf an einer **anpassungsfähigen und intelligenten Referenz** auf ein Objekt **(eine Komponente)** besteht.

Tipp:
**Intelligente Proxies bieten
Anknüpfungspunkte für
Adaptionsmechanismen.**

+IKA Reverse Proxy

Der Proxy vertritt den Client!



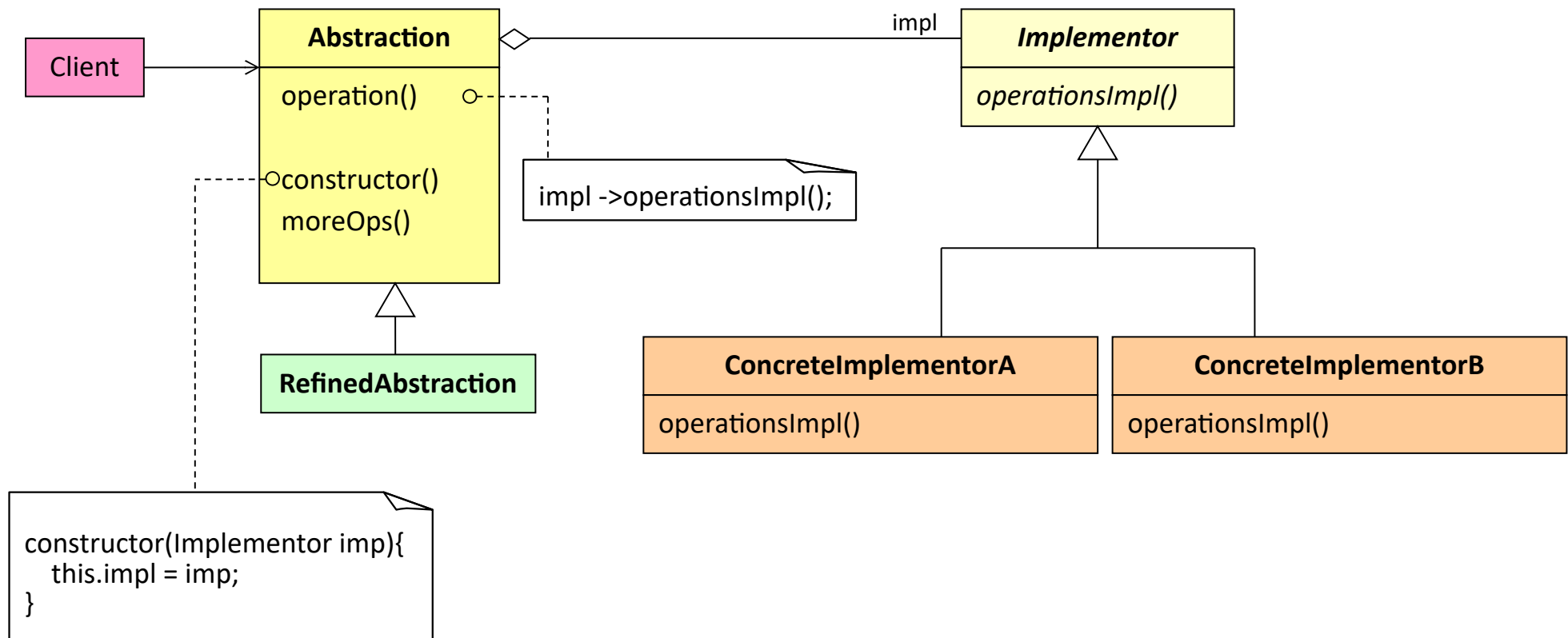
- Der Proxy agiert als Agent für den Client.
- Jede Kommunikation läuft über den Proxy.
 - Jeder Client bekommt seinen eigenen Agenten oder
 - viele Clients werden durch einen Agenten vertreten.
 - Agenten können dienstspezifisch sein oder Dienste kombinieren.

+IKA Reverse Proxy (Aufgaben des Agenten)

- **Caching, um Verbindungsabbrüche zu maskieren:**
 - Eingehende Nachrichten des Clients queuen und sukzessive weiterleiten.
 - Antworten des Servers entgegennehmen und vorhalten.
- **Antworten für den Client optimieren:**
komprimieren, priorisieren, filtern, paketieren, ...
- **Den Client über Ereignisse informieren:**
Änderung der Bandbreite, des Tarifs, ...
- **Spezifische Funktionalität auslagern:**
 - Für den Client: Auslagern von Anmeldevorgängen (Single Sign-on)
 - Für den Server: Organisation der Lastverteilung, Anonymisierung, ...

+IKA Brücke (Bridge)

Die Brücke trennt Interface (Abstraktion) und Implementierung, sodass beide unabhängig angepasst werden können. Die Abstraktion definiert und implementiert das Interface, während die eigentliche Implementierung ausgelagert wird.

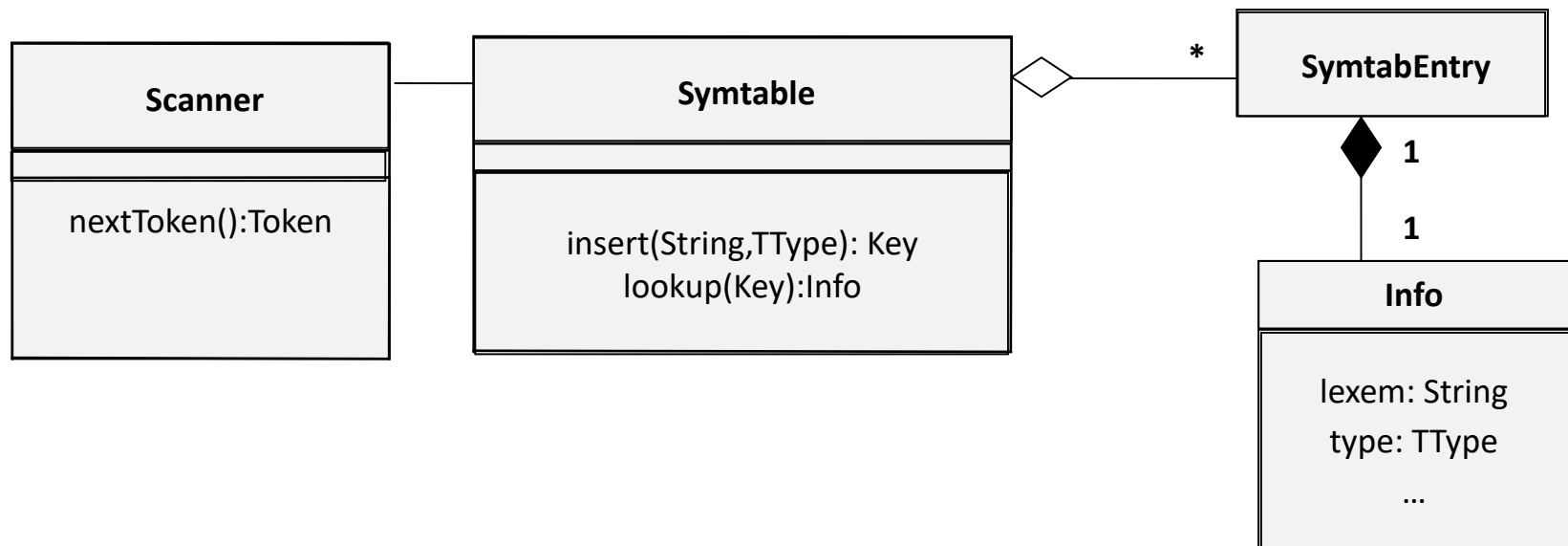


Man verwendet das Brückenmuster, wenn

- **eine permanente Bindung zwischen Abstraktion und Implementierung vermieden werden soll:**
 - Eine Abstraktion kann verschiedene Implementierungen nutzen.
 - Eine Implementierung kann in verschiedenen Abstraktionen benutzt werden.
- **der Austausch einer Komponente keinen Einfluss auf den Client haben soll:**
 - Der Code muss nicht neu übersetzt werden.
- **die Implementierung vollständig verborgen bleiben soll.**

Problem:

Für die Realisierung eines Parsers benötigt man eine Symboltabelle, in der die Informationen über die Tokens (Lexeme) abgelegt werden können.



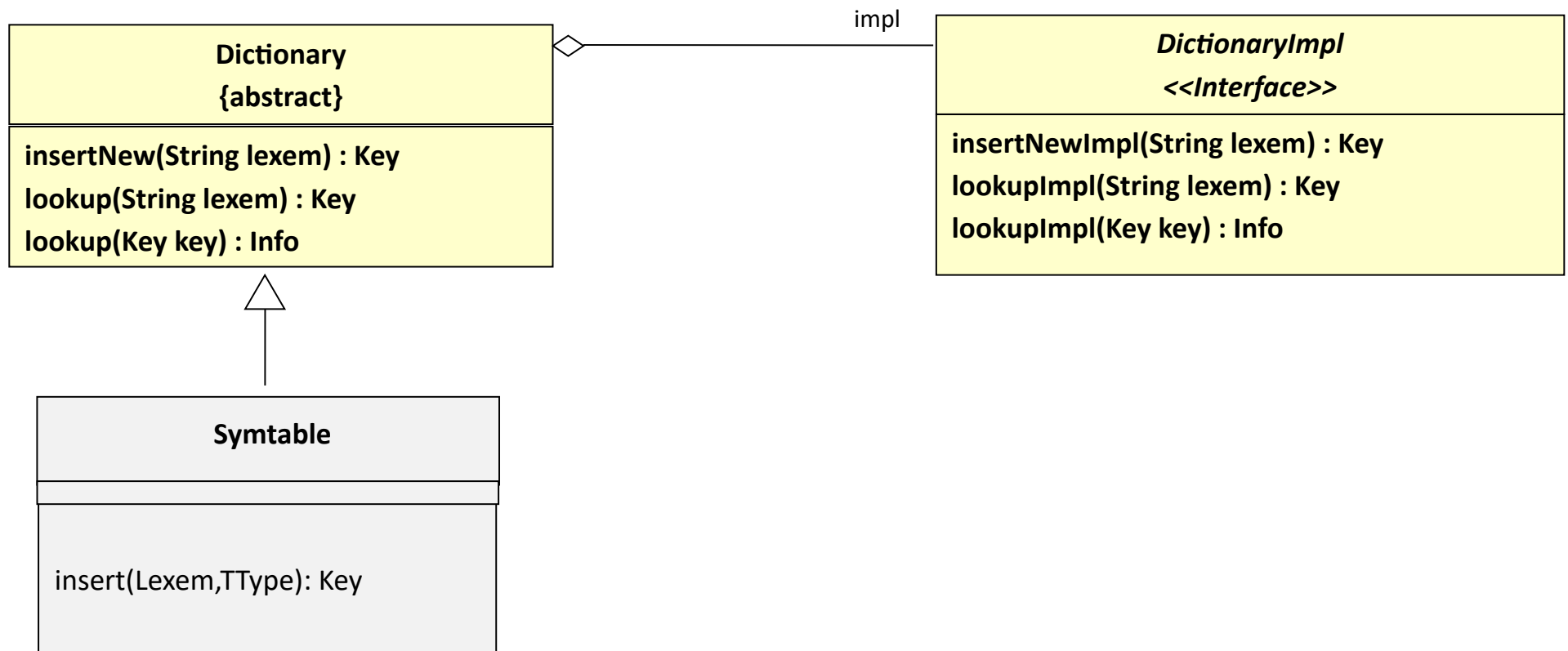
Idee:

Um eine möglichst große Freiheit und Sicherheit hinsichtlich der Umsetzung zu erlangen, entscheidet man sich dafür, die Implementierung hinter einer Brücke zu verstecken. Als Schnittstelle hierfür wählt man eine leicht vereinheitlichte Form, ein „Dictionary“.

Dictionary <<Interface>>
insertNew(String lexem) : Key lookup(String lexem) : Key lookup(Key key) : Info

Realisierung:

Damit ergibt sich für die zu realisierende Brücke folgende Struktur:



+K A Beispiel

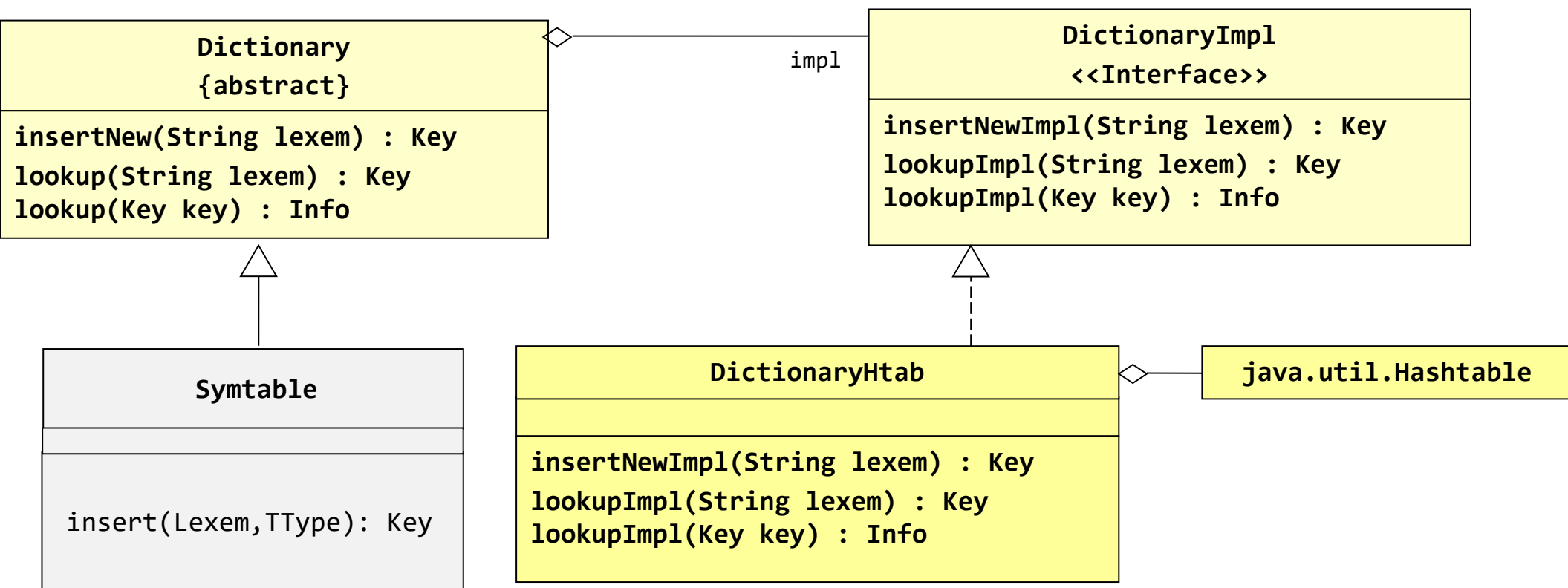
```
abstract class Dictionary {  
    private DictionaryImpl impl;  
    public Key insertNew(String lexem) {  
        return this.impl.insertNewImpl(lexem);  
    }  
    public Key lookup(String lexem){  
        return this.impl.lookupImpl(lexem);  
    }  
    public Info lookup(Key key){  
        return this.impl.lookupImpl(key);  
    }  
    public Dictionary(DictionaryImpl impl){  
        this.impl = impl;  
    }  
}
```

```
interface DictionaryImpl{  
    Key  insertNewImpl(String lexem);  
    Key  lookupImpl(String lexem);  
    Info lookupImpl(Key key);  
}
```


+IKA Beispiel

```
class Symtable extends Dictionary{  
  
    public Symtable(DictionaryImpl impl) {  
        super(impl);  
    }  
  
    public Key insert(String lexem, TType type){  
        Key k = this.lookup(lexem);  
        if (k == null){  
            k = this.insertNew(lexem);  
            this.lookup(k).setType(type);  
        }  
        return k;  
    }  
}
```

Aus Zeitgründen entscheidet man sich vorerst, statt einer eigenen „effizienten“ Implementierung, für eine Java-Hash-Tabelle. Dank der Brücke, bleibt diese Entscheidung verborgen und kann später revidiert werden.



```
class DictionaryHtab implements DictionaryImpl{
    private Hashtable htab = new Hashtable();

    public Key insertNewImpl(String lexem) {
        if (this.htab.containsKey(lexem))
            return null;
        Key key = new Key();
        Info info = new Info(lexem);
        this.htab.put(lexem, key);
        this.htab.put(key, info);
        return key;
    }

    public Key lookupImpl(String lexem) {return (Key)this.htab.get(lexem);}

    public Info lookupImpl(Key key) {return (Info) this.htab.get(key);}
}
```

Eine Alternative mit zwei typisierten Hashtabellen.

```
class DictionaryHtab implements DictionaryImpl{
    private HashMap<Key,Info> htab = new HashMap<>();
    private HashMap<String,Key> keyTab = new HashMap<>();

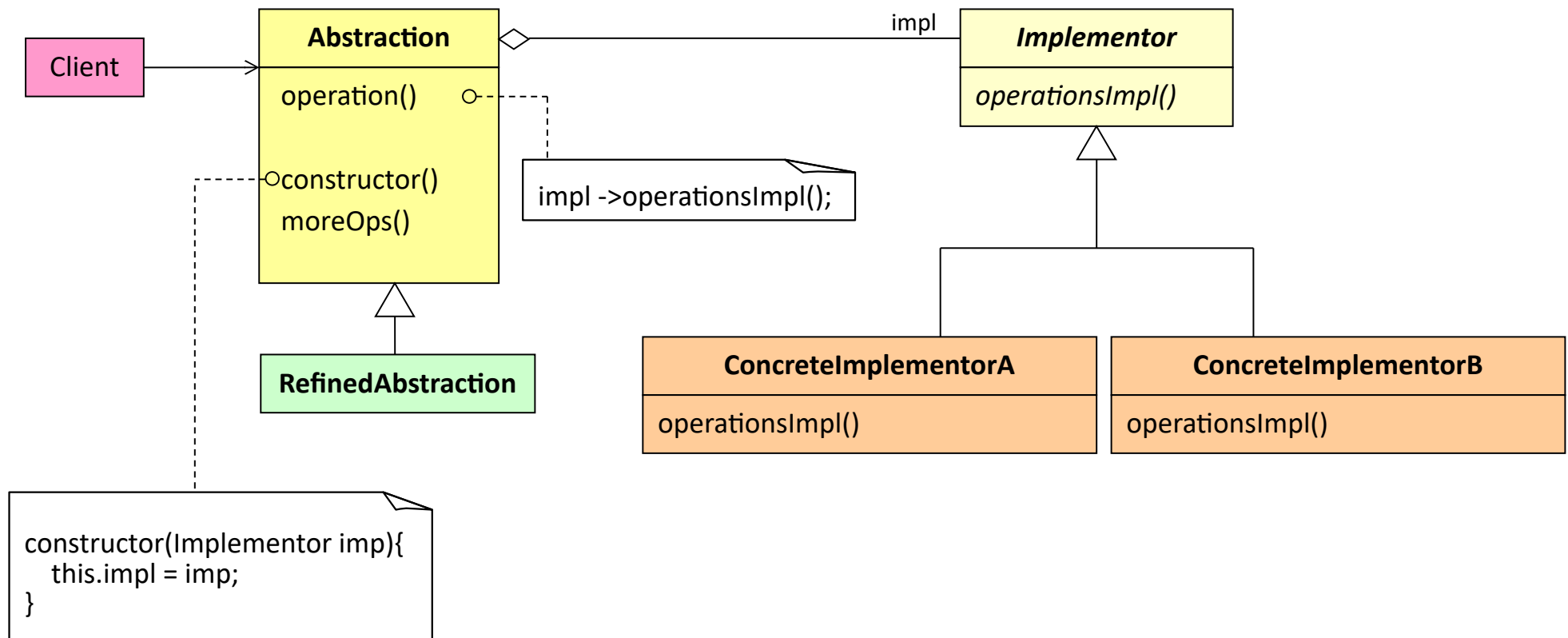
    public Key insertNewImpl(String lexem) {
        if (this.keyTab.containsKey(lexem))
            return null;
        Key key = new Key();
        Info info = new Info(lexem);
        this.keyTab.put(lexem, key);
        this.htab.put(key, info);
        return key;
    }

    public Key lookupImpl(String lexem) {return this.keyTab.get(lexem);}

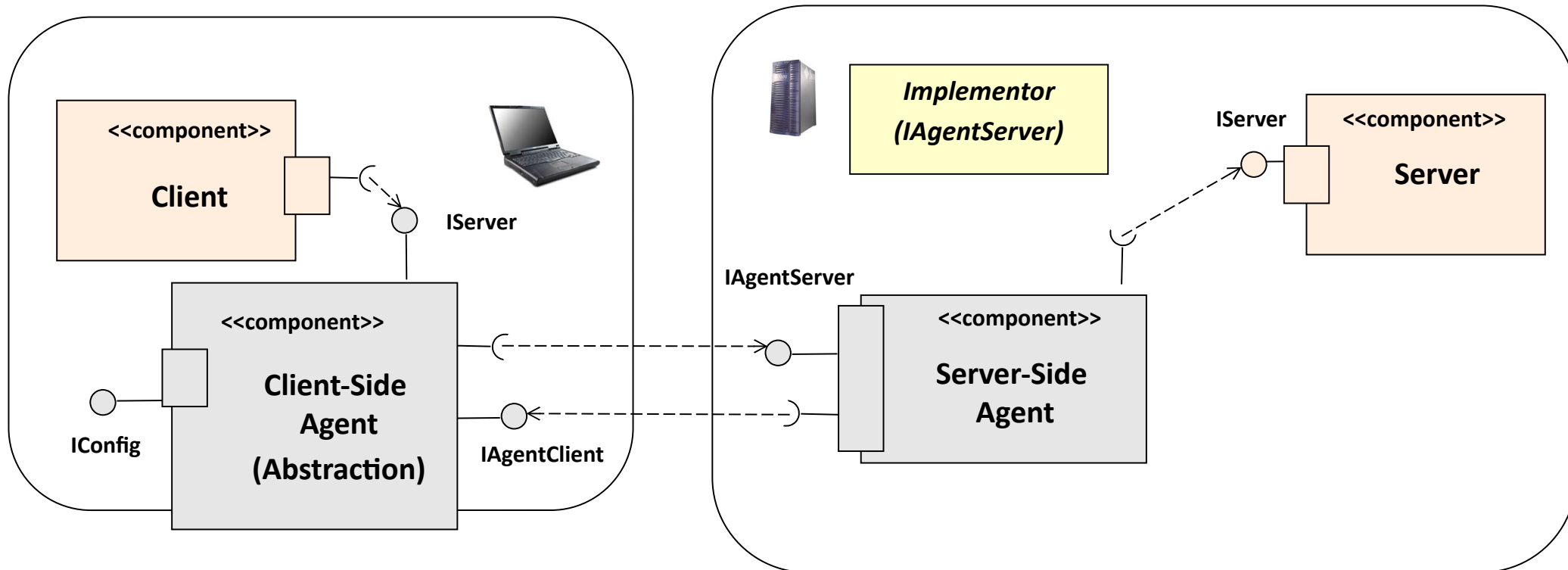
    public Info lookupImpl(Key key) {return this.htab.get(key);}
}
```

+IKA Brücke (Bridge)

Die Brücke trennt Interface (Abstraktion) und Implementierung, so dass beide unabhängig angepasst werden können. Die Abstraktion definiert und implementiert das Interface, während die eigentliche Implementierung ausgelagert wird.



+IKA Die Brücke als Interceptor



**Alle Formen von Adaptionsmöglichkeiten sind umsetzbar.
Die Adaptionismechanismen werden in den kooperierenden Agenten
implementiert.**

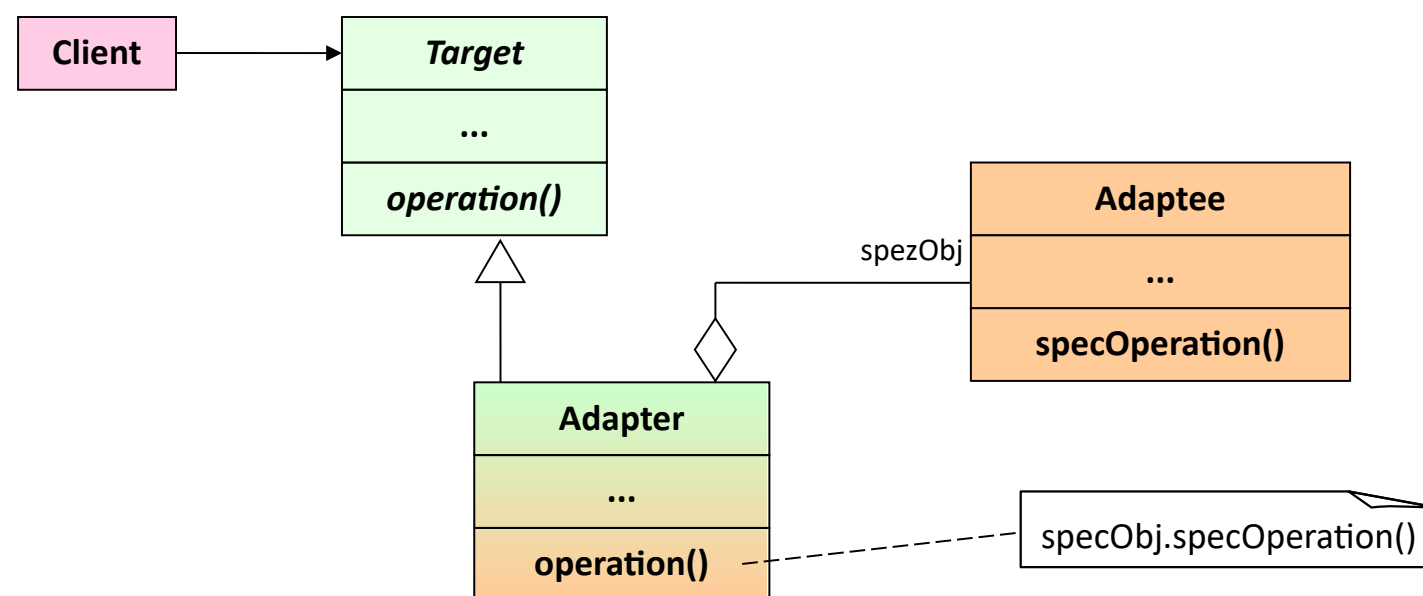
Zweck:

Anpassung einer bestehenden Schnittstelle (einer Komponente) an die Bedürfnisse eines Klienten. Damit können Komponenten zusammenarbeiten, die wegen inkompatibler Schnittstellen dazu nicht in der Lage wären.

Man verwendet das Adaptermuster, wenn:

- eine existierende Komponente verwendet werden soll, deren Schnittstelle nicht passt,
- eine Komponente zu einem Zeitpunkt erstellt wird, zu dem die übergeordneten Komponenten noch in der Entwicklung sind und die genauen Schnittstellen deshalb undefiniert.
- verschiedene austauschbare Komponenten verwendet werden sollen, es aber unpraktisch ist, eine gemeinsame Schnittstelle zu definieren.

„Objekt-Adapter“



Ein Objekt-Adapter verwendet Komposition und Delegation.

**Ganze Komponenten können so aneinander
angepasst werden.**

- **Adapter können sehr komplex werden:**
 - einfaches Schnittstellen-Rewriting oder
 - die Erstellung völlig neuer Operationen (**insertNewImpl**)
- **Zweiweg-Adapter:**

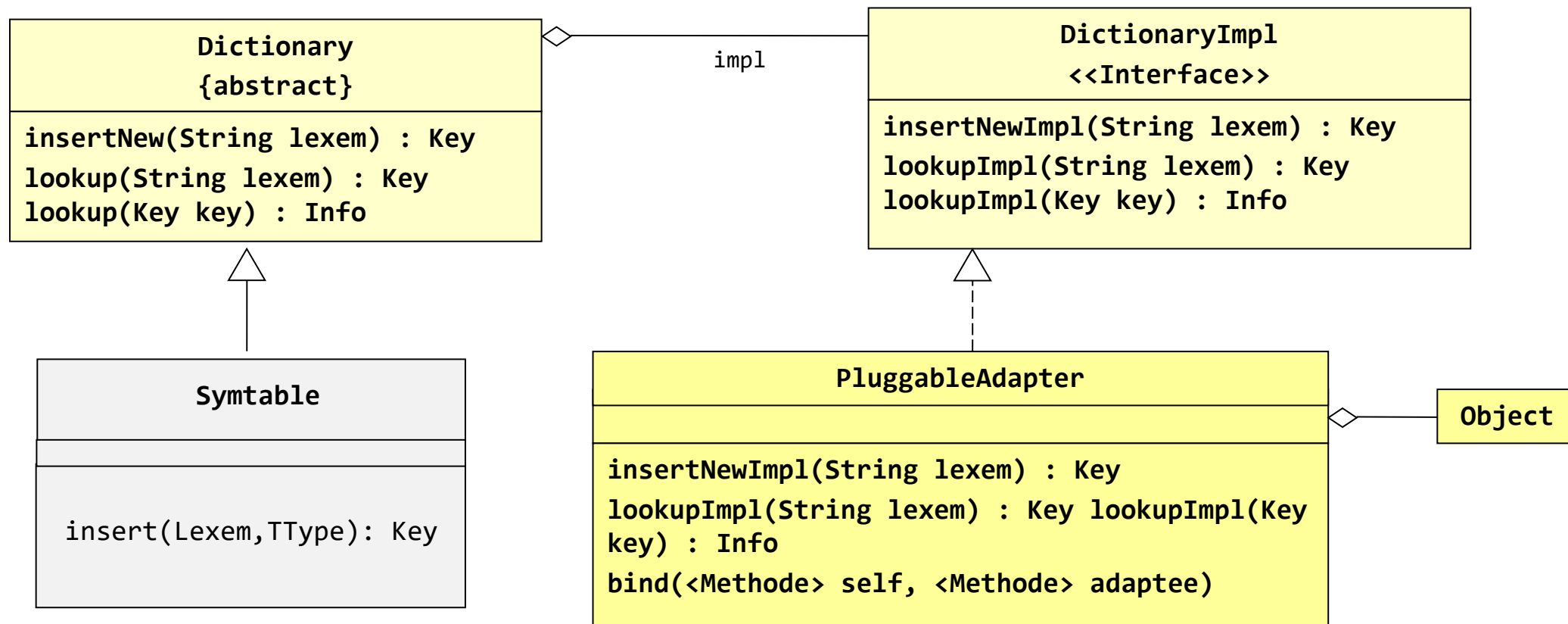
Zur Steigerung der Transparenz können auch beide Schnittstellen angeboten werden (*Ziel und die* adaptierte Komponente)
- **Pluggable-Adapter:**

Komponenten können leichter wiederverwendet werden, wenn sie von Anfang an adaptierbar sind. Dies ist beispielsweise der Fall, wenn generische Adapter zur Verfügung stehen, die eine Schnittstelle auf eine oder mehrere andere abbilden.

+IKA Beispiel: Pluggable-Adapter

Die Idee, die Realisierung der Klasse DictionaryImpl auf eine bestehende Hash-Tabellen-Implementierung zurückzugreifen, ist naheliegend.

Noch mehr Flexibilität bietet nur ein Pluggable-Adapter.



+KA Beispiel: Pluggable-Adapter

```
class PluggableAdapter implements DictionaryImpl{
    private Object adaptee;
    private Class adapteeType;
    private Hashtable<String,String> mapping = new Hashtable<>();

    public PluggableAdapter(Object obj){
        this.adaptee = obj; this.adapteeType = obj.getClass();}

    public Key insertNewImpl(String lexem){
        String adapteeMethod = this.mapping.get("insertNewImplString");
        if (adapteeMethod != null)
            return (Key) this.adapteeType.getMethod(adapteeMethod,String.class).
                invoke(this.adaptee, lexem);
        return null;}

    public void bind(String targetMethod,String adapteeMethod,String signature){
        if (this.mapping.containsKey(targetMethod+signature))
            return;
        this.mapping.put(targetMethod+signature, adapteeMethod);}
}
```

+IKA Patterns für den Persistence Layer

[Fowler, Martin. Patterns of Enterprise Application Architecture. Addison-Wesley, 2003.]

Es gilt den Zugriff auf die Datenbank zu organisieren und mit den Anforderungen an Performance, Flexibilität, Skalierbarkeit etc. in Einklang zu bringen.